UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA ANTIFRAUDE EN REDES DE TELECOMUNICACIÓN BASADO EN LA ESPECIFICACIÓN DE API OPEN GATEWAY

ALEJANDRO PALMIER GÓMEZ

2024

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA ANTIFRAUDE EN REDES DE TELECOMUNICACIÓN BASADO EN LA ESPECIFICACIÓN DE API OPEN GATEWAY

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS TELEMÁTICOS

Autor
ALEJANDRO PALMIER GÓMEZ

Tutor
JAVIER CONDE DÍAZ

2024

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN
TRABAJO FIN DE MÁSTER

Título: Diseño e implementación de un sistema antifraude en redes de telecomunicación basado en la especificación de API Open Gateway

Autor: D. Alejandro Palmier Gómez

Tutor: D. Javier Conde Díaz

Departamento: Departamento de Ingeniería de Sistemas Telemáticos

MIEMBROS DEL TRIBUNAL

Presidente: D .....
Vocal: D .....
Secretario: D .....
Suplente: D .....

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

Madrid, a _____ de _____ de _____

Resumen

Ante un mercado estático y saturado, los operadores de telecomunicaciones están buscando nuevas fuentes de ingresos que les permitan obtener mayores ganancias. Es aquí donde surge el proyecto *Open Gateway*. *Open Gateway* tiene como objetivo la exposición de las capacidades de red a través de API (*Application Programming Interface*) estandarizadas.

El desarrollo de este TFM (Trabajo Fin de Máster), tiene como finalidad diseñar e implementar una arquitectura que permita a un operador de telecomunicaciones, crear, gestionar, e integrar en sus sistemas las API *Open Gateway* de una manera escalable y sencilla. El diseño incluye la definición de una base de clientes adecuada para los requisitos del proyecto, un diseño basado en arquitecturas de microservicios, la centralización del punto de acceso mediante un API GW (API *Gateway*), mecanismos de seguridad a través de IAM (*Identity and Access Management*), integración con los sistemas de gestión de fallos y calidad y la automatización del ciclo de pruebas y despliegue.

La implementación de la arquitectura, se basa en un *stack* que incluye diversas tecnologías y en el desarrollo de tres API *Open Gateway*. Las tres API serán *Know Your Customer*, *Sim Swap* y *Device Status*. En el *stack* de tecnología, podemos encontrar *Spring Boot* para el desarrollo de las API, *Maven* como herramienta para la gestión de dependencias, *Mongo DB* como base de datos para almacenar la información de los clientes, *HTML*, *Javascript* y *CSS* para la creación de un portal web de administración, *Keycloak* como IAM para cumplir con los flujos de seguridad, *Kong* para el desarrollo del API GW, *Grafana* y *Prometheus* para la monitorización de métricas, *Logstash*, *Elasticsearch* y *Kibana* para la centralización y visualización de *logs*, *Postman* con el objetivo de automatizar las pruebas de las API, *Jenkins* como herramienta para automatizar las pruebas y el despliegue, *Docker* y *Kubernetes* para la contenerización y despliegue de la solución.

Una vez se tienen las API desplegadas, se ha diseñado e implementado una aplicación *Android* para demostrar un posible caso de uso antifraude gracias a las API de *Open Gateway*. La aplicación consistirá en la simulación de la aplicación de un banco. Las API de *Open Gateway*, van a permitir detectar si un usuario se está registrando con datos incorrectos y si se están haciendo transferencias fraudulentas. El diseño se realiza en *Figma* y se implementa con *Android Studio*.

Palabras clave: API, *Open Gateway*, microservicio, *Android*, *Kubernetes*.

Abstract

In a static and saturated market, telecommunications operators are looking for new revenue sources that can provide higher profits. This is where the Open Gateway project comes into play. Open Gateway aims to expose network capabilities through standardized APIs.

The development of this Master's Thesis aims to design and implement an architecture that allows a telecommunications operator to create, manage, and integrate Open Gateway APIs into their systems in a scalable and straightforward manner. The design includes defining a suitable customer base for the project requirements, a design based on microservices architectures, centralization of the access point through an API GW, security mechanisms through IAM, integration with fault management and quality systems, and the automation of the testing and deployment cycle.

The implementation of the architecture is based on a stack that includes various technologies and the development of three Open Gateway APIs. The three APIs will be Know Your Customer, Sim Swap, and Device Status. In the technology stack, we can find Spring Boot for API development, Maven as a tool for dependency management, Mongo DB as a database to store customer information, HTML, JavaScript, and CSS for creating a web administration portal, Keycloak as IAM to comply with security flows, Kong for API GW development, Grafana and Prometheus for metrics monitoring, Logstash, Elasticsearch, and Kibana for log centralization and visualization, Postman to automate API testing, Jenkins as a tool to automate testing and deployment, Docker and Kubernetes for containerization and deployment of the solution.

Lastly, an Android application is designed and implemented to demonstrate a potential anti-fraud use case using the Open Gateway APIs. The application will consist of simulating a bank's app. The design is done in Figma and implemented with Android Studio.

Keywords: API, *Open Gateway*, microservicio, *Android*, *Kubernetes*.

Agradecimientos

A Javier, por su comprensión, trabajo, crítica y entrega que han hecho que mejore en mis habilidades como ingeniero.

A mis compañeros de Telefónica. Gracias a Aure, Alex y David por darme la oportunidad de trabajar en un proyecto tan fascinante como *Open Gateway* y por permitirme compatibilizar los estudios con el trabajo.

A mi familia y amigos que siempre me han apoyado.

Índice general

Resumen III

Abstract V

Agradecimientos VII

Índice General IX

Índice de Figuras XI

Índice de Tablas XIII

Lista de Acrónimos XV

1. Introducción **1**
1.1. La evolución de las comunicaciones móviles a redes más abiertas 1
1.2. Objetivos 2

2. Estado del arte **4**
2.1. Introducción de los dominios de una red de telecomunicaciones (Acceso móvil, CORE móvil, Redes de Transporte, Sistemas IT, Operaciones, Seguridad y Privacidad). 4
2.1.1. El estándar de las comunicaciones móviles 4
2.1.2. Red de acceso móvil 5
2.1.3. CORE de la red móvil 8
2.1.4. Red de transporte 13
2.1.5. Modelo operativo y gestión de redes 15
2.1.6. Sistemas IT 17
2.1.7. Seguridad y Privacidad 20
2.2. Evolución del CORE 5G a la exposición de servicios. 22
2.3. El proyecto *Open Gateway*. 26

3. Diseño e Implementación de arquitectura de microservicios para la exposición de API en *Open Gateway*. **31**
3.1. Diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. 31
3.1.1. Diseño basado en una arquitectura centralizada. 31
3.1.2. Arquitectura de microservicios frente a arquitecturas monolíticas. 34

3.1.3. Seguridad en la exposición de API. 37
3.1.4. El rol del API GW en arquitecturas de microservicios. 40
3.1.5. Sistema de monitorización para la gestión de *Performance Management*. 42
3.1.6. Centralización de logs para la gestión de *Fault Management*. . . 43
3.1.7. Resumen del diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. 44
3.2. Implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. 45
3.2.1. Implementación de la base de datos *Open Gateway*. 45
3.2.2. Implementación API de provisión y gestión de usuarios. 52
3.2.3. Implementación del API *Open Gateway Sim Swap*. 58
3.2.4. Implementación del API *Open Gateway Device Status*. 60
3.2.5. Implementación del API *Open Gateway Know Your Customer*. 63
3.2.6. Implementación del servidor de autorización con *Keycloak*. . . . 66
3.2.7. Implementación del API GW con *Kong* y *Konga*. 70
3.2.8. Implementación de portal web para gestión de base de clientes y entorno de pruebas. 72
3.2.9. Implementación del sistema de monitorización con Prometheus y Grafana. 75
3.2.10. Implementación del sistema de centralización y visualización de *logs* con *Logstash*, *Elasticsearch* y *Kibana*. 78
3.2.11. Despliegue de la solución con *Kubernetes*. 80
3.2.12. Automatización del flujo CI/CD con *Postman* y *Jenkins*. 85
3.2.13. Resumen de la implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. . . 87

4. Diseño e Implementación de la aplicación antifraude. 89
4.1. Aplicación *Sandbank*. 89
4.2. Diseño de Sandbank. 91
4.3. Implementación de *Sandbank* con *Android Studio* 96
4.4. Pruebas de la aplicación. 98

5. Conclusiones y trabajos futuros 102

Bibliografía 105

A. Aspectos éticos, económicos, sociales y ambientales 110
A.1. Introducción 110
A.2. Descripción de impactos relevantes relacionados con el proyecto 110
A.3. Conclusiones 111

B. Presupuesto económico 112

Índice de figuras

2.1. Arquitectura RAN 5G [1]. 6
2.2. Evolución de RAN 4G a 5G siguiendo los principios de *Open Ran [2]*. . 8
2.3. Arquitectura 5G [1]. 8
2.4. Arquitectura 5G NSA [3]. 12
2.5. Arquitectura 5G SA [3]. 13
2.6. Arquitectura de *switch* clásico frente a arquitectura con SDN [4]. . . . 14
2.7. Arquitectura NFV [5]. 15
2.8. Arquitectura SNMP [6]. 17
2.9. Arquitectura BSS [7]. 19
2.10. Evolución de BSS y OSS hacia ODA según *TM Forum* [8]. 20
2.11. Modelo de operación y gestión de la seguridad [9]. 21
2.12. Diagrama de flujo para eventos originados en el AMF [10]. 24
2.13. Diagrama de flujo para eventos originados en el UDM [10]. 25
2.14. Nuevo Flujo de QoS [10]. 26
2.15. Modificación de flujo de QoS existente [10]. 26
2.16. Modelo de referencia de CAMARA para el proyecto *Open Gateway* [11]. 28

3.1. Arquitectura descentralizada para consulta de datos. 32
3.2. Arquitectura descentralizada para consulta de datos. 33
3.3. Diseño basado en arquitectura monolítica. 35
3.4. Diseño basado en arquitectura de microservicios. 36
3.5. Flujo OAuth 2.0. 37
3.6. Diseño basado en arquitectura de microservicios con servidor de autorización. 39
3.7. Diagrama secuencial UML del Diseño basado en arquitectura de microservicios con servidor de autorización. 39
3.8. Diseño basado en arquitectura de microservicios con servidor de autorización y API GW. 41
3.9. Diagrama secuencial UML del Diseño basado en arquitectura de microservicios con servidor de autorización y API Gateway. 41
3.10. Diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. 44
3.11. Formato del IMSI. 48
3.12. Captura de Mongo DB. 51
3.13. Modelo de arquitectura para implementación de las API. 53
3.14. Base de datos *Redis* para almacenar MCC y los países asociados. . . . 63
3.15. Consola de administración de *Keycloak*. 68

3.16. Servicios creados en *Kong*. 71
3.17. Diseño basado en arquitectura de microservicios con servidor de autorización y API GW como punto único de entrada. 72
3.18. Menú principal de la web de administración. 73
3.19. Consulta de usuarios a través del portal web. 74
3.20. Prueba de API *Sim Swap* a través de la web. 75
3.21. Ejemplo de *endpoints* expuestos por *actuator*. 76
3.22. Dashboard de Grafana para el API *Sim Swap*. 77
3.23. Visualización de los *logs* de las API a través de *Kibana*. 80
3.24. Arquitectura de recolección y centralización de *logs*. 83
3.25. CI/CD *pipeline*. 86
3.26. Interfaz de *Jenkins* tras el resultado de la automatización. 86
3.27. Esquema de la implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*. 87

4.1. Arquitectura de integración entre *Sandbank* y API *CAMARA*. 91
4.2. Página de Bienvenida de *Sandbank*. 92
4.3. Página de inicio de sesión de *Sandbank*. 93
4.4. Página de creación de cuenta de *Sandbank*. 93
4.5. Página de error en la creación de cuenta de *Sandbank*. 94
4.6. Vista del menú principal de *Sandbank*. 94
4.7. Página de lanzamiento de transferencia de *Sandbank*. 94
4.8. Página de perfil de usuario de *Sandbank*. 94
4.9. Página de error al realizar una transferencia en *Sandbank*. 95
4.10. Página de actualización de datos de cliente en *Sandbank*. 95
4.11. Prueba de la ventana de registro de *Sandbank*. 99
4.12. Prueba de la ventana de transferencia de *Sandbank*. 100
4.13. Prueba del menú principal de *Sandbank* tras transferencia. 100
4.14. Prueba del menú de actualización de datos en *Sandbank*. 100

Índice de tablas

2.1. Equivalencia de los elemento de CORE 4G a 5G 9
2.2. Eventos monitorizables que se pueden notificar a través de NEF 24

3.1. *Endpoints* del API de provisión. 54
3.2. *Endpoints* del API de *Sim Swap*. 58
3.3. Tabla resumen de los *Pods* desplegados. 84
3.4. Tabla resumen de los objetivos, funcionalidades y herramientas empleadas. 88

B.1. Costes de personal. 112
B.2. Costes de recursos materiales. 113
B.3. Costes totales. 113

Lista de Acrónimos

OFDM: Orthogonal Frequency Division Multiplexing.

MIMO: Multiple Input Multiple Output.

NAAS: Network as a Service.

OGW: Open Gateway.

IAM: Application Programming Interface.

MSISDN: Mobile Station Integrated Services Digital Network.

IMSI: International Mobile Subscriber Identity.

IMEI: Mobile Equipment Identity.

KYC: Know Your Customer.

FW: Framework.

MS: Microservicio.

IAM: Identity and Access Management.

API GW: Application Programming Interface Gateway.

K8: Kubernetes.

API: Application Programming Interface.

SA: Standalone.

NF: Network Function.

AF: Application Function.

NEF: Network Exposure Function.

AMF: Access and Mobility Management Function.

PCF: Policy Control Function.

NSSF: Network Slice Selection Function.

AUSF: Authentication Server Function.

SMF: Session Management Function.

UPF: User Plane Function.

NWDAF: Network Data Analytics Function.

MME: Mobility Management Entity.

PCRF: Policy and Charging Rules Function.

HSS: Home Subscriber Server.

UDM: Unified Data Management.

UDR: Unified Data Repository.

EPC: Evolved Packet Core.

UL: Uplink.

DL: Downlink.

PDU: Packet Data Unit.

QoS: Quality of Service.

3GPP: 3rd Generation Partnership Project.

TS: Technical Specification.

GSM: Global system for mobile communication.

UMTS: Universal Mobile Telecommunications System.

LTE: Long Term Evolution.

RAN: Radio Access Network.

COTS: Commercial Off The Shelf.

RRC: Radio Resource Control.

RLC: Radio Link Control.

RRU: Radio Resource Unit.

BBU: Base Band Unit.

gNB: g-Node B.

UE: User Equipment.

NAS: Non Access Stratum.

AS: Access Stratum.

MICO: Mobile Initiated Connection Only.

RAT: Radio Access Technology.

SCP: Service Control Proxy.

ONF: Open Networking Foundation.

NFV: Network Functions Virtualization.

NFVI: Network Functions Virtualization Infrastructure.

SDN: Software Defined Networks.

VNF: Virtualized Network Functions.

EM: Element Management.

OSS: Operations Support Systems.

BSS: Business support systems.

FCAPS: Fault, Configuration, Accounting, Performance and Security.

ISO: International Organization for Standardization.

SNMP: Simple Network Management Protocol.

CRM: Customer Relationship Management.

B2B: Business to Business.

B2C: Business to Consumer.

ODA: Open Digital Architecture.

TMF: TM Forum.

CSP: Communication Service Provider.

DNI: Documento Nacional de Identidad.

SOC: Security Operations Center.

SIEM: Security Information and Event Management.

GDPR: The General Data Protection Regulation.

OTT: Over The Top.

ARPU: Average Revenue Per Use.

MWC: Mobile World Congress.

GSMA: Global System for Mobile Communications.

QoD: Quality On Demand.

QCI: Quality of Service Class Identifier.

IETF: Internet Engineering Task Force.

DDoS: Distributed Denial of Service (DDoS).

SQL: Structured Query Language.

NoSQL: No Structured Query Language.

MCC: Mobile Country Code.

MNC: Mobile Network Code.

ITU: International Telecommunication Union.

ELK: Elasticsearch, Logstash, Kibana.

PromQL: Prometheus Query Language.

Capítulo 1

Introducción

1.1. La evolución de las comunicaciones móviles a redes más abiertas

Durante los últimos 50 años, el mundo de las telecomunicaciones móviles ha cambiado drásticamente. Sobre 1980, se desarrolló la primera generación de comunicaciones móviles que estaba centrada en dar servicios de voz a través de la transmisión analógica. El sector fue evolucionando rápidamente y en los años 90 se introdujo la segunda generación, que incluía como novedad la transmisión digital y servicios de datos limitados [12].

Aunque con la segunda generación ya se disponía de los datos móviles que se conocen hoy en día, no fue hasta la tercera generación (3G) a principios de la década de los 2000 que se dio el verdadero impulso a la banda ancha de alta calidad. Este salto cualitativo, permitió introducir nuevos servicios como las videoconferencias [12].

A finales de la década de los 2000, se empezó a utilizar la cuarta generación de las comunicaciones móviles (4G). Esta nueva evolución tenía como objetivo buscar una mayor eficiencia y aumentar las velocidades de datos alcanzables. Para lograrlo, se introdujeron nuevas tecnologías como la transmisión basada en OFDM (*Orthogonal Frequency Division Multiplexing*) y MIMO (*Multiple Input Multiple Output*) [13] [14].

Las velocidades y prestaciones de la cuarta generación parecen suficientes para el uso cotidiano de una persona que utiliza el dispositivo móvil con moderada frecuencia. Esto supone un desafío para los operadores de telecomunicación que buscan nuevas maneras de crear servicios y generar nuevas fuentes de ingresos. Por este motivo, la quinta generación de redes móviles (5G), que se empieza a desarrollar alrededor del año 2012, ofrece nuevas capacidades y mejoras que permiten pensar en nuevos casos de uso [15].

Ejemplo de estos nuevos escenarios son los casos de uso para internet de las cosas (IoT), control de misión crítica y banda ancha de móvil mejorada. El internet de las cosas, se refiere a la conectividad de objetos a través de aplicaciones máquina a máquina (M2M). El 5G, permite desarrollar este área gracias a una mejora de

la eficiencia energética, consiguiendo grandes ahorros de batería que permiten a los dispositivos IoT durar entre 5 y 10 años. Con respecto a los casos de uso de control de misión crítica, se encuentran algunos como la conducción autónoma o servicios de telemedicina. Para lograr esto, el 5G es capaz de reducir el tiempo total de la comunicación, también conocido como latencia, por debajo de 1 ms. Por último, la banda ancha de móvil mejorada, logra velocidades de hasta 20 Gbps, permitiendo desarrollar nuevas tecnologías como el metaverso [16].

Aunque ya ha quedado claro la revolución que pretende ser el 5G, hay algunas otras de sus características que suponen un gran cambio con respecto a la generación anterior y que abren nuevas posibilidades. En primer lugar, en esta generación se hace especial énfasis en el desacoplamiento y virtualización de las funciones de red. Esto permite entre otras cosas realizar despliegues dinámicos y en entornos cloud desligando el software de la infraestructura [13].

Por otra parte, el 5G *standalone* (SA) introduce un nuevo núcleo de redes móviles (CORE) en el que la comunicación entre las distintas funciones de red (NF), son estándares y abiertas. Además, se introduce una nueva función de red llamada *Network Exposure Function* (NEF) que abre las capacidades del CORE a terceros. Esto introduce la posibilidad de utilizar la información que gestiona el núcleo de red (gestión de sesiones, localización del usuario, enrutamiento de sesiones, etc) para el desarrollo y exposición de servicios [17].

Durante este trabajo, se indagará en las propiedades de la red 5G que facilitan e introducen el proyecto *Open Gateway*. *Open Gateway* es un proyecto que va a permitir a las operadoras de telecomunicaciones, exponer sus capacidades de forma estandarizada a terceros.

1.2. Objetivos

El objetivo de este TFM es desarrollar una aplicación móvil que ilustre cómo pueden ser de utilidad las API de *Open Gateway* en casos de uso de antifraude. Para ello, se va a dividir el trabajo en tres fases principales. En la primera fase, se diseña un entorno de exposición de API a través de microservicios que permita tener control de acceso, monitorización de los recursos utilizados por la infraestructura y registro de quién accede a qué API y cuándo. El objetivo es tener un escenario lo más realista posible, representando las necesidades que podría tener una operadora de telecomunicaciones.

En una segunda fase, se implementa la arquitectura diseñada previamente mediante tecnologías *opensource*. Se eligen tecnologías que ofrezcan buen rendimiento y capacidades sin coste alguno y que puedan ser desplegadas con *Kubernetes* o *Docker*. Esto último permite el despliegue en múltiples entornos, aprovechando las ventajas de resiliencia y escalabilidad.

En la última fase del proyecto, se diseña e implementa una aplicación *Android* que integre las API de *Open Gateway* para detectar transacciones o registros fraudulentos.

Capítulo 2

Estado del arte

2.1. Introducción de los dominios de una red de telecomunicaciones (Acceso móvil, CORE móvil, Redes de Transporte, Sistemas IT, Operaciones, Seguridad y Privacidad).

En esta sección se tratarán los aspectos más relevantes de una operadora de telecomunicación y se enfocará en los características que facilitan la exposición de servicios así como los impactos e implicaciones que tiene.

Se va a comenzar haciendo una breve introducción de algunos de los dominios más relevantes que tiene una operadora de telecomunicaciones así como los campos más innovadores de cada uno de los dominios.

2.1.1. El estándar de las comunicaciones móviles

Si se habla del estándar de las comunicaciones móviles, hay que referirse al 3GPP (3rd Generation Partnership Project). El 3GPP es una asociación de estándares globales que reúne a organizaciones de la industria de las telecomunicaciones para desarrollar y especificar estándares para sistemas de comunicaciones móviles, incluyendo 2G, 3G, 4G, 5G y las futuras generaciones que vengan. Fue fundado en 1998 para garantizar la interoperabilidad y la compatibilidad de los sistemas de comunicaciones móviles en todo el mundo. Para la mayoría de referencias relacionadas con esta sección, se utilizará el estándar como fuente de información [18].

El 3GPP se compone de varias organizaciones de telecomunicaciones líderes de todo el mundo, incluyendo operadores de redes móviles, fabricantes de equipos de telecomunicaciones, proveedores de servicios, institutos de investigación y otros interesados en el desarrollo de estándares de telecomunicaciones. Su objetivo principal es desarrollar y mantener especificaciones técnicas para los sistemas de comunicaciones móviles, incluyendo la arquitectura de red, los protocolos de comunicación, las interfaces de usuario, los requisitos de seguridad y mucho más [19] [20].

El trabajo del 3GPP está organizado en grupos de trabajo técnicos (TSG, por sus siglas en inglés) que se centran en áreas específicas de la tecnología de las comunicaciones móviles, como el acceso radioeléctrico, el CORE de red, la seguridad, la calidad de servicio y otros aspectos técnicos. Estos grupos de trabajo colaboran para desarrollar y mantener las especificaciones técnicas que definen los sistemas de comunicaciones móviles y garantizan su interoperabilidad a nivel mundial [20].

Junto con las *releases*, se publican las especificaciones técnicas (TS, *Technical Specifications*) que detallan los estándares y las normas para implementar las diferentes características y funcionalidades en las redes de telecomunicaciones móviles. Las especificaciones técnicas definen cómo se deben implementar y operar los sistemas móviles para garantizar la interoperabilidad entre los equipos de diferentes fabricantes y proporcionar una experiencia uniforme para los usuarios finales.

El 3GPP ha sido fundamental en el desarrollo y la estandarización de tecnologías de comunicaciones móviles como GSM, UMTS, LTE y 5G, y continúa desempeñando un papel crucial en la evolución y el desarrollo de futuras generaciones de tecnología móvil. Sus estándares son ampliamente adoptados por la industria de las telecomunicaciones y proporcionan la base para la implementación y el despliegue de sistemas de comunicaciones móviles en todo el mundo.

2.1.2. Red de acceso móvil

El dominio de acceso móvil esta formado por las redes de radio y las estaciones base que permiten la conexión inalámbrica de los dispositivos móviles a la infraestructura de la red de telecomunicaciones. Este dominio es esencial debido a que proporciona la conectividad para los servicios de voz y datos móviles, y su actividad afecta directamente a la experiencia del usuario final. Con la evolución hacia tecnologías como 5G, el acceso móvil ha experimentado una transformación significativa para ofrecer velocidades de datos más rápidas, menor latencia y capacidad mejorada para admitir un amplio abanico de aplicaciones y servicios tal y como se ha comentado en la introducción.

La estandarización del 5G se comienza a definir en la *Release* 15 (2018) NR non-*standalone*, NR *standalone* y en la *Release* 16 Mejoras en MIMO; espectro sin licencia del 3GPP. La red de acceso radio (NG-RAN) está formada por un solo tipo de nodo: la estación base o "g-Nodo B" (*g-Node B*; gNB). Por su parte, la estación base se puede dividir en la unidad central (gBN-CU) y las unidades remotas (gNB-DU). Para los que están más familiarizados con los términos de 4G, estos componentes extienden la funcionalidad que tenía el BBU (*Base Band Unit*). Al igual que en 4G, el 5G continua con una arquitectura RAN (*Radio Access Network*) en la que se desagrega el RRU *(Radio Resource Unit*) de la estación base, de manera que, distintas RRU pueden estar conectadas a la misma estación base. Esto supuso un gran ahorro en términos de coste para las operadoras de telecomunicaciones.

En el caso de 5G, la unidad central se encarga de las capas de control superiores de la comunicación inalámbrica. Esto incluye la gestión del protocolo de control de radio (RRC, *Radio Resource Control*), que es responsable de establecer y mantener la conexión entre el dispositivo móvil y la red, así como otras funciones de control de la interfaz de radio. Por otra parte, las unidades remotas se encargan de las capas inferiores de la comunicación inalámbrica como por ejemplo, el procesamiento de las señales de radio a nivel físico (PHY), control de acceso al medio (MAC, *Medium Access Control*), control de enlace de radio (RLC, *Radio Link Control*). También se encarga de la modulación, la codificación y la decodificación de las señales de radio, así como del control de potencia y la sincronización [1].

La arquitectura se representa en la siguiente imagen:

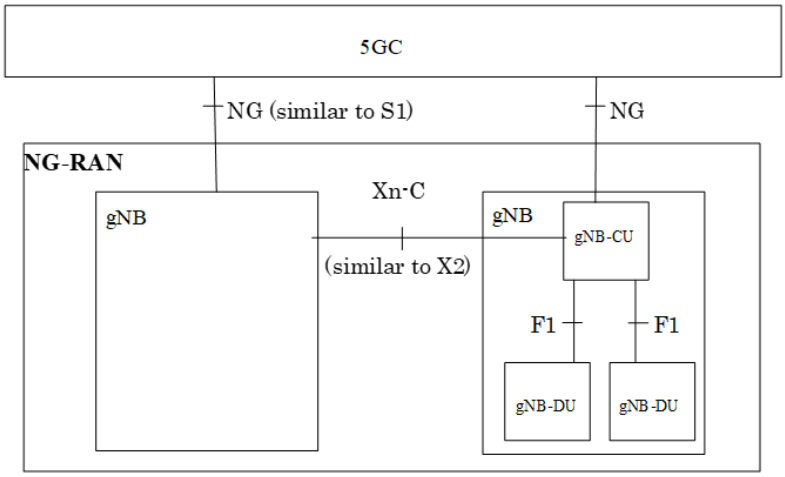


Figura 2.1: Arquitectura RAN 5G [1].

El 5G trae también consigo una serie de innovaciones y avances. Las velocidades de transmisión alcanzan hasta 20 Gb/s para la bajada y 10 Gb/s para la subida , a parte de una eficiencia espectral de 30 b/s/Hz de bajada. El 5G se muestra claramente por encima de lo que se tenía en 4G. Además, es capaz de reducir drásticamente la latencia, llegando a milisegundos e incluso menos, lo que es muy importante para las aplicaciones ya comentadas [21].

El 5G también destaca en la diversidad de usos gracias a sus mejoradas tecnologías de acceso por radio y una gestión más eficiente de recursos y conectividad. Utiliza frecuencias de hasta 53 GHz y técnicas avanzadas como el MIMO masivo y el *beamforming* para mejorar la cobertura y la calidad de la señal.

El *beamforming* hace referencia a una técnica que consiste en dirigir la transmisión y recepción de señales de radio a través de antenas de forma selectiva. Mediante el uso de múltiples antenas, el *beamforming* mejora la calidad de la señal y la eficiencia

energética al enfocar la energía de la señal directamente hacia los dispositivos de usuario, en lugar de dispersarla en todas direcciones. Esto aumenta la intensidad de la señal en las áreas deseadas, y también reduce la interferencia en las que no lo son, lo que resulta en conexiones más rápidas y estables [22].

Por otro lado, el MIMO masivo lleva la idea de múltiples antenas un paso más allá al utilizar decenas de antenas en una única estación base para manejar simultáneamente múltiples señales de entrada y salida. Esto permite que las redes manejen un volumen de datos mucho mayor, mejorando la capacidad y la eficiencia de la red al mismo tiempo que se reducen los errores y se aumenta la velocidad de la red [23].

Por último, se va a explicar brevemente acerca de *Open Ran* ya que es la evolución a la que cada vez más están tendiendo las arquitecturas de acceso radio. Tradicionalmente, todos los componentes de la RAN han estado formados por un único proveedor (*Ericsson*, *Nokia*, *Huawei*, *Mavenir*, etc) que ofrecían interfaces propietarias cerradas. Este hecho provoca que la compra de distintos componentes a distintos fabricantes sea inviable debido a que la integración de los mismos sería muy costoso. El resultado son redes con un cierto grado de *vendor lock-in* (*vendor lock-in* hace referencia a una situación en la que un cliente está atado a un cierto proveedor ya que le es muy difícil cambiarlo) [24].

Es aquí donde surge el *Open Ran*. El *Open Ran* intenta conseguir precisamente esto, redes de acceso móvil abiertas en la que los componentes puedan ser de diferente proveedor. Para ello, se basa en tres principios. Estos principios son interfaces abiertas, COTS (*Commercial Off-The-Shelf*) como hardware, virtualización y funcionalidades definidas por software.

El principio de interfaces abiertas se refiere a la estandarización de las interfaces entre los diferentes componentes de la red de acceso radio. Muchas de las interfaces, se definían ya en el 3GPP, sin embargo, existen otras interfaces que no están definidas o que el estándar da varias opciones. Esto implica que cada proveedor hace una personalización a su gusto. Al estandarizar estas interfaces, se permite la correcta integración entre cualquier componente independientemente del suministrador [2].

Con respecto a COTS como hardware, la idea consiste en que se utilice hardware de propósito general para los componentes de la RAN en contrapartida del hardware específico de los suministradores. Este principio encaja perfectamente con el de la virtualización y funcionalidades definidas por software. En este último principio, se busca desagregar el software de las estaciones base del hardware en el que se despliega. Esto permite redes más flexibles y escalables que reaccionan de mejor manera a posibles cambios de configuración. Se indagará más en estos conceptos en la sección de redes de transporte. Estos principios, en la arquitectura de la RAN, se representan en la siguiente figura:

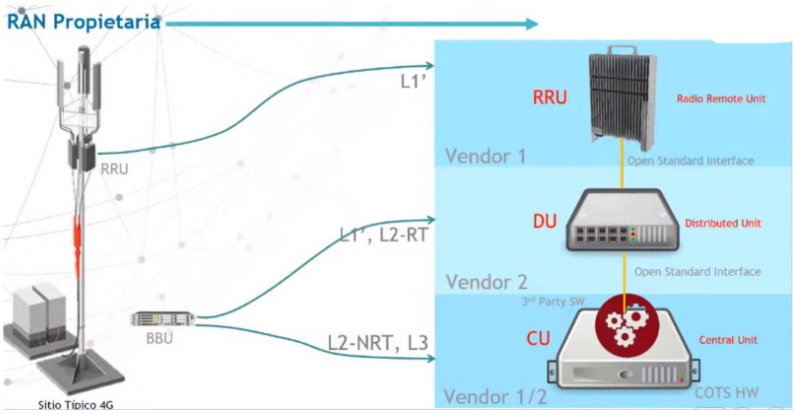


Figura 2.2: Evolución de RAN 4G a 5G siguiendo los principios de *Open Ran [2].*

2.1.3. CORE de la red móvil

En esta sección, se va a introducir el CORE móvil de la quinta generación, también conocido como CORE de la red 5G *Standalone*. El CORE de una red móvil, también conocido como núcleo de red, desempeña un papel crucial en la gestión de las comunicaciones de datos y voz a través de toda la red de telecomunicaciones. Actúa como el cerebro central de la red, donde se procesan y enrutan todas las llamadas y conexiones de datos desde y hacia los usuarios móviles. Esto incluye la autenticación y autorización de los usuarios, la gestión de la movilidad de los dispositivos a medida que se desplazan entre diferentes áreas de cobertura, el manejo del intercambio de datos con otras redes, como Internet o redes fijas, y la implementación de políticas de facturación. Además, el CORE de la red proporciona funciones esenciales de seguridad para proteger tanto la privacidad de los usuarios como la integridad de los datos transmitidos [25].

A continuación, se muestra una imagen de la arquitectura de 5G donde se ilustra el CORE como actor principal.

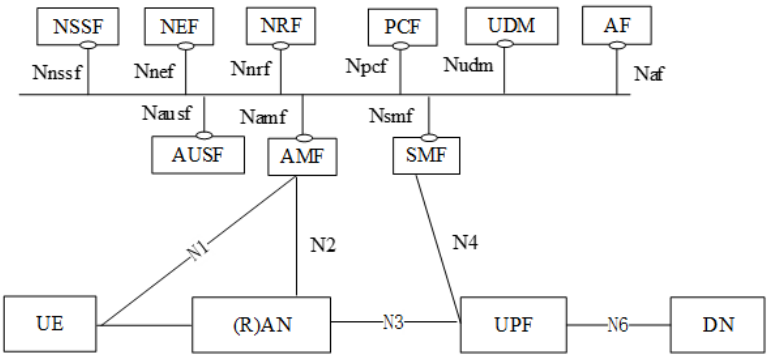


Figura 2.3: Arquitectura 5G [1].

Antes de contar en detalle la arquitectura del nuevo 5G, se va a mostrar a continuación, una tabla de equivalencias con respecto a lo que se tenía en 4G. De manera que aquel que tuviese una idea del funcionamiento de la generación anterior, pueda entender fácilmente la evolución. La tabla es la siguiente:

Elemento de 4G	**Función 4G**	**Mapeo a 5G**
MME	Gestión de movilidad	AMF
MME	Gestión de sesión	SMF
MME	Función de autenticación	AUSF
S-GW	Función del plano de control	SMF
S-GW	Función del plano de usuario	UPF
P-GW	Función del plano de control	SMF
P-GW	Función del plano de usuario	UPF
PCRF	Función de políticas de control	PCF
HSS	Gestión de suscripciones	UDM

Tabla 2.1: Equivalencia de los elemento de CORE 4G a 5G

Por supuesto que esta tabla es una simplificación y que no recoge todos los detalles que tiene la nueva generación. Sin embargo, es una buena aproximación para empezar con el tema.

En la figura 2.3, se muestra el CORE con todas las funciones de red que lo componen (NSSF, NEF, NRF, PCF, UDM, AF, AUSF, AMF, SMF, UPF). A continuación, se explica un poco más en detalle cual es la misión de cada una de estas funciones [1] [10]:

- AMF: La Función de Gestión de Acceso y Movilidad (AMF, por sus siglas en inglés) en el CORE, tiene como objetivo gestionar la movilidad de los dispositivos de usuario (UEs). Entre las principales tareas que realiza el AMF se incluyen: la terminación de la señalización del *Non Access Stratum* (NAS), la seguridad de la señalización NAS, y el control de la seguridad del *Access Stratum* (AS). Además, maneja la señalización entre nodos CN para la movilidad entre redes de acceso 3GPP y la accesibilidad del UE en modo inactivo, lo que incluye el control y ejecución de la retransmisión de *paging*.

  También se encarga de la gestión de áreas de registro, la autenticación y

autorización de acceso, lo cual incluye la verificación de derechos de roaming, y el control de la gestión de movilidad basada en suscripciones y políticas. Por último, ofrece soporte para el *Network Slicing* y la selección del SMF a usar. Estas funcionalidades son cruciales para asegurar una gestión eficiente y segura del tráfico de usuarios y su movilidad en distintas condiciones de red.

- SMF: *Session Management Function* (SMF) trabaja conjuntamente con el AMF para apoyar en los esquemas de gestión de movilidad personalizados, como el "*Mobile Initiated Connection Only*"(MICO) o mejoras en la RAN como el estado "*RRC Inactive*". Entre sus principales funciones se encuentran la gestión de sesiones, la asignación y gestión de direcciones IP de los usuarios (UE), la selección y control de la Función de Plano de Usuario (UPF), y la configuración de la dirección del tráfico en la UPF para enrutamiento hacia el destino apropiado.

  Además, el SMF controla parte de la ejecución de políticas y la calidad de servicio (QoS) y maneja las notificaciones de datos en descarga. Estas tareas son fundamentales para asegurar que las sesiones de datos se manejen de manera eficiente y que el tráfico de red se distribuya y controle adecuadamente, manteniendo la calidad y la seguridad del servicio.

- UPF: La Función del Plano de Usuario (UPF, por sus siglas en inglés) realiza diversas tareas esenciales dentro de la red móvil, actuando como un componente crítico en el manejo del tráfico de datos. Entre sus principales funciones se encuentran: actuar como punto de anclaje para la movilidad Intra-/Inter-RAT (*Radio Access Technology*) cuando es aplicable; servir como el punto de interconexión de sesión PDU externa hacia la Red de Datos; y llevar a cabo el enrutamiento y reenvío de paquetes. Además, el UPF maneja la inspección de paquetes y la parte del plano de usuario de la ejecución de reglas de políticas, así como el reporte del uso del tráfico.

  En términos de calidad de servicio (QoS), el UPF es responsable del manejo de QoS para el plano de usuario, incluyendo filtrado de paquetes, control de acceso (*gating*) y la aplicación de reglas de subida (UL) y bajada (DL). También clasifica el tráfico de subida para apoyar el enrutamiento de flujos de tráfico hacia una red de datos y es el punto de ramificación para soportar sesiones PDU (*Packet Data Unit*) multi-conectadas. Finalmente, verifica el tráfico de subida para asegurar la correspondencia correcta de flujo de QoS a SDF (*Service Data Flow*) y gestiona el almacenamiento temporal de paquetes de bajada y el disparo de notificaciones de datos de bajada. Estas funciones son vitales para garantizar que el tráfico de datos sea manejado de manera eficiente y segura a lo largo de la red.

- PCF: El *Policy Control Function* es el encargado de gestionar el comportamiento de lar red en base a una serie de reglas. Estas reglas se basan en la información de los suscriptores que obtiene de un repositorio centralizado, llamado *Unified Data Repository* (UDR), asegurando que las decisiones sobre el manejo de datos

y prioridades sean justas y eficientes.

Por ejemplo, el PCF puede decidir cómo se gestionan los datos cuando la red está muy ocupada o puede ajustar prioridades para diferentes tipos de tráfico, como vídeos en *streaming* frente a navegación web. También aplica políticas específicas para manejar sesiones de datos, asegurando que todos los datos fluyan donde deben ir según lo planeado. En resumen, el PCF ayuda a que la red 5G funcione de manera óptima, aplicando reglas que mantienen el tráfico organizado.

- NEF: El acrónimo NEF viene de *Network Exposure Function*. En esta sección no se dedicará mucho tiempo a esta función de red ya que contará con la suya propia al ser pieza fundamental en la exposición de funcionalidades 5G. De una manera breve, el NEF permite abrir las capacidades del CORE 5G a terceros de una manera estandarizada.

- NRF: En este caso, el acrónimo hace referencia a *Network Repository Function*. El NRF actúa como un directorio centralizado que facilita la gestión y descubrimiento de los servicios dentro de la red. Ayuda a los diversos componentes de la red a encontrar e interactuar entre sí eficientemente. Este registro almacena perfiles y direcciones de diferentes funciones de la red (NF) y servicios de control proxy de sesión SCP (*Service Control Proxy*), permitiendo que se descubran mutuamente según sea necesario.

  Por ejemplo, si una función de la red necesita comunicarse con otra, puede consultar al NRF para obtener la dirección y los detalles de esa función específica. Además, el NRF mantiene un registro actualizado del estado de salud de estas funciones, asegurando que la red sea robusta y confiable al notificar cambios como registros nuevos, actualizaciones o bajas de componentes. En esencia, el NRF es esencial para mantener organizada y eficiente la comunicación interna dentro de las redes 5G.

- UDM: El UDM (*Unified Data Management*) es una función de red cuya misión principal es centralizar y manejar toda la información de los usuarios y sus suscripciones. En términos simples, el UDM es como el archivo principal de un usuario en la red. Almacena y administra datos esenciales como la identidad del usuario y las credenciales para autenticación segura. Además, gestiona permisos como saber si un usuario puede acceder a la red cuando está en *roaming* o no, y mantiene un registro actualizado de qué servicios específicos de red está utilizando cada usuario. En resumen, el UDM asegura que la información del usuario esté segura, organizada y accesible para otros sistemas de la red que necesitan usarla.

- AUSF: El AUSF (*Authentication Server Function*) en las redes 5G es crucial para garantizar que solo los usuarios autenticados puedan acceder a la red. Básicamente, el AUSF se encarga de la autenticación de los usuarios tanto para el acceso a redes 3GPP (como las redes móviles estándar) como para el acceso a

redes no 3GPP no confiables, es decir, redes que no siguen los estándares 3GPP como podrían ser ciertas redes WiFi o redes privadas. Esto está especificado en la norma técnica TS 33.501.

Además, el AUSF también soporta la autenticación de usuarios para servicios en situaciones de *roaming* durante desastres, asegurando que los usuarios puedan acceder a la red en circunstancias críticas. En resumen, el AUSF actúa como un guardián que verifica las credenciales de los usuarios antes de permitirles el acceso a la red, manteniendo así la seguridad y la integridad del servicio de telecomunicaciones.

- NSSF: El acrónimo de esta función de red hace referencia a *Network Slice Selection Function*. Como su propio nombre indica, esta función de red es clave en *Network Slicing*. El *Network Slicing* permite dividir una única red física en múltiples redes virtuales independientes *slice*. Cada *slice* es una red virtual completa y autónoma que puede ser configurada y optimizada para cumplir con los requisitos específicos de diferentes aplicaciones.El NSSF permite administrar y gestionar la asignación de *slices* de una manera adecuada.

Aunque se han mencionado y explicado brevemente la mayoría de las funciones de red, hay otras que no se han representado ni comentado como por ejemplo el NWDAF (*Network Data Analytics Function*).

Como es evidente, la transformación del 4G al 5G no se hace de manera inmediata reemplazando la RAN y CORE de cuarta generación por el nuevo estándar que hemos visto. El primer paso que han dado los operadores de telecomunicación es hacia lo que se conoce como el *5G non standalone*. La arquitectura se muestra a continuación [3]:

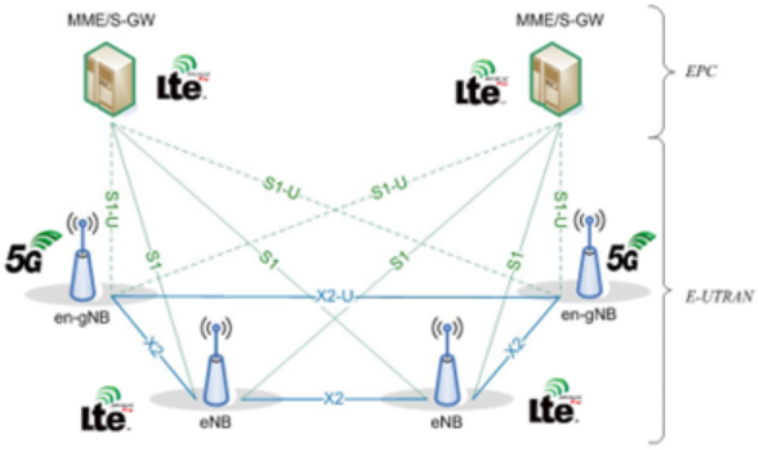


Figura 2.4: Arquitectura 5G NSA [3].

Como se puede observar, el *5G Non Standalone* permite que la RAN de nueva y antigua generación convivan entre sí, manteniendo el CORE de 4G (*Evolved Packet*

Core). Esta evolución es un primer paso pero no es suficiente para disfrutar de todas las capacidades del 5G ya que el CORE sigue siendo el de la generación anterior.

La evolución que incluye tanto la RAN como el CORE de quinta generación se conoce como *5G Standalone*, tal y como se puede observar en la siguiente figura:

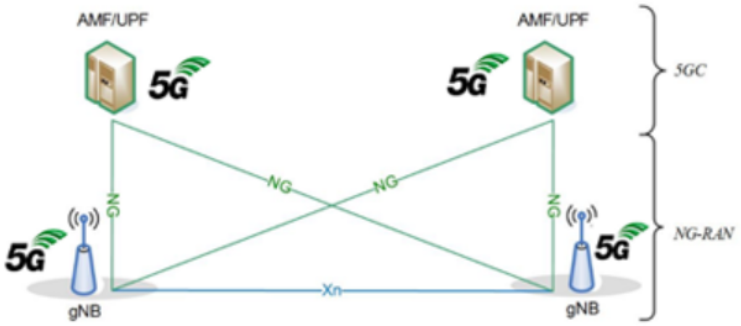


Figura 2.5: Arquitectura 5G SA [3].

2.1.4. Red de transporte

A nivel de red de transporte, esta sección se centrará en el transporte en capas de nivel lógico (IP, Ethernet, TCP, etc) y no se entrará en los mecanismos y tecnologías de transporte de la capa física como la fibra óptica, radio enlaces o la comunicación vía satélite. Dicho esto, se va a repasar y dar una breve explicación de la evolución y tendencia de los últimos años en las redes de transporte.

Tradicionalmente, los *switches* o *routers* de la red de transporte se han implementado como equipos especializados en el que cada fabricante gestionaba su producto. Esto supone una gran barrera de entrada para nuevas empresas que quisiesen entrar en el mercado. Además, implica que las actualizaciones de los equipos son poco flexibles y hay poco margen para incluir nuevas funcionalidades o probar nuevos conceptos, desembocando en un ciclo de innovación muy lento. En este modelo tradicional, cada nodo tiene un plano de gestión, datos y control. A medida que la red va creciendo, se intensifica la necesidad de centralizar el plano de control para hacer una red mucho más eficiente [26].

Es de esta necesidad, que surge lo que se conoce como SDN (*Software Defined Networks*). SDN tiene como objetivo separar el plano de control del plano de datos, tal y como se observa en esta figura:

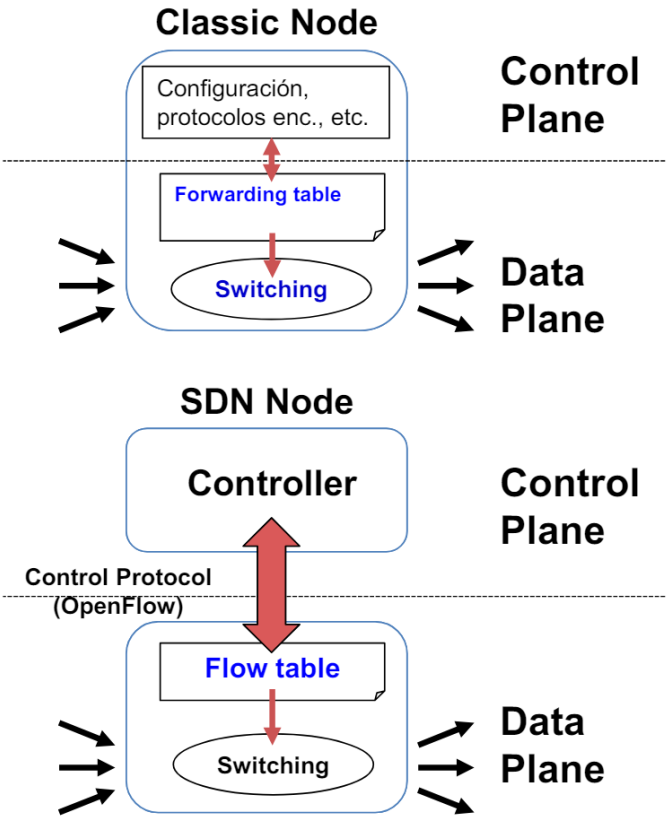


Figura 2.6: Arquitectura de *switch* clásico frente a arquitectura con SDN [4].

El controlador es el responsable de configurar a los *switches*, de esta manera se tiene un único controlador para múltiples *switches*, optimizando la gestión de las redes. Para la comunicación entre el controlador y los *switches*, se estandariza el protocolo *OpenFlow*. *OpenFlow* fue estandarizado por la *Open Networking Foundation* (ONF) en su versión inicial en 2011. *OpenFlow* permite tener una interfaz abierta, dejando de lado protocolos propietarios o personalizados de los fabricantes, dando lugar a un entorno más flexible [4].

Además de SDN, existe otra tendencia y evolución en las redes que permite desagregar el software de la infraestructura. Esto se conoce como NFV (*Network Functions Virtualization*). NFV propone la implementación de funciones de red mediante software corriendo sobre hardware estándar (COTS). Esto implica un gran ahorro de costes ya que aumenta la competencia y además proporciona una gran agilidad para desplegar, actualizar o eliminar los servicios. Este enfoque de NFV ya se ha visto en el capítulo de la RAN ya que es un concepto que no se ha quedado únicamente en las redes de transporte sino que se extiende a la mayoría de dominios. La arquitectura NFV fue estandarizada por la ETSI en 2012. La arquitectura se representa en la figura [27] [5]:

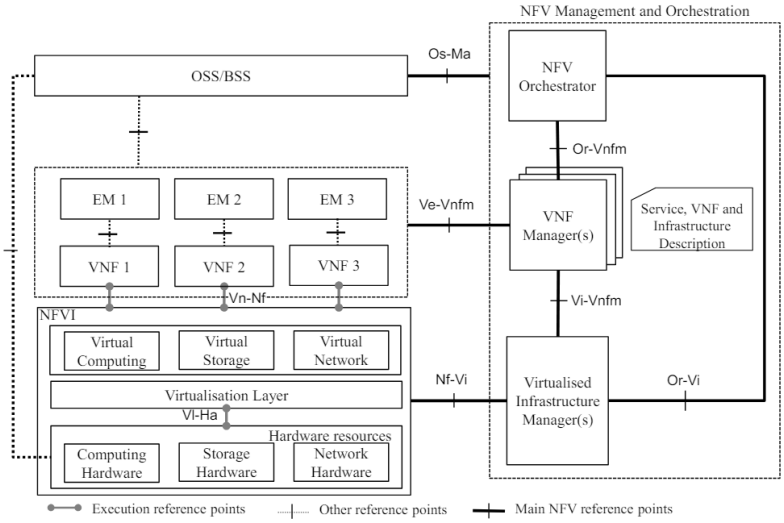


Figura 2.7: Arquitectura NFV [5].

En esta arquitectura podemos encontrar las siglas de EM y VNF. EM hace referencia a *Element Management* y VNF *Virtualised Network Function*. VNF son las funciones de red virtualizadas como podría ser, por ejemplo, un *router*. Estas funciones de red virtualizadas, se despliegan en lo que se conoce como NFVI (*Network functions virtualization infrastructure*) y es el hardware en el que se despliegan las funciones de red virtualizadas. En la parte derecha vemos la arquitectura de como se deberían gestionar este tipo de redes. Como vemos en la parte superior, se tienen los OSS y BSS que como veremos en las próximas secciones, son los sistemas a través de los cuales se gestionan los recursos y requisitos de negocio. Para mas detalles de la arquitectura, se puede hacer uso de la documentación oficial de la ETSI [5].

2.1.5. Modelo operativo y gestión de redes

En los comienzos de las comunicaciones móviles, las redes eran pequeñas y las tareas de gestión se podían hacer de una manera manual y sencilla. A medida que las redes fueron creciendo, se vio la necesidad de crear un proceso de gestión más adecuado que permitiese gestionar las nuevas redes más grandes y complejas.

Para realizar esta gestión, el ISO (*International Organization for Standardization*) define el acrónimo FCAPS. FCAPS hace referencia a las siguientes áreas funcionales [6] [28]:

- Gestión de Fallos: Un modelo operativo adecuado, debe ser capaz de gestionar los fallos de los elementos de red. Esto incluye, la detección de fallos tan pronto como sea posible, a través de mecanismos como *polling* u otros. Además, la gestión debe aislar el sistema que ha fallado, realizar un diagnostico del problema e intentar mediante mecanismos de correlación, encontrar patrones en los errores para intentar prevenirlos en un futuro. Por otra parte, el sistema de gestión debe reactivar el sistema caído para que no haya impactos en el servicio e intentar

solucionarlo en el menor tiempo posible.

- Gestión de Configuración: La gestión de configuración intenta garantizar que los recursos de red estén correctamente inicializados, configurados y administrados. Esto incluye la inicialización de recursos para integrar nuevos dispositivos en la red, el aprovisionamiento para asignar servicios a esos recursos, y la autodetección para identificar automáticamente dispositivos recién conectados. También cubre la copia de seguridad y restauración, asegurando que la configuración se pueda recuperar en caso de fallos. Por otra parte, debe permitir una actualización fluida de las configuraciones. Por último, se encarga también de hacer la gestión de inventario, manteniendo un registro actualizado de todos los elementos en la red.

- Gestión de Contabilidad: La gestión de contabilidad tiene como objetivo monitorizar y controlar los costes asociados con el uso de los recursos de la red. Se determinan los costes de servicio, ayudando a asignar el precio adecuado para cada recurso. Por otra parte, se establecen límites de contabilidad para fijar barreras en el consumo, evitando sobrecargas o un uso indebido. Además, las auditorías periódicas son necesarias para garantizar que las operaciones sean transparentes.

- Gestión de Rendimiento: En esta caso, la gestión de rendimiento es clave ya que las redes están diseñadas para cumplir con un cierto grado de calidad. Esto significa que incluso en las peores condiciones, los sistemas deben cumplir con la calidad de servicio *QoS* especificada. Para conseguir esto, se realizan análisis de datos, recolección y examinación de *logs*, monitorización del rendimiento, generación de informes de rendimiento y errores, etc. Por último, se deben realizar planes y pruebas exhaustivas tanto de capacidad como de calidad de una manera periódica. De esta manera, una empresa es capaz de asegurar el correcto funcionamiento de sus redes.

- Gestión de Seguridad: La seguridad es un pilar fundamental en cualquier ámbito de la tecnología y más si hablamos de redes. La gestión debe incluir el acceso selectivo a recursos, permitiendo solo a usuarios autorizados interactuar con ciertos elementos críticos de la red. Por otra parte, se deben incluir alarmas de seguridad que permitan a los administradores actuar rápidamente ante accesos no autorizados. La lectura de los logs de los sistemas, también permite a los administradores conocer quienes acceden a qué recursos. Por último, se debe mantener un registro de auditoría de seguridad que proporcione un historial detallado para verificar la integridad de los sistemas.

Aunque la gestión de redes siguiendo el modelo FCAPS es algo mucho más amplio, se define el estándar *Simple Network Management Protocol* (SNMP) para la gestión de redes. SNMP fue diseñado para monitorear y administrar dispositivos conectados en una red IP, por ejemplo, routers, switches, servidores u otros elementos. A través de un sistema centralizado (*manager*), SNMP obtiene información de los dispositivos administrados, también conocidos como agentes. Cada agente, almacena datos en

una base de datos llamada *Management Information Base* (MIB), que organiza la información en un formato estandarizado. El gestor puede consultar o modificar estos datos para evaluar el rendimiento, detectar problemas y recibir alertas de los dispositivos cuando ocurren ciertos eventos. La arquitectura se muestra en la figura 2.8.

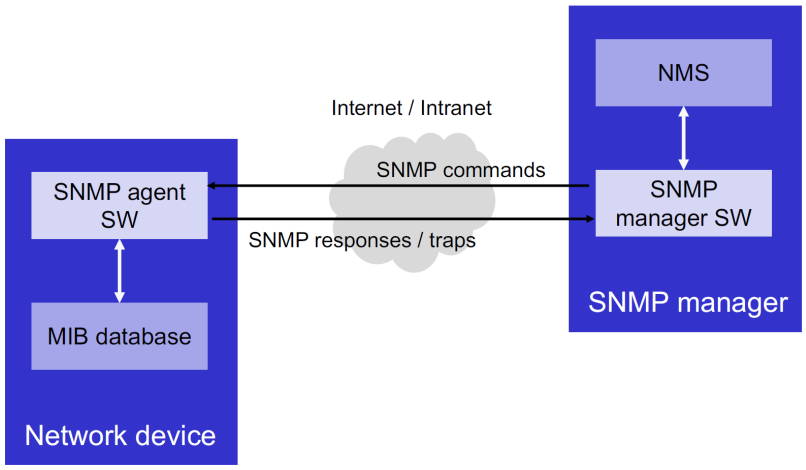


Figura 2.8: Arquitectura SNMP [6].

SNMP tuvo una gran acogida debido a su simplicidad, convirtiéndolo en el protocolo de gestión predominante durante muchos años. Sin embargo, hoy en día aunque se sigue utilizando en muchos sistemas, otros protocolos como NETCONF están ganando más fuerza en los nuevos entornos [29]. Además, también se están empezando a usar otro tipo de tecnologías como la telemetría basada en *streaming*. Esto permite monitorización en tiempo real mediante el envío continuo de datos de rendimiento, fallos y otros eventos desde los dispositivos a un sistema central. Por último, también existen API estandarizadas por el *TM Forum* que permiten que los sistemas OSS obtengan la información de la red, tal y como veremos en el siguiente apartado [28].

2.1.6. Sistemas IT

Para un operador de telecomunicaciones, no sólo es importante tener unas redes sofisticadas y que ofrezcan un servicio con la mejor calidad posible para los usuarios, también necesitan de una serie de sistemas que les permita gestionar la facturación, la tarificación, el inventario de recursos, la creación de nuevos recursos, los datos del cliente, los elementos de red, los consentimientos y bases legales, etc.

Para todas estas actividades, las operadoras de telecomunicación cuentan principalmente con dos sistemas, el OSS (*Operation Support System*)y el BSS (*Business Support System*). El OSS es un conjunto de sistemas y herramientas que permite a las operadoras de telecomunicaciones gestionar de manera eficiente la infraestructura, los servicios, el inventario y la experiencia del cliente. El OSS monitoriza, controla

y optimiza los recursos de red y los servicios asegurando que estén alineados con los objetivos de negocio. Por otro lado, son capaces de automatizar la provisión de nuevos servicios y ajustar los existentes de acuerdo a nuevos requisitos. por ejemplo, si un cliente cambia de contrato y aumenta la capacidad de la línea contratada, el OSS se encarga de recibir la petición e interactuar con la red para cumplir con el nuevo contrato de una manera automática [7].

Las capacidades de este sistema no se quedan ahí, es capaz de detectar aislar y resolver los fallos, coordinándose con los equipos de mantenimiento y soporte. Por ejemplo, si detecta el no correcto funcionamiento de un *router*, se encarga de identificar el problema, generar una alarma y abrir un *ticket* al equipo de soporte de manera automática. También proporciona un repositorio centralizado de todos los recursos dentro de la empresa, permitiendo un agestión sencilla y que da pie a un escalado y planificación eficiente. Por otro lado, si detecta que la calidad de un servicio no está acorde con los SLA (*Service Level Agreement*), se encarga de ajustar los parámetros necesarios en la red para intentar cumplir con la calidad contratada por el cliente [30].

Evidentemente, para que el OSS pueda realizar todas estas funciones que hemos comentado, debe ser capaz de orquestar diferentes peticiones e integrarse con todos los recursos de la cadena incluyendo desde elementos de la RAN, pasando por routers, switches o elementos del CORE hasta los sistemas BSS. Como vamos a ver más adelante, la organización *TM Forum* intenta estandarizar mediante API la comunicación entre los diferentes componentes de los OSS y BSS. Por otra parte, la comunicación con los elementos de red se puede hacer con los protocolos que hemos visto o API definidas en TM Forum.

En cuanto al BSS, se utiliza para gestionar los aspectos comerciales de la operadora, por ejemplo, las interacciones con los clientes, la facturación o los ingresos. La gestión de clientes se hace a través del CRM (*Custom Relationship Management*) que es donde se almacenan la información de los mismos, permitiendo hacer cierta segmentación y ofreciéndoles servicios personalizados. Esta información de clientes incluye el historial de compras, suscripciones, preferencias, etc.

La funcionalidad del BSS también incluye la facturación de todos los servicios prestados. Aquí se incluye calcular las tarifas según los planes de servicio, manejar descuentos y promociones, y generar facturas detalladas. También puede administrar cuentas por cobrar, hacer un seguimiento de los pagos y generar informes financieros. Por otra parte, permite diseñar, crear y gestionar el catálogo de producto para clientes B2B (*Business to Business*) y B2C (*Business to Consumer*), facilitando la definición de precios, promociones, retirada y creación de productos según la demanda, etc. El BSS además, debe gestionar los pedidos con el OSS para que tras el proceso de compra de un producto, se active en los recursos lo necesario para prestar el servicio. Por último, ofrece a los clientes la capacidad de gestionar sus cuentas, ver facturas, realizar cambios en sus servicios o reportar problemas a través de portales web.

Podemos encontrar una representación de una arquitectura BSS en la figura 2.9.

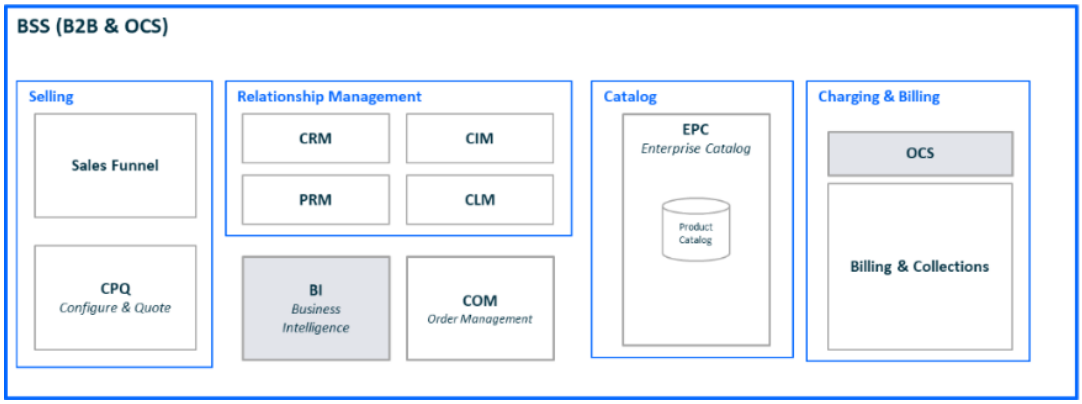


Figura 2.9: Arquitectura BSS [7].

Como se ha comentado previamente, existe una organización llamada *TM Forum* que desarrolla estándares, marcos y mejores prácticas para la transformación digital de los proveedores de servicios de comunicaciones. *TM Forum* es realmente una alianza entre más de 800 compañías que surgió en 1989. Para tratar de conseguir esta transformación digital, *TM Forum* propone una mejor interoperabilidad de los sistemas a través de iniciativas como *Open API* y *Open Digital Architecture* (ODA) [8].

TM Forum ha desarrollado una serie de *Open* API diseñadas para proporcionar interfaces estandarizadas entre diferentes componentes de los sistemas OSS/BSS y facilitar la integración. Las *Open* API, permiten que los diferentes sistemas OSS/BSS de múltiples proveedores se comuniquen sin problemas, mejorando la interoperabilidad y reduciendo los costos de integración. Al proporcionar interfaces abiertas y comunes, las *Open* API ayudan a los operadores a integrar servicios nuevos o de terceros rápidamente, teniendo un tiempo de comercialización más rápido. Algunos ejemplos de *Open* API que existen para la gestión de red, pueden ser TMF 642 gestión de alarmas,TMF 628 gestión de rendimiento, TMF 656 gestión de problemas, TMF 677 gestión de consumo, etc [30].

En el caso de ODA, es un marco modular basado en la arquitectura de microservicios, que *TM Forum* propone como la siguiente generación para los sistemas OSS/BSS. Su objetivo es transformar la forma en que los CSPs diseñan, implementan y gestionan sus servicios. Se propone una arquitectura basada en componentes autónomos que permite a las operadoras integrar o reemplazar partes sin afectar al sistema completo. Esto se consigue gracias a que cada todos los componentes se deben comunicar mediante las *Open* API y el componente lo único que debe hacer es exponer las API definidas. ODA es una evolución natural tal y como explica *TM Forum* en su documentación.

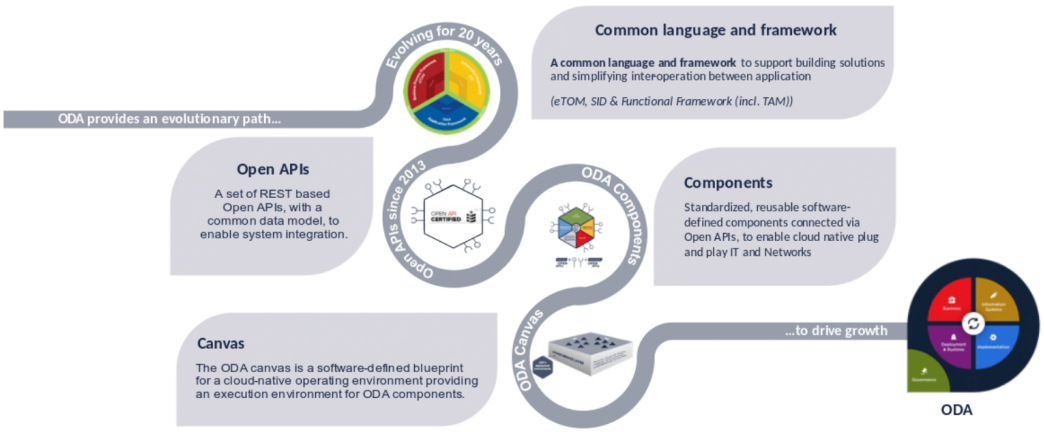


Figura 2.10: Evolución de BSS y OSS hacia ODA según *TM Forum* [8].

2.1.7. Seguridad y Privacidad

El ámbito de la seguridad es algo muy amplio y no es el objetivo de esta sección contar en detalle todos los procedimientos existentes para asegurar y gestionar los requisitos de seguridad. Se va a aprovechar este punto para destacar la importancia los mecanismos de seguridad en una empresa como una operadora de telecomunicaciones, ya que son compañías que manejan información de un gran número de personas. Las operadoras de telecomunicaciones guardan datos muy relevantes de los clientes como pueden ser algunos identificadores como el DNI/NIE, nombre y apellidos u otros campos cuasi identificadores, por ejemplo, la dirección del cliente, el código postal, la fecha de nacimiento, el género, etc.

Cualquier ataque que suponga la pérdida de esta información tendría un impacto muy grande en la empresa. Por este motivo, es necesario realizar una correcta gestión de la seguridad tratando de evitar cualquier tipo de vulnerabilidad y reaccionando de una manera efectiva ante cualquier incidente.

Para hacer una correcta gestión de la seguridad, el primer paso es definir el análisis de riesgos y la política de seguridad, en segundo lugar, implementar los servicios de seguridad y la infraestructura asociada. Por último, se realiza la operación y gestión a través de alarmas, registros, informes de seguridad, etc y se certifica a través de pruebas de penetración, auditorías u otros mecanismos la validez del modelo . Normalmente, esta gestión se hace desde el SOC (*Security Operations Center*). La siguiente figura, resume las actividades de la gestión y operación de la seguridad:

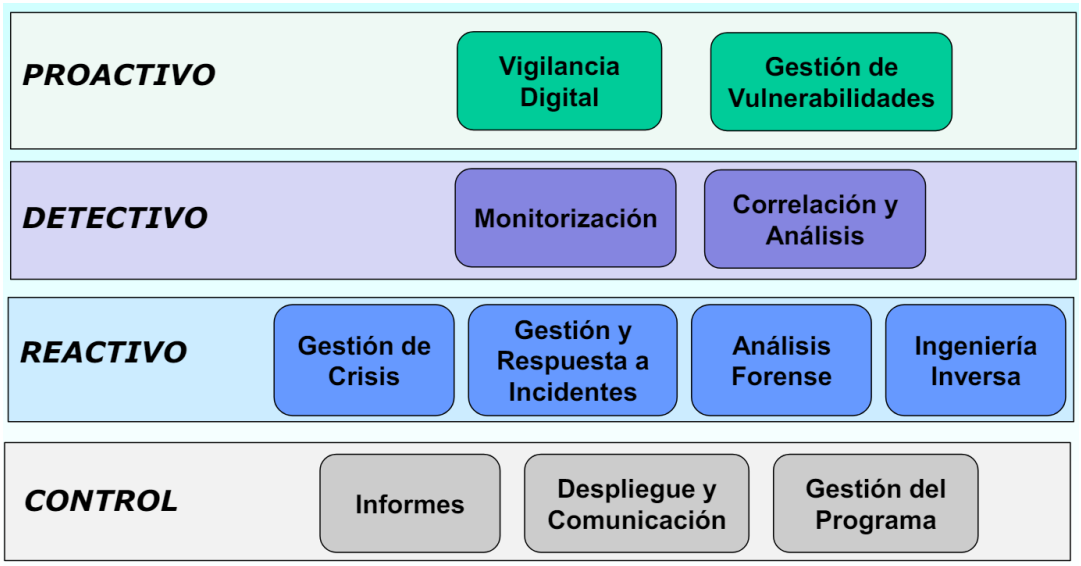


Figura 2.11: Modelo de operación y gestión de la seguridad [9].

La capa de control proporciona los informes y la comunicación necesarios para mantener a todos los equipos informados y alineados con las políticas y procedimientos de seguridad. También incluye la gestión del programa, asegurando que los esfuerzos de seguridad estén coordinados, organizados y se ejecuten de acuerdo con las estrategias establecidas. Por otra parte, la capa reactiva trata acerca de la gestión de crisis, la respuesta a incidentes, el análisis forense y la ingeniería inversa. La gestión de crisis trata de minimizar el impacto de cualquier incidente que afecte a la seguridad. Con respecto a la gestión y respuesta a incidentes se busca aislar y solucionar lo más rápido posible los problemas que puedan impactar en la seguridad. El análisis forense supone el estudio detallado de los eventos para identificar la causa raíz. Por último, la ingeniería inversa ayuda a descubrir amenazas complejas para entender su funcionamiento [9].

A nivel de detección, es decir, de detectar las amenazas de seguridad para evitar cualquier explotación de vulnerabilidades, se gestiona de una manera efectiva a través de un SIEM (*Security information and event management*). El SIEM es una tecnología que expone n en tiempo real el análisis de alertas generadas por dispositivos conectados, así como también sistemas operativos y aplicaciones. Con esta información el SIEM tiene la capacidad de generar alertas,reportes e incidentes de seguridad, con fines de cumplimiento o de gestión de seguridad. A nivel de medidas proactivas, destacan las actividades de *hacking* ético en el que se intenta poner a prueba a través de ataques la vulnerabilidad del sistema o la inteligencia de amenazas [9].

A nivel de privacidad, las operadoras de telecomunicación deben cumplir el Reglamento General de Protección de Datos o GDPR por sus siglas en ingles. Esto es así ya que cualquier operación o conjunto de operaciones realizadas sobre datos personales, por ejemplo, recogida o registro, organización y estructuración, adaptación

o modificación, extracción, consulta, etc se considera tratamiento de datos personales y debe cumplir con la regulación. Como es habitual, este tipo de empresas recoge datos personales de los clientes para darles de alta en sus servicios.

En el modelo de privacidad, encontramos diferentes roles. En primer lugar, esta el interesado que es la persona de la cual se recogen los datos. Por otra parte, se tiene al responsable que es el actor encargado de decidir qué información recoger, de quien y los fines para los que se usan los datos. Además, está el encargado que es quien trata los datos por cuenta del responsable, sin necesidad del consentimiento del usuario. Por último, se encuentra el tercero, que es un nuevo responsable para sus propios tratamientos pero puede no tener relación directa con el interesado. Debe tener una base lega para el procesamiento ya que se hace cargo de los datos.

Atendiendo al GDPR, los datos recogidos solo podrán usarse para una finalidad determinada, no podrán recabarse datos que sean excesivos. La recogida debe ser proporcional a la finalidad para la cual .se van a utilizar los datos. Además, se debe cumplir el principio de licitud por el cual el interesado debe dar consentimiento previo, libre, explícito e informado. También se cumple este principio si se recoge para funciones de obligación legal, el tratamiento de los datos tiene como fin proteger un interés vital del titular de los datos, el tratamiento es necesario para el cumplimiento de una misión realizada en interés público o por interés legítimo del responsable y no prevalezca el interés o los derechos y libertades fundamentales del titular de los datos [31] [32].

A parte de lo ya comentado, el interesado gana una serie de derechos como son el derecho de acceso, el derecho de rectificación, el derecho de la supresión, el derecho a la limitación del tratamiento, el derecho a la portabilidad de los datos, el derecho a la oposición y el derecho a no ser objeto de decisiones individualizadas. Por otra parte, el responsable tiene una serie de obligaciones. El responsable debe registrar las actividades del tratamiento, evaluar el impacto, incrementar la transparencia, aplicar principios de privacidad desde el diseño y por defecto, tener un delegado de protección de datos y notificar violaciones de seguridad a la autoridad competente [31].

2.2. Evolución del CORE 5G a la exposición de servicios.

En la sección anterior, se ha introducido la arquitectura del CORE 5G y se ha dado una breve descripción de cada uno de los componentes. En esta sección, vamos a indagar acerca de la relevancia del NEF para la exposición de servicios y capacidades del CORE 5G y se comentarán alguno de los flujos más relevantes que pueden ser monetizados a través de proyectos como *Open Gateway*.

El NEF viene definido por primera vez en el TS 23.501 del 3GPP al igual que el resto de NF que se han visto anteriormente. Se usará esta documentación como base para repasar los conceptos del NEF. Como ya se ha explicado, el rol principal del NEF es la exposición controlada de servicios de red. Sin embargo, no es su única función y

se dejará para el final de esta sección [10].

En primer lugar, el NEF permite tener un punto de acceso único al CORE con una interfaz estándar definida. Esto supone varias ventajas, comenzando quizás con la más obvia, permite abstraer a los AF (*Application Function*) o usuarios de la complejidad de las interfaces de la red. Es decir, no es necesario que se conozcan en detalle todas las interfaces de los componentes del CORE para acceder a cierta información, sino que solo es necesario interactuar con un único elemento, el NEF.

Al ser el punto de entrada central a las capacidades del 5G, es también el candidato ideal para realizar las tareas de seguridad como el control de accesos. En este sentido, el NEF ofrece mecanismos para autenticar, autorizar a los AF así como para realizar tareas de *throttling* o *rate limit*.

Otro de las funcionalidades más interesantes que ofrece el NEF es que es capaz de redirigir las peticiones de un AF a otro NEF cuando la consulta del AF específica algún tipo de información local. Por otro lado, el NEF recibe información ed otras NF y el NEF almacena la información como datos estructurados en el UDR, utilizando la interfaz estándar. La información almacenada en el UDR puede servir a otras NF para analíticas u otros propósitos, o ser expuesta por NEF.

Ahora sí, la función principal del NEF y la más relevante es la exposición de las capacidades del CORE 5G a través de API estándar. Tal y como se ha venido comentando, el NEF expone las capacidades del CORE a través de API REST. Este tipo de API son las API *Northbound* del NEF, es decir, las API que sirven de comunicación entre el NEF y los AF. Las API *Northbound* vienen especificadas en el TS 29.522. En este documento, se detallan para cada una de las API, el objetivo, los parámetros de entrada, los parámetros de salida y los métodos que tiene. A pesar de que el documento es bastante claro, la descarga de este *technical stream* también incluye las especificaciones YAML para poder visualizar las API en herramientas como *Swagger*.

Además de las API, el 3GPP define lo que se conoce como *procedures*, que son los procedimientos para obtener algún tipo de información. Esto no incluye únicamente el API *Northbound* del AF con el NEF sino que también incluye todos los servicios y API de todos los elementos del CORE necesarios para disponer de la información deseada. Por otra parte, se definen también los servicios o *services* que es una abstracción superior al API donde se define la funcionalidad que ofrece un componente de la red. El API es la implementación concreta del servicio.

Dentro de todos los procedimientos y servicios que existen, hay uno que cobra especial importancia debido a la gran cantidad de usos que se le puede dar a la información obtenida. Este es el procedimiento de monitorización de eventos. La monitorización de eventos tiene como objetivo monitorizar los cambios de algunos parámetros de la red y notificar cuando haya cambios a las aplicaciones suscritas. En

la siguiente tabla se recogen los eventos más relevantes que se pueden obtener de este procedimiento.

Evento	**Criterio de detección**	**NF que detecta el cambio**
Pérdida de conexión	La red detecta que el UE no es alcanzable para señalización ni comunicación del plano de usuario	AMF
Informe de localización	El evento es detectado en base a los criterios especificados en la petición de monitorización (Periodicidad, duración, número máximo de informes, etc.)	AMF
Estatus de *roaming*	El evento se detecta cuando el estatus de *roaming* de un UE cambia. (*Roaming* ->No *Roaming* y viceversa)	UDM
Fallo en la comunicación	El evento es detectado cuando el nivel de fallo en la RAN es detectado, basado en la liberación de conexión	AMF

Tabla 2.2: Eventos monitorizables que se pueden notificar a través de NEF

El servicio y el API en concreto que exponen esta información es *Nnef_EventExposure Service* API y *Nnef_EventExposure Service API* respectivamente. Ambas están definidas en el el TS 29.591 [33]. Por otro lado, el procedimiento y los flujos asociados entre los distintos NFs viene especificado en el TS 23.502 [34]. Aquí se detallan las interacciones entre los distintos componentes del CORE para exponer los eventos previos. Se pueden observar los flujos de información sobre las operaciones de servicio de AMF y UDM en las figuras 2.12 e 2.13 respectivamente.

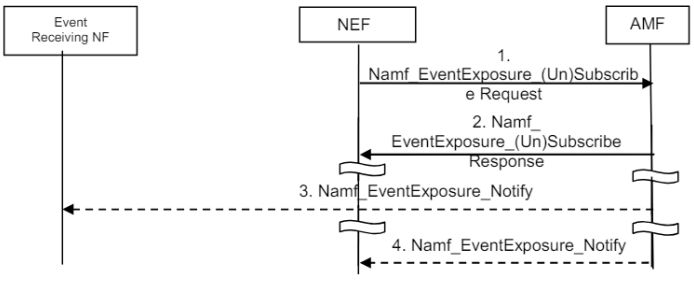


Figura 2.12: Diagrama de flujo para eventos originados en el AMF [10].

Aunque en estas figuras no aparece, el AF se suscribe y es notificado a través de las API que hemos comentado previamente de *Event Exposure*. En el caso de flujos de información sobre las operaciones de servicio de AMF, el NEF se suscribe a estos eventos a través de *Namf_ EventExposure Subscribe request*. Con esta petición, el NEF se suscribe a sí mismo o a otras NF a una serie de eventos identificados por un *event id*. En el caso de que sea otra NF la que se está suscribiendo a través del NEF, el NEF debe incluir en la petición el *endpoint* a quien el AMF se lo debe enviar. Tras recibir la petición, el AMF manda un ACK. Cuando se produce un evento al cual hay una NF suscrita, el AMF manda la notificación a la NF ya sea el NEF u otra (3 y 4) [10].

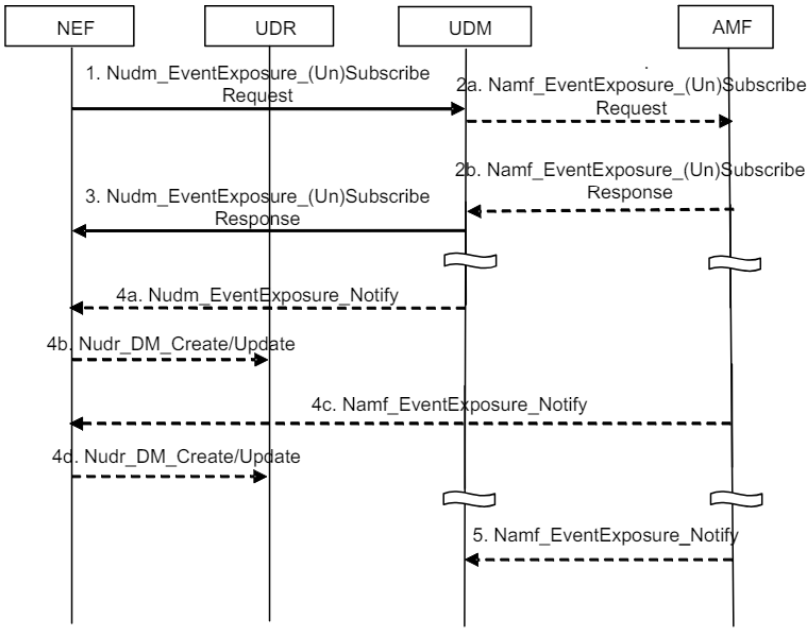


Figura 2.13: Diagrama de flujo para eventos originados en el UDM [10].

Para los flujos de información sobre las operaciones de servicio de UDM, el proceso es bastante similar. El NEF se suscribe al UDM a través de *Nudm_ EventExposure Subscribe request*, posteriormente, el UDM devuelve un ACK (paso 3) o bien, hay algunos eventos en el que el UDM se tiene que suscribir al AMF, tal y como se describe en el flujo anterior, como por ejemplo el evento de pérdida de conexión. En el caso de eventos que no necesitan la información del AMF, el UDM notifica al NEF directamente cuando detecta algún evento al cual el NEF se ha suscrito, lo hace a través de *Nudm_ EventExposure Subscribe response* (Paso 4.a). Con la información del evento, el NEF puede que guarde el evento de una manera estructurada en el UDR, aunque este paso es opcional. Para los casos en los que el origen es el AMF, es el mismo AMF quien notifica al NEF (Paso 4.c). Por último, también existen eventos en los que es el UDM el que debe recibir la notificación y no el NEF tal y como se describe en el punto 5. Un ejemplo podría ser un cambio de AMF por reasignación y que debe notificar al UDM del cambio [10].

Por último, existe otros procedimientos y servicios que también pueden tener muchos casos de uso e impacto. Es el caso del procedimiento que permite modificar un flujo de datos con un QoS en específico o crear una nueva sesión PDU con una calidad de sesión dada. El servicio utilizado para esto es *Nnef_AFsessionWithQoS service* y viene especificado en el TS 29.522 [35]. De una manera gráfica, lo que se consigue es los siguiente:

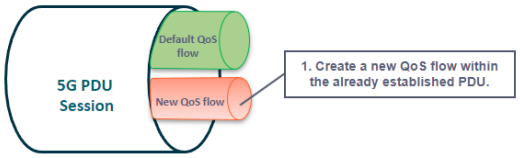


Figura 2.14: Nuevo Flujo de QoS [10].

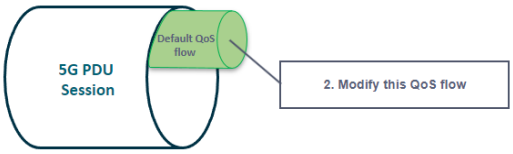


Figura 2.15: Modificación de flujo de QoS existente [10].

Los flujos detallados para lograr esto están explicados en el TS 23.502 (4.15.6.6). Entre toda la información que se puede obtener del núcleo (CORE) a través de la monitorización, existen casos de uso claros para los datos de localización y *roaming*. Conocer la ubicación de un usuario o si está o no en *roaming* puede ser muy relevante en casos de antifraude o para llevar a cabo campañas de marketing más especializadas.

En cuanto a los cambios en las sesiones de calidad de servicio (QoS), las posibilidades son infinitas, desde mejoras en la conexión para juegos en tiempo real hasta aplicaciones que requieran toma de decisiones en tiempo real, como la navegación de drones, coches autónomos, sistemas de seguridad, etc. También abren la puerta a conceptos más futuristas como el metaverso.

2.3. El proyecto *Open Gateway*.

Open gateway es el pilar fundamental en el que se basa este trabajo de fin de máster. Para entender el racional que busca este proyecto, hay que entender el estado actual al que se enfrentan las operadoras de telecomunicaciones. El mercado actual está copado principalmente por aplicaciones OTT (*Over The Top*) en las que los usuarios pasan varias horas del día y que generan una gran cantidad de tráfico, un ejemplo claro son aplicaciones como *YouTube*, *Instagram*, *Facebook*, *Tik Tok* o X (antiguamente conocido como *Twitter*) [36].

Por otra parte, las operadoras no están monetizando este tráfico e incluso se ven obligadas a invertir en nuevas redes con más capacidad para soportar todo este tráfico. Además, tanto el crecimiento en la captación de usuarios como el ARPU (*Average Revenue Per User*) son parámetros que se suelen mantener estables.

En este contexto, las operadoras de telecomunicaciones están buscando nuevas formas de monetizar sus redes. Es aquí donde *Open Gateway* entra en juego. *Open*

Gateway trata de abrir las redes de telecomunicaciones a terceros en forma de API. El objetivo es que desarrolladores de todo el mundo puedan usar estas API, adaptando la red a los requisitos de sus aplicaciones. Por decirlo de otra forma, la red pasa de ser algo inaccesible donde cualquier cambio es crítico, a tener redes de telecomunicaciones programables para cada caso de uso.

Cabe destacar que las API deben ser estándar entre todos los operadores del mundo. La razón es simple, el desarrollador no puede programar su aplicación en función de la empresa que tiene contratada el usuario, sería un fracaso total. De esta manera, la GSMA es la encargada de estandarizar todas las API a través de CAMARA, un proyecto de la fundación Linux. El proyecto *Open Gateway* fue lanzado oficialmente en el MWC (*Mobile World Congress*) de Barcelona del 2022 de la mano de 22 operadores a nivel mundial. Desde entonces, ha sido tema recurrente en las últimas ediciones, en las que ha habido varias demostraciones de casos de uso con diferentes API [37].

Al hablar de API estandarizadas en *CAMARA*, es posible que alguno piense en otro organismo como el *TM Forum* del que se ha hablado en la parte de la transformación de los sistemas IT y se pregunte cual es la diferencia. El *TM Forum* estandariza lo que se conoce como API de Operación mientras que las API de *Open Gateway*, que estandariza CAMARA, son API de servicio. Las API de *CAMARA* son únicamente API *Northbound*, buscan conectar directamente con el usuario final para ofrecerles un servicio [11].

Aunque ya se ha comentado que las API son estándar y abiertas, de manera que, todas las operadoras tienen que hacer la misma implementación, aún es posible que la llegada al mercado sea compleja si cada uno de los participantes tiene su propio *marketplace* en el que oferta las API de *Open Gateway*. De nuevo, este enfoque no tendría sentido ya que el desarrollador se tendría que preocupar nuevamente de a que sitio direccionar sus peticiones en función de la empresa que tenga contratada el usuario. Es en este punto donde entran otro de los actores principales del proyecto que son los *hyperscalers* o *marketplace*.

Hyperscalers se refiere a empresas como *AWS*, *Azure* o *Google Cloud* que ya ofrecen a desarrolladores un portal de venta de API y que ofrecen servicios de computación en la nube a gran escala . Lo mismo ocurre con las empresas denominadas *marketplace*, como son *Vonage* o *Twilio*, que ya tienen una plataforma en la que ofrecen este tipo de soluciones. Aunque en la figura 2.16 se abre la puerta a ambas posibilidades y es bueno tener múltiples puntos de venta, lo cierto es que la solución preferida pasa por ir con empresas que ya tienen capturados muchos clientes que usan las API de su portal. Para casos de uso B2B si que es posible que la solución a través de un *marketplace* de la operadora, cobre más sentido. Al fin y al cabo, los clientes B2B van a tener los servicios contratados solo con una operadora. La arquitectura se muestra a continuación:

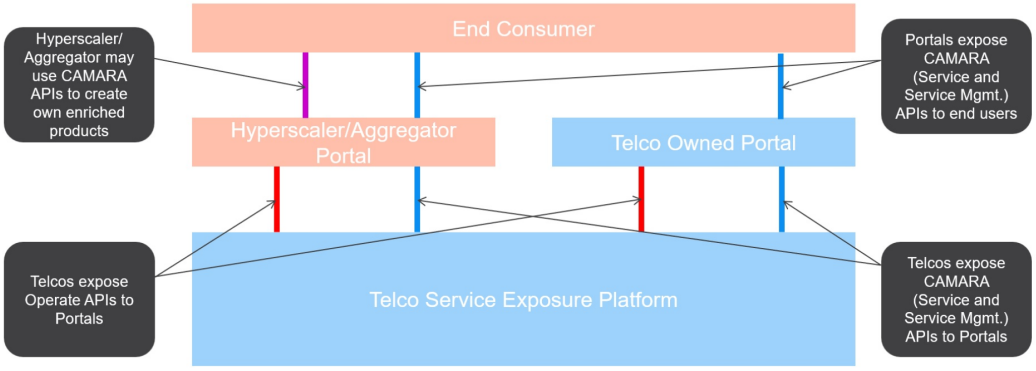


Figura 2.16: Modelo de referencia de CAMARA para el proyecto *Open Gateway* [11].

Todas las especificaciones de las API se encuentran en el repositorio oficial de *GitHub* de *CAMARA*. Las especificaciones se deciden entre un comité formado por trabajadores de las operadoras que son miembro del proyecto. Todas las especificaciones del API siguen el estándar de *Open API*. Seguir el formato *Open API* permite que las especificaciones y descripciones de las API sigan un formato consistente. *Open API* define reglas y formatos para especificar los *endpoints*, parámetros de entrada, tipos de datos y respuestas esperadas.

A parte de seguir el formato *Open API*, las API definidas son REST obteniendo todas las ventajas que ello conlleva. REST es en los últimos años se ha convertido en el estilo de API más usado, en contrapartida de SOAP, debido a su simplicidad y escalabilidad. Las API REST utilizan el protocolo HTTP con sus métodos estándar (*GET*, *POST*, *PUT*, *DELETE*, etc). Las API REST permiten el intercambio de datos en formato JSON, HTML, XML entre otros, para las API de CAMARA el formato es JSON.

A continuación, se va a comentar brevemente algunas de las API que ya están estandarizadas y casos de uso asociados:

- *Sim Swap*: Este API permite obtener la información del *timestamp* de cuando fue la última vez que el usuario cambió de SIM. Los principales casos de este API son antifraude. Algunas empresas bancarias reclaman que existe muchos casos de estafa donde el estafador adquiere los datos del usuario mediante algunas técnicas como el *phising*. Una vez que tiene la información del usuario como DNI, número de teléfono, nombre y apellidos, etc solicita un cambio de SIM a la operadora móvil. En caso de éxito, el estafador tiene acceso a todas las aplicaciones del usuario lo que facilita robos y compras fraudulentas en las cuentas de los usuarios.

- *Device Location*: Este API permite verificar cual es la localización de un usuario. Para ello, el cliente debe proporcionar como parámetros de entrada una posición, determinada por la longitud y latitud, y un radio. Con esta información, se tiene

una circunferencia. El API devuelve *true* o *false* dependiendo de si el usuario se encuentra dentro de la circunferencia o no. Para obtener esta información, lo lógico es hacer uso de la información del AMF o GMLC como se vio en la sección anterior. La localización que se obtiene de este proceso viene dado por la posición de la antena a la que se conecta el usuario y el radio de cobertura de la misma. Por lo tanto, lo que se está comparando, a grandes rasgos, es el solapamiento entre la circunferencia de entrada y la que se obtiene de la posición de la antena y el radio de cobertura. Como es lógico, puede haber casos en los que con este método, no se sea capaz de distinguir si una esta contenida dentro de la otra o no (*True*, *False*) y lo que hay es cierto solapamiento entre ambas. En estos casos, el API devuelve *Partial match*.

Los casos de uso de este API también están relacionados en su mayoría con el antifraude, por ejemplo, el banco puede estar interesado en verificar si el usuario está donde se realiza la compra. Sin embargo, el API tiene otros casos de uso como podrían ser las campañas de marketing personalizadas. El usuario obtendría anuncios personalizados del lugar en el que esté actualmente.

- *Quality On Demand*: Este API es quizás la que más impacto tiene para las operadoras y para los usuario. A efectos prácticos, es de las pocas API en la que el usuario final obtiene un servicio. A nivel de red es la más compleja. La razón es que el API tiene como objetivo garantizar una cierta capacidad o latencia. Esto se hace a través de los perfiles que se definen como parámetros de entrada en el API. Una primera idea para mejorar la calidad de un usuario es usar una estrategia basada en QCI. QCI es el acrónimo de *Quality of Service Class Identifier* y se utiliza en las redes móviles para especificar y distinguir entre diferentes clases de servicios y las calidades de servicio asociadas con ellos, para más información acerca de este identificador se puede visitar el TS 23.203 de 3GPP [38]. La estrategia consistiría en cambiar el QCI de la sesión a uno con más calidad, para hacer esto de puede usar el API de NEF que hemos visto en el apartado anterior o en su defecto directamente a través del PCF. Sin embargo, esta estrategia no es suficiente ya que no garantiza la capacidad sino que solo da una priorización a los paquetes de la sesión del usuario. Es por esto, que supone todo un reto el hecho de garantizar recursos en base a la demanda.

  Aunque no se ha comentado, todas las API están condicionadas por temas de regulación principalmente por los temas de privacidad que regula el GDPR como ya hemos visto. En el caso particular de *Quality On Demand*, tiene también un impacto en la neutralidad de red. La neutralidad de red exige que los proveedores de Internet deben tratar a todo tráfico de datos que transita por la red de igual forma. Esto puede suponer un problema para el desarrollo de este API, sin embargo, esta ley no aplica para servicios especializados. Esto se puede verificar en la regulación establecida en la documentación oficial de España [39]. En el caso de otros países, funciona de manera similar o no está regulado.

Los casos de uso para este API son muy amplios tal y como se describió en la sección anterior. Este API puede ser utilizada para mejorar el rendimiento de juegos en línea, IoT, *streaming*, coches autónomos, etc.

- *Device Status*: Este API proporciona la información del estado de roaming de un usuario. Además, en la última versión que existe ahora mismo (v0.5.0) incluye también otro tipo de información acerca de la conexión del usuario. Para obtener la información de necesaria de esta API lo natural es usar el API del NEF para monitorización de eventos o si no se cuenta con esta NF disponible, chequear la información directamente en el UDM. Los casos de uso de este API son similares a los ya comentados para *Sim Swap* y *Device Location*.

- *Know Your Customer*: El API de *Know Your Customer* permite validar la información que el usuario está dando con la que tiene una operadora acerca del mismo. La información que valida este API actualmente (v0.1.0), incluye el nombre, apellidos, dirección, fecha de nacimiento, DNI, dirección, código postal , género y otros campos relacionados con la información personal del usuario. Es evidente que para obtener información de este tipo, es necesario acceder a los BSS y en concreto al CRM. Aunque este API es muy útil, hay que tener en cuenta que las operadora de telecomunicaciones solo tienen la información de los titulares de la cuenta. Además, es posible que la información que tiene la operadora acerca del titular de la cuenta este desactualizada. Estos dos puntos pueden limitar la validez de respuesta del API y el número de clientes objetivo. De cualquier forma, el caso de uso del API es claro, verificar que la información que proporciona el usuario a una aplicación dada es correcta.

Algunas de estas API tienen varias versiones que incluyen nueva funcionalidad. En estos puntos se ha intentado mostrar una visión general del funcionamiento, objetivos y casos de uso.

Capítulo 3

Diseño e Implementación de arquitectura de microservicios para la exposición de API en *Open Gateway.*

En este capítulo se va explicar el proceso que se ha seguido para diseñar implementar una solución basada en microservicios para la exposición de API en *Open Gateway*. Las API que se van a implementar serán *Device Status* (v0.4.0), *Sim Swap* (v0.4.0) y *Know Your Customer* (v0.1.0). La solución estará basada en contenedores e incluirá las herramientas necesarias para cumplir con requisitos de observabilidad, gestión y almacenamiento de logs, seguridad, automatización de despliegue y gestión de base de usuarios.

En la parte de diseño, se destacará la importancia de cada una de las herramientas para cubrir los puntos mencionados. En la implementación, se usarán tecnologías concretas que permitan cumplir con estos requisitos. Por último, se mostrará en funcionamiento la solución desplegada.

3.1. Diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway.*

Para toda arquitectura que exponga API al exterior, hay puntos que son clave para asegurar el correcto funcionamiento del servicio. En esta sección se indagará en estos puntos clave, tomando las decisiones que se han considerado más adecuadas para este tipo de solución.

3.1.1. Diseño basado en una arquitectura centralizada.

El primer paso para realizar el diseño es conocer de donde se va a obtener la información necesaria para realizar las API. Esto es, de que sistema es necesaria extraer el *timestamp* de la última vez que un usuario cambio de SIM, de que sistema se va a obtener la información de *roaming* y de que sistema se obtendrán los datos del titular de la línea. Como se ha visto previamente, en una operadora cada una de esta

información se extrae de un sistema en particular. Un primera aproximación de diseño es pensar que los servicios que implementan el API debe hacer una consulta a cada uno de los sistemas pertinentes.

Sin embargo, en un entorno real esto podría tener varios inconvenientes. Tomando como ejemplo el API de KYC, es posible que la información del titular de la cuenta este distribuido por varios sistemas del BSS. La razón puede ser porque hay sistema que almacenan la información de la dirección del cliente y otros que almacenan datos del tipo nombre, apellidos, DNI, etc. además, puede que los sistemas que almacenan información de clientes B2B y B2C fuera distinta. O incluso sistemas diferentes para clientes prepago y pospago. Aunque el API carece de sentido para clientes prepago ya que en este tipo de contratos no se suele pedir la información del usuario. también es posible que los sistemas se estén migrando según la evolución que hemos visto en la sección de Sistemas IT. Por lo que habría parte de la información en un sistema antiguos y parte en en otros más nuevos.

Para el caso de *SIM Swap* y *Device Status* quizás es menos claro pero aplica de igual manera. La información de *SIM Swap* también se a en los sistemas y también aplicarían los mismos problemas que se han visto previamente. En el caso de *Device Status*, el problema puede surgir si además de cubrir los usuarios en red 5G se tiene que cubrir además los de la red 4G. En este caso,no valdría con la información proporcionada por NEF sino que habría que consultar directamente al HSS.

Se muestra a continuación de una manera visual y genérica del tipo de arquitectura que se está hablando:

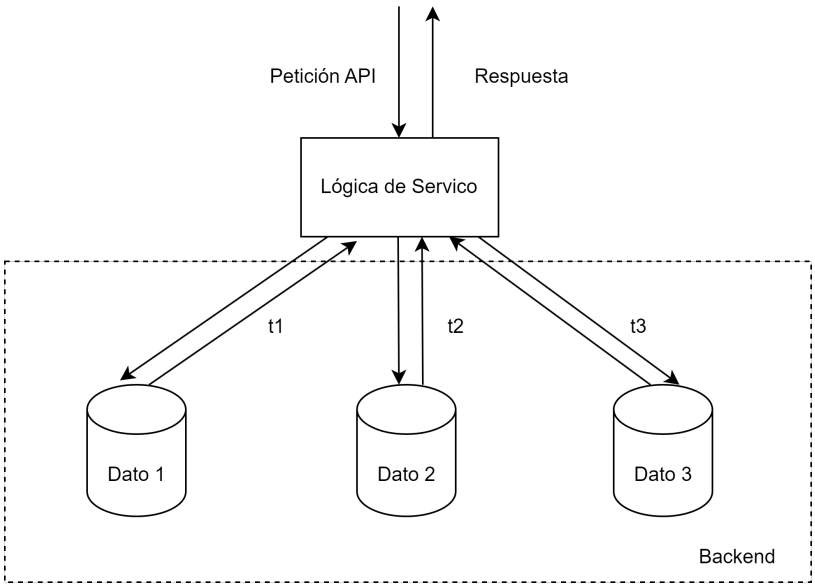


Figura 3.1: Arquitectura descentralizada para consulta de datos.

Este tipo de arquitecturas en la que la lógica del servicio tiene que hacer una consulta a varias bases de información, pueden tener varias desventajas. En primer lugar, si las consultas se realizan de una en una para obtener la información, el tiempo de respuesta va a ser igual a la suma de los tiempos de respuesta de cada base de datos. Si para optimizar estos tiempos se decide hacer las peticiones en paralelo, el tiempo de respuesta se reducirá drásticamente ya que será igual al máximo tiempo de respuesta de entre todas las bases. Sin embargo, de esa forma se estarían gastando recursos y saturando las fuentes de información continuamente. A nivel de cuantas peticiones por segundo (TPS) es capaz de ofrecer esta solución, siempre va a ser igual al menor de los TPS máximos que ofrezca cada base.

Teniendo en cuenta estos puntos, se decide buscar otro tipo de arquitectura que quizás sea más eficiente en términos de latencia y transacciones por segundo. Para esto, se propone la siguiente arquitectura:

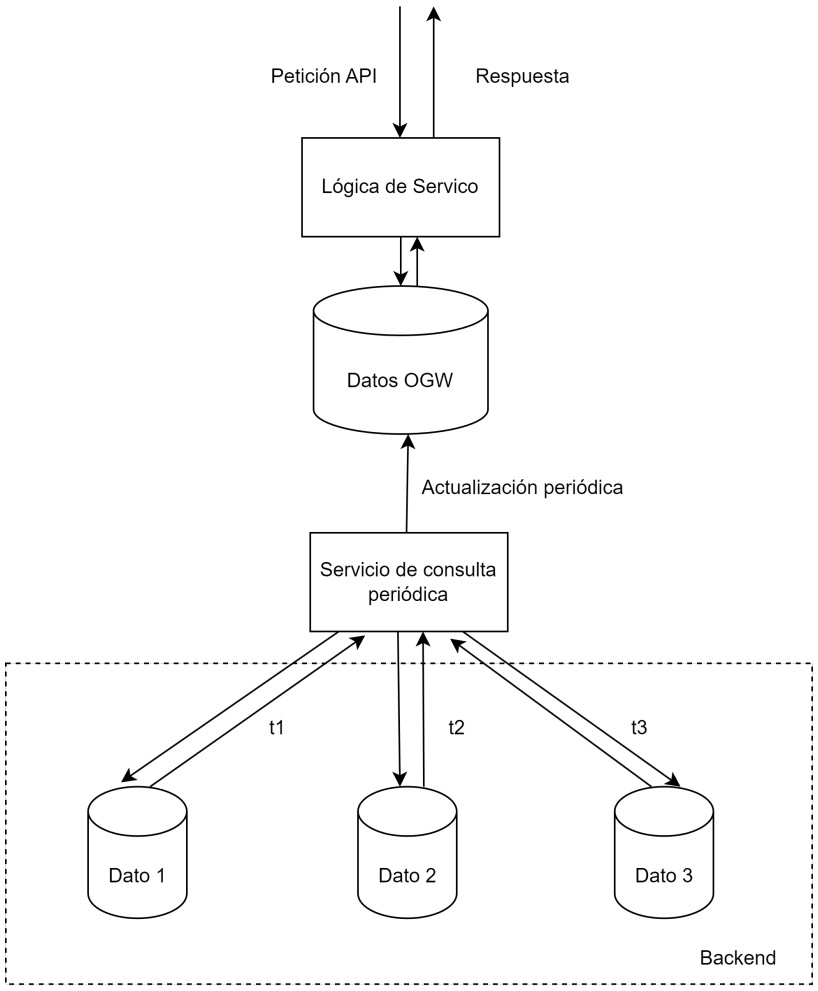


Figura 3.2: Arquitectura descentralizada para consulta de datos.

En esta nueva arquitectura, se propone hacer uso de una base de datos que

centralice la información de las API de *Open Gateway*. Esto va a permitir introducir bases de datos más recientes y *cloud native*, dejando a un lado las posibles bases de datos legadas de las operadoras. Por otro lado, a nivel de tiempo de respuesta, siempre se va a tener como resultado el tiempo de respuesta de la base de datos centralizada. Lo mismo aplica para las transacciones por segundo. La información en esta base de datos se debe actualizar periódicamente para evitar que un cliente obtenga información obsoleta. El tiempo con el que se debe actualizar los datos depende del impacto y frecuencia con la que cambian los datos. por ejemplo, en el caso de los campos de *Know Your Customer*, *Device Status* o *Sim Swap* es posible que el tiempo de refresco pueda ser más grande ya que los clientes no cambian habitualmente de nombre, domicilio, SIM o entran continuamente en *roaming*. Por ejemplo, podría ser suficiente una actualización al día. En el caso de otras API como *Device Location* si que puede ser crítico el tiempo de actualización. Podría ser interesante actualizar cada minuto esta información.

Por último, esta arquitectura evita también tener que consultar a fuentes de información que quizás son sensibles y críticos como los elementos del CORE de la red. Habiendo comentado estos puntos, esta arquitectura implica un tiempo de desarrollo para incluir toda esta base de información en la base centralizada pero supone una gestión más eficiente.

3.1.2. Arquitectura de microservicios frente a arquitecturas monolíticas.

Una vez se ha definido como va a ser la fuente de información, el siguiente paso es decidir como va a ser la arquitectura de los servicios que van a exponer las API. Un modelo tradicional es la arquitectura monolítica. En este tipo de arquitecturas, todos los componentes se construyen y despliegan en la misma aplicación.Por ejemplo, en el caso de las API de *Open Gateway* que se van a implementar, sería un único programa que incluyese distintos *endpoint* para cada una de las API. Esto supondría algunas ventajas como es la reutilización de código, mayor facilidad para la gestión de errores, facilidad para la gestión de *logs* o la facilidad del despliegue ya que únicamente se despliega una aplicación.

Por otra parte, esta arquitectura también presentaría una serie de desventajas. Cualquier cambio en la aplicación supone un esfuerzo mayor debido a que el programa es más grande y a la interconexión de sus componentes. Por otro lado, a medida que crece el programa es más complicado añadir cambios. Se muestra a continuación un diagrama con la solución monolítica.

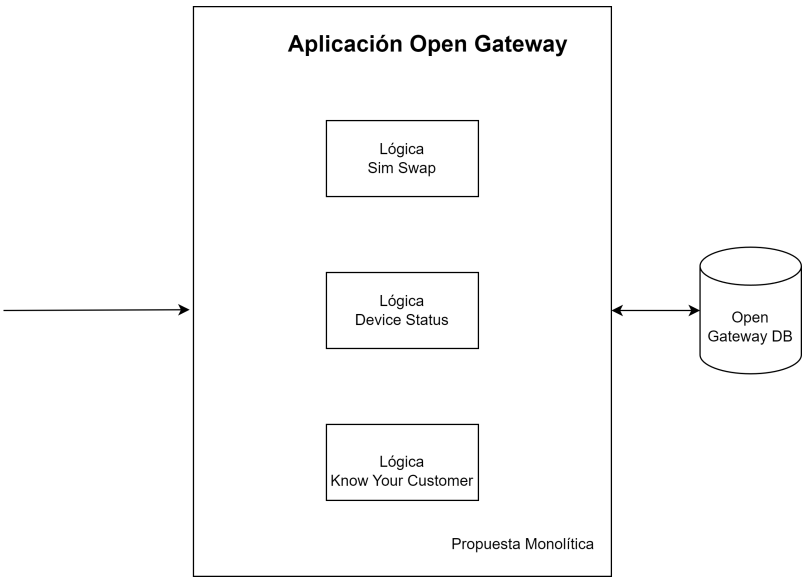


Figura 3.3: Diseño basado en arquitectura monolítica.

En contra partida, se encuentras las arquitecturas basadas en microservicios. Este tipo de arquitecturas fragmentan el programa en múltiples servicios que realizan una funcionalidad específica. Siguiendo con el ejemplo que aplica a este trabajo, se tendría una aplicación por API. Este modelo conlleva grandes ventajas aunque también cuenta con desventajas.

Al tener componentes independientes, se puede actualizar y realizar modificaciones a cada una de las aplicaciones sin impactar el funcionamiento del resto. Cualquier fallo o error en uno de los servicios va a tener únicamente impacto en la funcionalidad de ese servicio y el resto seguirán funcionando correctamente. Por otra parte, al tener tamaños menores el código es más legible y fácil de comprender. En términos de escala, si un API necesita recibir un mayor número de peticiones, no es necesario que escalen todas las aplicaciones sino que basta con un aumento en el servicio oportuno. Por último, dado que las aplicaciones están desacopladas, sería posible tener aplicaciones hechas en diferentes lenguajes de programación.

Como contrapartidas, tener más aplicaciones conlleva una gestión más compleja. A la hora del despliegue, el administrador debe desplegar no una aplicación sino que varias. Para el tema de monitorización o recolección de *logs* también supone una dificultad tener múltiples aplicaciones. Se añade también en este caso una complejidad extra para hacer el *testing* ya que se tiene que interactuar con múltiples servicios [40].

Habiendo comentado estos dos tipos de arquitecturas, es importante elegir el diseño de nuestros servicios que mejor se ajuste a las necesidades del proyecto. La tendencia de la actualidad es ir a microservicios debido a las ventajas que ya se han comentado. Sin embargo, este tipo de arquitectura busca desacoplar totalmente los servicios entre sí y se suele proponer trabajar con una base de datos por microservicio. Esto claramente

choca claramente con el primer punto del diseño que es tener una base de datos centralizada. por este motivo, la arquitectura elegida va a tender a una arquitectura de microservicios pero con un enfoque algo distinto y que va a diferir con las arquitecturas de manual.

Por otro lado, siempre es un punto interesante si se quiere trabajar con entornos de microservicios, pensar en cuanto se debe fragmentar la aplicación. Es decir, en el ejemplo que se ha puesto previamente se ha supuesto un servicio por API que se quiera exponer pero podría no ser la única opción. Alguno podría pensar en hacer un servicio por cada *endpoint* del API dado que pueden existir API con muchos *endpoint*. Por otro lado, dado que van a compartir la base de datos, se podría incluir un servicio de acceso a base de datos que se encargase de la comunicación con la misma [40].

Para el caso de las API que se quieren implementar, no es el caso que tengan muchos *endpoint* por lo que esta opción se descarta ya que implicaría tener que gestionar en lugar de tres servicios, cinco sin tener claro el beneficio que aportaría. En el caso de crear un servicio de acceso a las bases de datos, esto supondría un error ya que ligaría totalmente los servicios a una base de datos única creando una interdependencia. En el caso de que cada una implemente su propia conexión a la base de datos permite que en un futuro se pueda cambiar fácilmente sin tener que usar la base centralizada.

Por lo tanto, se va a decidir implementar una arquitectura en la que se tiene una base de datos centralizada y que tiene un servicio por cada API que se va a implementar. Para algunos esta arquitectura será una arquitectura basada en microservicios y para otros no. De igual manera, a efectos prácticos la arquitectura presenta mayoritariamente las ventajas y desventajas de una arquitectura basada en microservicios. Se muestra a continuación el diseño propuesto:

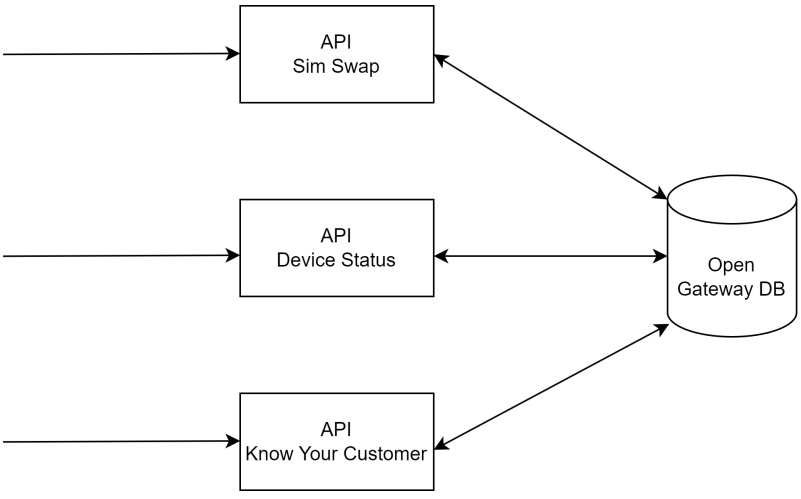


Figura 3.4: Diseño basado en arquitectura de microservicios.

3.1.3. Seguridad en la exposición de API.

Uno de los puntos más comunes a tratar a la hora de exponer los servicios en forma de API es la seguridad. Se debe tener un control de quien accede a los recursos mediante procesos de autenticación y autorización. Para poder tener este control, se suele utilizar un servidor de autorización. El servidor de autorización es el responsable de habilitar o no el acceso a los recursos protegidos. Pero el servidor de autorización no es el único actor en un flujo de autorización.

Existen muchas maneras y mecanismos de proteger los recursos expuestos en la web a través de API. Sin embargo, uno de los mecanismos que más se utiliza es el flujo de autorización *Oauth* 2.0. Este flujo viene estandarizado en el RFC 6749 del IETF (*Internet Engineering Task Force*). En este trabajo ya se ha hablado de diversos estándares como la GSMA para las API *CAMARA* y *TM Forum* o el 3GPP para las redes móviles. En el caso de IETF, es una organización de carácter internacional cuyo objetivo es desarrollar y promover estándares voluntarios y protocolos para el Internet, con el propósito de mejorar su funcionalidad y accesibilidad. Esta organización no tiene miembros formales, lo que significa que cualquier persona interesada en los procesos de desarrollo de estándares de Internet puede participar en las actividades del IETF.

Para definir los estándares, se organizan grupos de trabajo que se centran en resolver problemas específicos. Los resultados se recogen en los RFC *Request for Comments*. Aunque los estándares del IETF no son obligatorios, en el caso de la especificación del flujo *OAuth* 2.0 es ampliamente utilizado por la industria. Para comprender bien la especificación se muestra el siguiente diagrama:

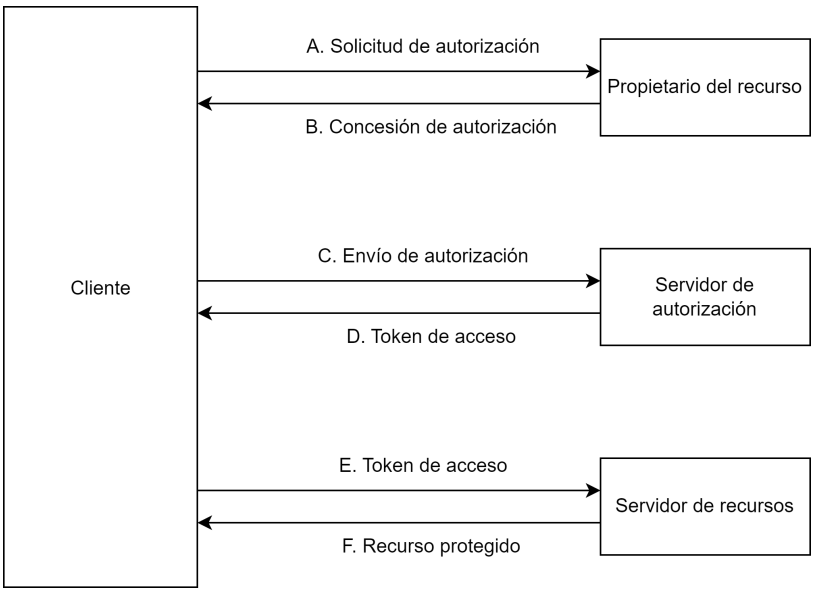


Figura 3.5: Flujo OAuth 2.0.

Los actores involucrados en este flujo son el cliente, el propietario del recurso, el

servidor de autorización y el servidor de recursos. En el ámbito de este trabajo, el cliente serán todos aquellos desarrolladores y empresas que quieran utilizar las API de *Open Gateway*. El propietario del recurso es la operadora que implementa las API. El servidor de autorización debe ser diseñado para cumplir con el flujo. Por último, el servidor de recursos es el servidor que aloja las API.

La explicación de los distintos pasos del flujo de la figura 3.5 es la siguiente:

- A. Solicitud de autorización: Este es el primer paso y consiste en que el cliente obtenga los credenciales necesarias para poder acceder a los recursos.

- B. Concesión de autorización: Si la entidad que solicita el acceso a los recursos protegidos debe tener acceso, el propietario del recurso le concede la autorización, dándole los credenciales de usuario. Como se verá más adelante hay varios tipos de credenciales que se pueden dar a los clientes.

- C. Envío de autorización: Con los credenciales obtenidos, se envía una solicitud de acceso al servidor de autorización. El servidor de autorización se encargará de verificar que esos credenciales son válidos. En caso de que sean válidos, se devolverá un token de acceso.

- D. Token de acceso: El servidor de autorización devuelve el token de acceso en caso de que la autorización haya sido exitosa.

- E. Token de acceso: El cliente envía el token de acceso al servidor de recursos.

- F. Recurso protegido: El servidor de recursos protegidos verifica que el token de acceso es correcto y permite al cliente obtener los recursos, en el caso del trabajo, acceso a las API.

Para securizar el acceso de nuestras API, se empleará por lo tanto, el flujo *OAuth* 2.0 descrito previamente. Esto implicará implementar al menos un servidor de autenticación y configurarlo correctamente. En el flujo que se espera, las peticiones para obtener los recursos protegidos de las API, deberán incluir un token de acceso. El cliente deberá obtener el token de acceso previamente del servidor de autorización.

Los pasos descritos en los puntos A y B no se tendrán en cuenta ya que se dará por supuesto que el cliente ya ha obtenido las credenciales de acceso (*client_id* y *client_secret*). Los credenciales de acceso deben estar correctamente configurados en el servidor de acceso. Por otra parte, los servicios que implementan las API deberán contar con mecanismos para validar el token de entrada con el servidor de acceso.

De esta manera, la arquitectura en la que se va a trabajar se representa en la figura 3.6

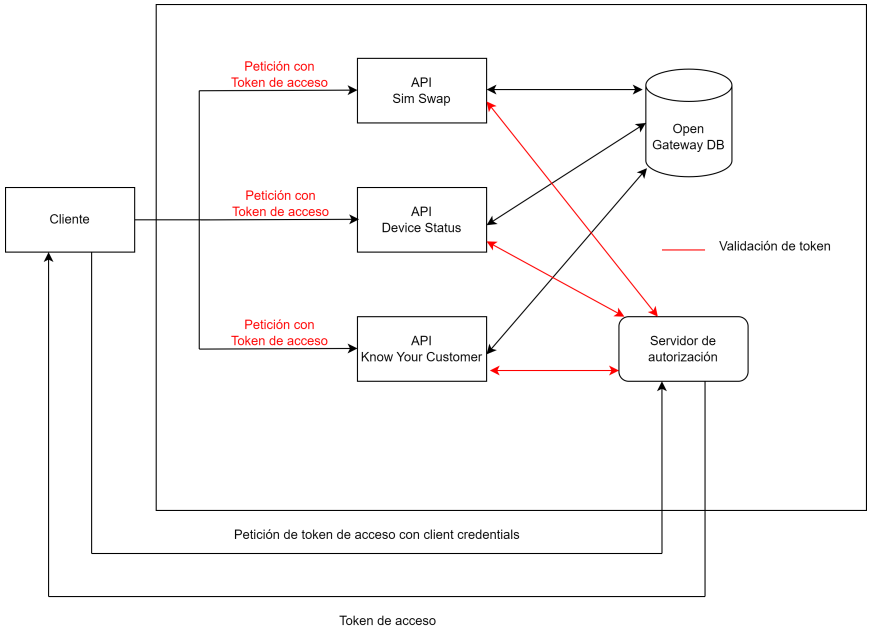


Figura 3.6: Diseño basado en arquitectura de microservicios con servidor de autorización.

Para que quede de alguna forma más clara la interacción entre los distintos elementos, se ilustra también un diagrama secuencial UML:

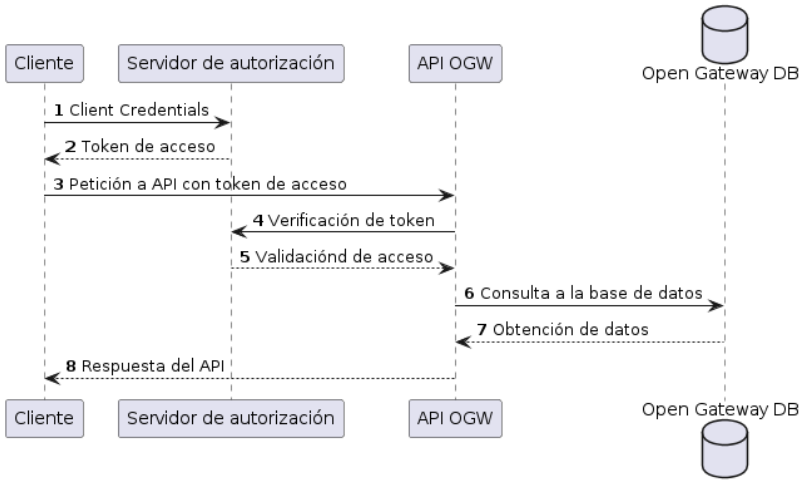


Figura 3.7: Diagrama secuencial UML del Diseño basado en arquitectura de microservicios con servidor de autorización.

Con este mecanismo, se tiene una arquitectura en la que se tiene un control de acceso a las API de *Open Gateway* mediante mecanismos de autorización.

3.1.4. El rol del API GW en arquitecturas de microservicios.

Una vez se ha llegado a la arquitectura de la figura 3.6, uno se pregunta como puede mejorarlo. A simple vista parece que la arquitectura no es muy eficiente ya que el cliente para acceder a las tres API tiene que conocer la dirección del servidor de autorización y posteriormente la dirección de cada una de las API. A medida que van creciendo el número de API esta situación parece algo insostenible ya que complica mucho la experiencia del cliente y es posible que busque otros proveedores.

Por otro lado, también parece un poco tedioso en la arquitectura que cada servicio tenga que estar validando con el servidor de autorización el token de acceso. De nuevo, a medida que crecen las API y los microservicios correspondientes, la situación es más compleja. Aunque este problema no afecta directamente a la experiencia del cliente, la gestión para la operadora sería muy difícil replanteando volver a una solución monolítica que simplificase este proceso.

Es aquí donde aparece lo que se conoce como *API Gateway* (API GW). El API GW simplifica mucho las arquitecturas de microservicios ya que proporciona un único punto de entrada a las API. Este elemento soluciona los problemas descritos anteriormente. En primer lugar, el cliente solo debe conocer la dirección del API GW y es este quien se encarga de enrutar las peticiones a los servicios correspondientes de una manera transparente al cliente. Esto es un gran beneficio. Por otro lado, al ser el punto de acceso común a las API, el API GW se puede encargar directamente de realizar la validación del token de acceso con el servidor de autorización. Esto soluciona el problema de gestión de raíz ya que desacopla los mecanismos de seguridad como son la autorización y autenticación de la lógica del servicio a implementar. Esto quiere decir que aunque crezcan el número de servicios, la complejidad de la gestión de la seguridad se mantiene constante.

Además de las ventajas mencionadas previamente, ofrece también otras muchas cosas. En el caso de que los servicios se desplegasen, por ejemplo, en un cluster de *Kubernetes*, el API GW puede hacer el papel de balanceador de carga, repartiendo el tráfico entre las diferentes instancias del servicio. Por otro lado, al ser punto de entrada único, ofrece capacidades de control de tráfico y *throttling*, pudiendo limitar el número de solicitudes hace cada cliente a cada API. Esto previene ante sobrecargas, abusos de servicio o ataques DDoS (*Distributed Denial of Service*).

Por otro lado, también puede realizar funciones de *caching* almacenando respuestas a solicitudes comunes en caché. Esto reduce los tiempos de respuesta para los cliente. Por destacar una última ventaja de las que este sistema ofrece, permite realizar transformaciones a las peticiones y respuesta. Esto puede ser por ejemplo la combinación de respuestas de dos API para unificarlo en una única respuesta al cliente, refactorización de parámetros en los cuerpos de las API u otro tipos de transformaciones.

Con todas estas ventajas en mente, se evoluciona la arquitectura incluyendo un API GW para centralizar las peticiones. La nueva arquitectura es la siguiente:

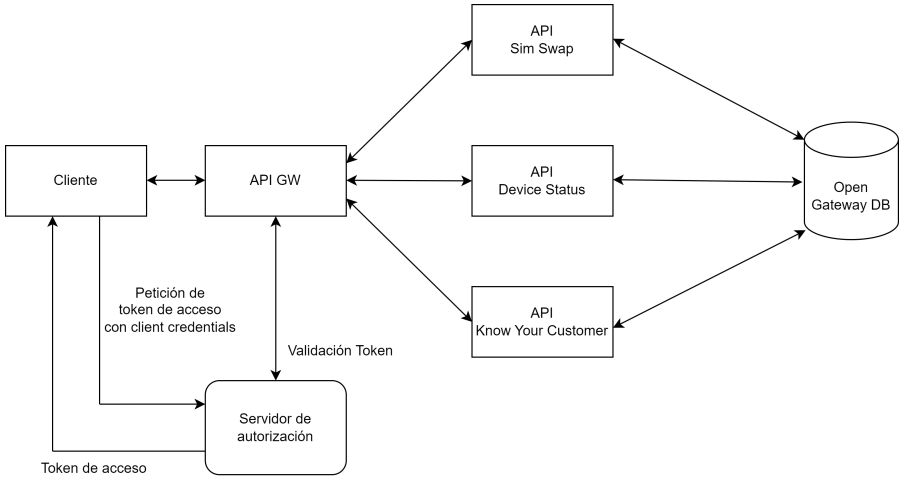


Figura 3.8: Diseño basado en arquitectura de microservicios con servidor de autorización y API GW.

De nuevo, se incluye también un diagrama de secuencia UML para aportar mayor calidad al flujo de información.

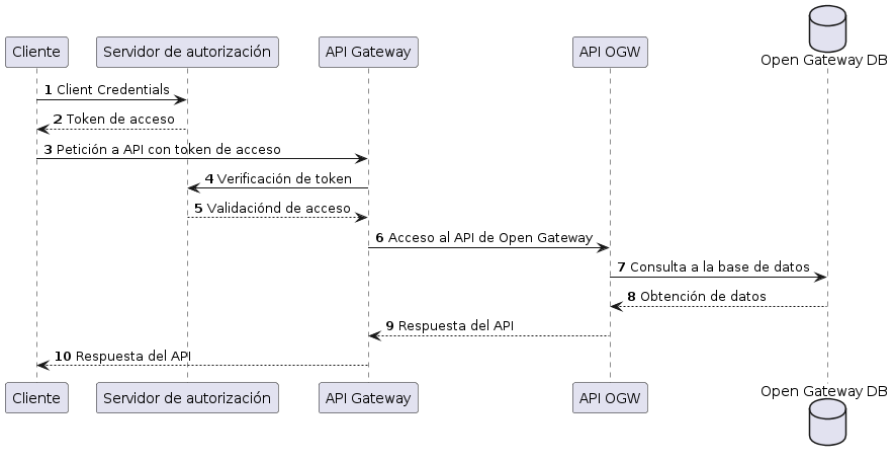


Figura 3.9: Diagrama secuencial UML del Diseño basado en arquitectura de microservicios con servidor de autorización y API Gateway.

Con esta nueva arquitectura, se sigue teniendo un entorno con control de acceso pero además de una forma óptima, teniendo un único punto de entrada, facilitando el acceso de los clientes y mejorando la gestión de la validación del token de acceso.

3.1.5. Sistema de monitorización para la gestión de *Performance Management.*

Otro de los temas más relevantes para la gestión de entornos de microservicios es poder tener una visión generalizada y unificada del rendimiento independientemente del número de microservicios desplegados. La visión del rendimiento se refiere poder ver por ejemplo cuantos recursos están consumiendo los servicios (Consumo de CPU, memoria utilizada, carga media, etc), los tiempos de respuesta, que *endpoints* y API se usan más, el número de peticiones por segundo y otros datos de interés.

Esta visión permite al administrador de sistemas detectar cuellos de botella en las distintas métricas. Por ejemplo, en el caso de las API que se van a implementar, se podría detectar a través del sistema de monitorización si existen comportamientos anómalos en las respuestas del API, monitorizando cuales son todos los códigos de respuesta devueltos. Por ejemplo, si se devuelven muchos códigos de error es posible que se deba revisar la implementación o el estado del servidor.

Otro ejemplo, monitorizando los tiempos de respuesta se puede observar cuales son los sistemas que tienen mayor impacto en el flujo, pudiendo optimizar los puntos de mejora. A nivel de capacidad, podrían ser estas métricas la que indiquen que hay que aumentar el número de nodos donde se despliega la solución.

El sistema de monitorización es útil también para detectar tendencias. Esto significa que es posible que a determinadas horas del día haya más carga que en otras. Por ejemplo, en horas nocturnas habrá menos peticiones. Este caso es un poco evidente pero con el sistema de monitorización se podrían identificar patrones que no son tan vistosos. Como resultado de esta detección, se puede realizar una mejor planificación de las capacidades necesarias para servir los recursos.

Para poder hacer esta gestión unificada de todos los servicios de la arquitectura, es necesario que cada servicio exponga las métricas que se requieran. Este punto volverá a salir en la implementación de este sistema ya que es un punto relevante como obtener los datos. Además, si se consigue que los servicios expongan los datos, cada cuanto los exponen es también crucial. Si el sistema de monitorización no tiene datos actualizados, el sistema empieza a perder valor. Esto se ve claro con un ejemplo, si se detecta un fallo ahora con el sistema de gestión pero los datos eran de hace un día, es posible que ninguna de las acciones que se puedan tomar vayan a satisfacer los problemas que se hayan podido generar a los clientes. Por otro lado, una actualización con mucha frecuencia puede sobrecargar el sistema. En definitiva, hay que buscar un balance de tiempos que permita reaccionar rápido en caso de fallos o anomalís pero tampoco demasiado pequeño de modo que pueda impactar en el rendimiento del API.

Para poder hacer esto, se propone que los servicios que implementan el API expongan como mínimo métricas como el uso de la memoria, el uso de la CPU, tiempo y conteo de la solicitudes, etiquetas con el estado de respuesta. En la sección de

implementación se vera como y con que tecnologías implementar esta monitorización para facilitar la gestión de los servicios.

3.1.6. Centralización de logs para la gestión de *Fault Management.*

Aunque la arquitectura incluye ya un sistema de monitorización y gracias a él es posible detectar problemas en las aplicaciones, muchas veces no es suficiente para tener trazabilidad de lo que está ocurriendo internamente en la aplicación y donde pueden estar los errores. Para poder tener este detalle interno de lo que ocurre en las aplicaciones, se suelen usar los registros de *logs*.

Los registros de *logs* son algo que lleva presenta en la programación desde los inicios. Su funcionalidad y uso es muy claro, estos registros permiten introducir mensajes en la ejecución de los programas para poder detectar posibles puntos de fallo en la ejecución. También es una forma muy útil de debuggear el código aunque quizás la más rudimentaria comparado con otras soluciones que ofrecen los IDE (*Integrated Development Environment*) más modernos.

Otra de las utilidades que tienen los *logs* es poder verificar y asegurar que se cumples ciertos requisitos legales o de políticas internas en caso de auditoría. Sin embargo, como se ha comentado el punto más importante es el de poder analizar el sistema y saber que ha ocurrido en caso de fallos.

Una vez que se ha identificado la utilidad de los *logs* en las aplicaciones, la pregunta es como hacer la gestión de *logs* en una arquitectura de microservicios donde cada API y sistema emite sus propios registros. Como ya se comento, este es uno de los puntos claves a resolver ya que en caso de no poder gestionarlo correctamente, la idea de ir a un modelo monolítico coge más fuerza. Al fin y al cabo solo habría que preocuparse de los *logs* de una única aplicación.

Por lo tanto, el objetivo es incluir en la arquitectura un servidor que centralice y almacene los *logs* de todas las API. Para ello, cada una de las API tendrá que escribir los *logs* no solo en el terminal sino en un fichero o en una base de datos centralizada que permita tener la visión entera de lo que ocurre en la arquitectura. Para poder tener esta visisibilidad, el API tendrá que escribir en todos los *logs* algunos campos relevantes como la fecha y hora, el nombre del hilo de ejecución desde donde se generó el *log*, el nivel de severidad del *log* y el mensaje del *log* propiamente dicho.

Con todos los mensajes de log que se recopilan de todas las API, se deberá adaptar y transformar con el formato adecuado para poder gestionarlos de una forma adecuada. Como se verá en la sección de implementación, existes herramientas y tecnologías que permiten hacer esta transformación, almacenar los logs y visualizarlos de una manera sencilla para un administrador.

3.1.7. Resumen del diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway.*

En este apartado se pretende realizar un breve resumen del diseño de la arquitectura. La arquitectura diseñada se puede observar en la siguiente figura:

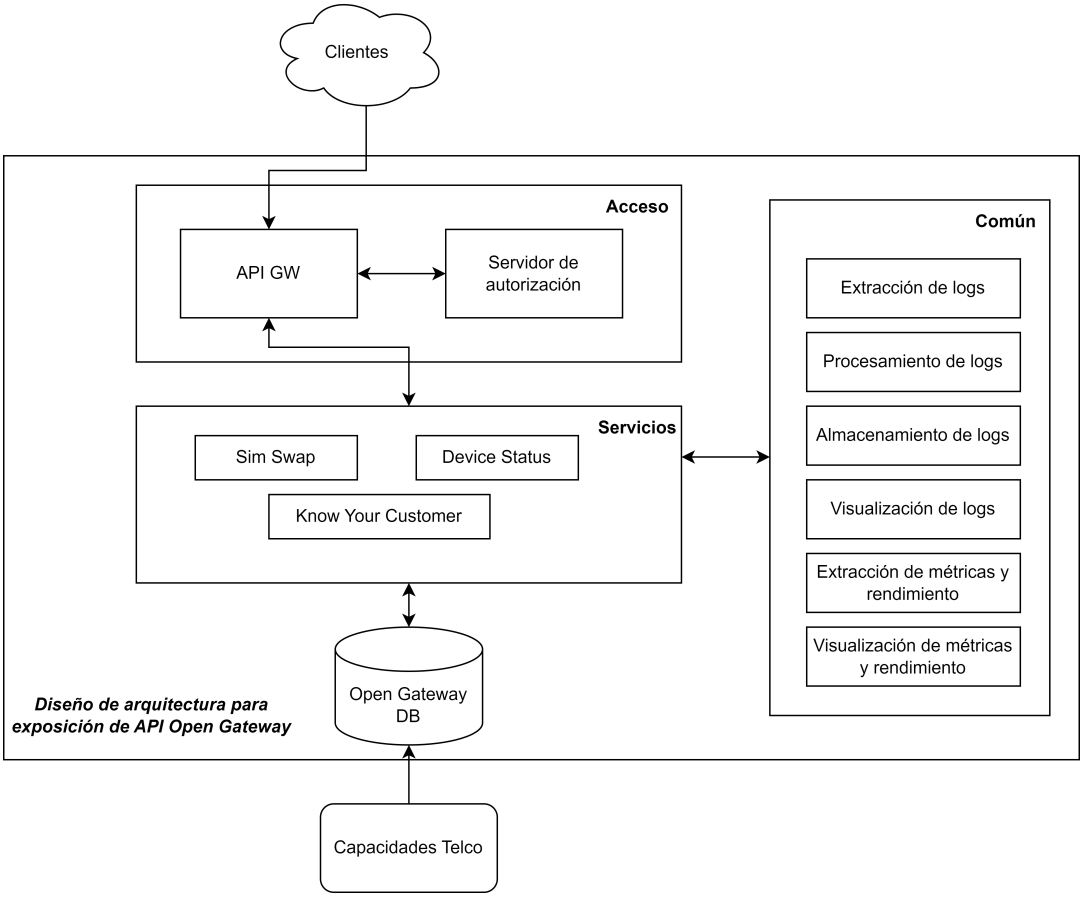


Figura 3.10: Diseño de la arquitectura basada en microservicios para la exposición de API en *Open Gateway.*

En la figura se muestra el API GW cuyo objetivo es centralizar las peticiones hacia las API y gestionar junto al servidor de autorización el acceso a los servicios. El API GW también puede servir para limitar el tráfico de los usuarios. Por otra parte, se han representado las plataformas comunes en todos los servicios. Estos elementos permiten principalmente la recolección y visualización de las métricas y *logs* de los servicios. El objetivo es claro, una empresa debe tener control de qué esta ocurriendo en sus producto, los recursos que necesita y si existen fallos. Por último se puede apreciar la base de datos centralizada para el proyecto que se nutre de los diferentes sistemas que se pueden encontrar de las operadoras de telecomunicación. De esta manera, se evita el acceso a múltiples sistemas que pueden tener diferentes características, limitando el rendimiento.

3.2. Implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway.*

En esta sección se va a explicar todo lo que se ha desarrollado para obtener la arquitectura que se ha diseñado en la sección anterior. Antes que nada, es importante mencionar que todo el código necesario para desplegar la solución se encuentra en un repositorio de *GitHub* [41].

Como primer paso en la implementación, se va a decidir en que lenguaje se van a programar las API. Para programar las API, se ha decidido utilizar el *framework* de Java *Spring Boot*. La razón para ello ha sido el conocimiento ya adquirido sobre este lenguaje de programación, la gran comunidad que hay dando soporte a este *framework*, las frecuentes actualizaciones que hay con correcciones de seguridad o nuevas características y la gran cantidad de dependencias bien documentadas y fácil de incorporar que hay.

Por otra parte, aunque se comentará más adelante, la solución se va a implementar de una manera natural utilizando contenedores. Es natural ya que manejar despliegues de este tipo de arquitecturas con varias aplicaciones y herramientas que desplegar sin utilizar contenedores es realmente complejo. Por otra parte, se aprovechan las ventajas que estos ofrecen como el rápido despliegue, la facilidad de despliegue en cualquier entorno independientemente del sistema operativo anfitrión o la facilidad para parar la ejecución y aplicar cambios.

Aunque existen varias tecnologías para desplegar contenedores, en este caso se utilizará *Docker compose* para hacer un primer despliegue en local y posteriormente, se implementará la solución en *Kubernetes*. Esto permitirá que la solución final pueda ser desplegada en cualquier *cluster*, aprovechando las características de replicación, resiliencia, tolerancia a fallos, etc.

Se indagará en los detalles del despliegue según se vaya explicando los diferentes puntos de al implementación. Aunque el despliegue con contenedores es independiente del sistema operativo anfitrión, se usará una maquina virtual con *Ubuntu* 22.04 por la comodidad que implica trabajar con el terminal y el sistema operativo de Linux. Para no tener problemas de rendimiento, se usará una máquina virtual con 20GB de RAM, 7 CPU y 100GB de almacenamiento.

3.2.1. Implementación de la base de datos *Open Gateway.*

Tal y como se ha explicado en la fase de diseño, el objetivo es tener una base de datos que centralice la información de todos los sistemas involucrados en *Open Gateway*. El objetivo es no perder rendimiento en la solución haciendo consultas a diferentes sistemas y sobre todo, no tener que lidiar con bases de datos y protocolos legados o propietarios.

El mundo de las bases de datos es muy amplio y existen de muchos tipos. En primer lugar se encuentran las clásicas bases de datos *SQL* o relacionales, algunas de las más famosas son *PostgreSQL*, *MySQL*, *SQlite*, *Oracle DB*, etc. Este tipo de base de datos proporciona una estructura con un esquema predefinido. Esto obliga a los datos a seguir un formato específico y uniforme, lo que facilita el mantenimiento del orden y la consistencia de los datos a lo largo del tiempo. Sin embargo, este tipo de bases de datos tiene principalmente como inconveniente la capacidad de escalar horizontalmente y gestionar tipo de datos no estructurados o semi estructurados.

Por otro lado, se encuentran las bases de datos NoSQL. Dentro de este gran grupo hay muchas variedades. Existen bases de datos de tipo clave valor como *Redis*, bases de datos orientadas a documentos como *Mongo DB*, otras de tipo columnas como *Cassandra* o incluso bases de datos de tipo grafo como Neo4J. No se van a repasar las propiedades de cada una de estas bases de datos ya que no es el objetivo. Si uno se fija en los datos que se pretenden guardar en la base de datos de *Open Gateway*, se da cuenta que las de clave valor y grafo no encajan con el objetivo.

Una vez llegado a este punto, se decide analizar las ventajas y desventajas que tendría usar una base de datos *SQL*, una *NoSQL* de columnas y otra *NoSQL* de tipo documento. Los datos que se van a guardar en ella para la implementación son los descritos en las *API* de *Sim Swap*, *Know Your Customer* y *Device Status*. Por resumir esta información, son datos que implican *timestamp* del último cambio de SIM, datos de la red móvil como el MSISDN, IMSI, MCC, MNC y datos personales como nombre, apellidos, DNI, dirección, género, código postal, etc.

Como se puede observar, este tipo de datos son datos estructurados que llevan a pensar claramente en utilizar una base de datos clásica *SQL*. Sin embargo, este proyecto implica no solo estas tres API a implementar sino que otras muchas definidas en *CAMARA* que no poseen este tipo de datos. Además, este tipo de servicios debe estar preparado para ser flexible y escalar horizontalmente de manera ágil ante incrementos de demanda. Por esta razón, se decide implementar la base de datos con una base de datos *NoSQL* orientada a documentos. De entre todas las opciones posibles, se elige usar concretamente *Mongo DB* ya que es *opensource* además el desarrollo será más fácil debido al conocimiento previo de esta base de datos.

Para el despliegue y puesta en marcha de *Mongo DB*, se va a utilizar directamente un contenedor *Docker*. Como breve repaso acerca de *Docker*, *Docker* es una plataforma de software que permite la creación, ejecución y administración de contenedores. Los contenedores son entornos de ejecución ligeros y portátiles que encapsulan todos los componentes necesarios para ejecutar una aplicación, incluyendo el código, las librerías, el sistema operativo y las dependencias. Esto facilita que las aplicaciones se ejecuten de manera consistente en cualquier máquina tal y como se explicó previamente.

No se va a entrar en la arquitectura y características concretas de *Docker* ya que no es el objetivo de este trabajo. Si que se mencionarán los conceptos relevantes para el desarrollo del trabajo. Es importante destacar que Docker cuenta con un repositorio oficial de imágenes, llamado *Docker Hub*. Como breve inciso, las imágenes y los contenedores de *Docker* se diferencian en que el contenedor es una instancia en ejecución de una imagen. *Docker Hub* servirá para obtener imágenes ya predefinidas con algunas de las herramientas que se usarán y además, va a permitir que las imágenes personalizadas que se creen, se encuentren en un sitio centralizado.

Por otro lado, la instalación de *Docker* en las nuevas versiones incluye también la instalación de *Docker Compose*. *Docker Compose* es una herramienta que va a permitir desplegar de manera simultánea varios contenedores. Esto se va a hacer a través de un fichero YAML. En este fichero YAML se especificará las características del despliegue del contenedor, como es la imagen a usar, el nombre que va a tener el contenedor, la red a la que se debe conectar, los puertos que debe exponer el contenedor, las variables de entorno, los volúmenes que usa, etc. Esta forma de despliegue es mucho más sencilla que especificando todas estas opciones en línea de terminal.

Una vez vistos los conceptos principales de *Docker* que se van a manejar, se procede a la instalación siguiendo los pasos de la web oficial para *Ubuntu* 22.04 [42]. El siguiente paso es saber que imagen utilizar. Para ello se busca la imagen oficial de mongo y una de las últimas versiones estables, por ejemplo, la v7.0.7. Con esto ya se puede desplegar *Mongo DB*. Ahora, el próximo paso es definir el modelo de información que se va a guardar en la base de datos y cuantos datos se deben guardar para que el funcionamiento de las API tenga algún sentido.

En base a estas dos preguntas, se decide realizar un *script* que genere un número dado de documentos tipo JSON con características y parámetros aleatorios. El *script* se puede encontrar en la carpeta *scripts* con el nombre *client_ generator.py* en el repositorio de *GitHub*. Como una primera aproximación, se considera que cien datos de clientes es una cifra lo suficientemente grande como para tener respuestas diversas en las API y que no es excesivamente grande como para aumentar la complejidad de la ejecución. En primer lugar, los campos que se necesitan son:

- MSISDN: El MSISDN (*Mobile Station International Subscriber Directory Number*) es el número que identifica de manera única a cada usuario de una red móvil. También conocido ordinariamente como número de teléfono. Este campo es esencial ya que es el parámetro de entrada para todas las API y que es la manera de identificar a los usuarios en *Open Gateway*. El MSISDN tiene un formato específico y que además viene ya especificado por una expresión regular en los *Swagger* de las API de *Open Gateway*. Esta expresión regular es `^\+?[0-9]{5,15}$` y básicamente especifica que es un *String* que debe estar formado como mínimo con cinco números y como máximo con 15. Para más detalle acerca del formato se puede visitar el TS 23.003 de la *release* 12 del *3GPP* [43]. Para poder generar cien números de teléfono aleatorios, se va a realizar una aproximación muy simple. Los números van a seguir una regla, se

van a generar en el rango [+34651481200 +34651481299] de manera ordenada, en cada iteración del *script* se incrementa en uno el número de teléfono. Esto es así para evitar repeticiones, ya que los MSISDN deben ser únicos.

- IMSI: El IMSI (*International Mobile Subscriber Identity*) es otro identificador de red móvil que identifica de manera unívoca una línea móvil. Cada tarjeta SIM, tiene asociado un IMSI. Cada vez que un usuario cambia de SIM, ya sea por robo o por pérdida de la SIM, recibe una nueva con un nuevo IMSI. Aunque un usuario cambie de operadora, su MSISDN sigue siendo el mismo, sin embargo, el IMSI será el asociado con la SIM de la nueva operadora. Esta campo es fundamental para la lógica de *Device Status* y además es importante para comprender los casos de uso de *Sim Swap*. Al no ser campo de entrada de ninguna de las API, no hay expresión regular que lo defina en *CAMARA*. Sin embargo, si visitamos de nuevo el TS 23.003 de la *release* 12 del 3GPP [43], vemos que el IMSI debe estar formado por no más de 15 dígitos y sigue la siguiente estructura:

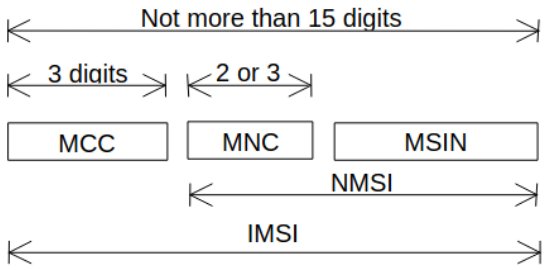


Figura 3.11: Formato del IMSI.

Es decir, el IMSI debe estar compuesto por el MCC (*Mobile Country Code*) que debe tener tres dígitos, el MNC (*Mobile Network Code*) que tiene entre dos y tres dígitos y el MSIN (*Mobile Subscriber Identification Number*) que es realmente el número que identifica al usuario. El MCC es un código que identifica el país y el MNC identifica al operador dentro de la red local. Por este motivo, operadores diferentes tienen distinto IMSI. Para conocer que códigos tienen los diferentes países y sus operadores, se puede realizar una búsqueda rápida en Internet [44]. El MCC de España es el 214 y el MNC de Telefónica, empresa donde trabajo, es el 07. Dado que el IMSI también debe ser único, se van a generar también siguiendo un orden en el *script*. El rango va a ser [214071234567800 - 214071234567899], se usará para todos los clientes el MCC de España y el MNC de Telefónica.

- MCC: Como se ha explicado previamente MCC es el acrónimo *Mobile Country Code* y es un código que identifica al país. Debe tener tres dígitos, se puede consultar en el enlace que se ha dado los diferentes códigos para los países existentes. Los códigos no siguen un orden del 000 a 999 sino que tienen algunas reglas. El valor del primer dígito indica la zona geográfica, esto es, si el primer dígito es un 2, es Europa. Si es un 3, es América del Norte, el 4 representa Asia, el 5 Australia y Oceanía, el 6 África, el 7 América del sur y central y el 9 para

el resto del mundo. Los dígitos 1 y 8 no se usan. Esto se especifica en el E.212 de ITU [44]. Se copian todos los posibles valores como una lista en el *script* y se elige uno de forma aleatoria pero dando más probabilidad a que sea 214 que es el de España. La razón es que la base de clientes pretende ser de una operadora de telecomunicaciones de España y lo lógico es que la mayoría no estén en *roaming*.

- MNC: El MNC (*Mobile Network Code*) es el código que identifica al operador dentro de los países. Debe tener entre dos y tres dígitos. Este campo realmente no va a ser estrictamente necesaria para ninguna API así que se dejará que tome un valor aleatorio. En el caso de que el MCC generado sea el de España si que se usará el MNC de Telefónica 07.

- *Cell id*: Este campo representa el número de celda. Estaba originalmente ideado para un API que finalmente no se ha implementado así que tomara simplemente un valor aleatorio entre 1 y 9999.

- *Given name*: Este campo representa el nombre del cliente. Parece un campo sencillo de implementar pero es difícil generar aleatoriamente *Strings* que formen un nombre con sentido en Español. Por este motivo, se ha definido un mapa con cincuenta nombres diferentes. Se escogerá aleatoriamente un nombre de este mapa. No importa que haya repetición de nombre. Por otro lado, aunque se explicará en el campo de genero, el mapa lo completa el genero implícito de cada nombre.

- *family names*: Este campo representa el apellido. Se va a proceder de la misma manera que con el nombre, se define una lista con cincuenta apellidos típicos españoles y se escoge uno aleatoriamente. Con esto y el nombre, se forma la identidad del cliente. Aunque es poco probable, puede ocurrir que se generen dos personas con mismo nombre y apellido. Cuantos más componentes añadiésemos en la lista menos probabilidades habría. De todas maneras, el número de teléfono y DNI seguirán siendo identificadores únicos.

- *Name*: Corresponde al nombre completo de la persona, concatenando nombre y apellido.

- *Id document*: Este campo representa el DNI de los clientes. Se genera de manera pseudo aleatoria ya que también tiene de manera fija los primeros dígitos 08359. Los tres siguientes dígitos son aleatorios y la letra correspondientes también. La letra se generan en mayúsculas. Esto se podría haber refinado un poco ya que oficialmente en España, se excluyen la I, la $\tilde{\text{N}}$, la O y la U.

- *email*: Este campo representa el *email*. Para generarlo simplemente se coge el nombre completo y se le añade un @gmail.com.

- *birthdate*: El campo representa la fecha de nacimiento del cliente. Para generarlo, se van a tener en cuenta únicamente personas con más de 18 y menos de 100 años. Esto es, nacidos entre 1924 y 2006, a partir de aquí, la generación es aleatoria.

- *Street name*: Indica el nombre de la calle. La estrategia es idéntica a la de los nombres y apellidos. Se ha creado una lista con una serie de nombre de calles de Madrid y se escogerá aleatoriamente un valor de esta lista. No importa la repetición del campo.

- *Street number*: Indica el número de la calle. Se genera simplemente un valor aleatorio entre 1 y 200.

- *House Number Extension*:Representa el piso donde vive el cliente. Se genera de manera aleatoria que el 80 % de los clientes vivan en una planta entre un primero a un tercero y con número de la A a la H y el otro 20 % se quedará el campo vacío suponiendo que el cliente no ha rellenado el campo. Con todas estas consideraciones, es posible que se de la circunstancia que dos clientes tengan la misma dirección (Nombre de calle, Número de calle y piso). Sin embargo, la probabilidad es baja. Se podría añadir una condición para comprobar el caso y desecharlo.

- *Postal Code*: Representa el código postal de donde vive el cliente. Dado que los clientes solo se tendrán en cuenta en Madrid, se han elegido solo códigos postales de Madrid. los códigos de Madrid empiezan todos por 28. Por acotar un poco más el rango, se usarán solo códigos postales de la ciudad de Madrid y no de toda la Comunidad por lo que se generaran únicamente comenzando por 280. Se genera de manera aleatoria los dos valores posteriores. Se podría haber afinado más ya que los códigos de la ciudad de Madrid no cubren todos los números del 28000 al 28099.

- *Region*: Representa la región donde vive el usuario. Como ya se ha comentado solo se tomará en cuenta clientes de Madrid.

- *Locality*: Representa la localidad donde vive el usuario. Como ya se ha comentado solo se tomará en cuenta clientes de Madrid.

- *Country*: Representa el país donde vive el usuario, que evidentemente, será España.

- *Latest Sim Change*: Representa la fecha de la última vez que un MSISDN cambió de SIM. Esta es la información relevante para el API de Sim Swap. Se genera aleatoriamente un año entre 2020 y 2024 . En el caso de que salga 2024, no se consideran fechas del futuro y solo se han considerado fechas hasta Abril de 2024. Para el resto de casos, se genera un mes día y hora aleatorios.

- *Titular*: Este campo es quizás el más complejo de explicar pero que cobra mucha importancia en el API de *Know Your Customer* tal y como se verá más adelante. El campo es un booleano que representa si el cliente es titular de cuenta o no. Esto es debido a que una operadora de telecomunicaciones tiene información de red (MSISDN, IMSI, MCC, MNC, etc) de todos sus usuarios sean los titulares o no, pero solo tiene los datos personales (nombre, apellido, DNI, fecha de nacimiento, etc) de los titulares de las cuentas. Por esta razón, se ha hecho

en el *script*, que se haya 75 clientes que vayan a ser titulares, y que los otros 25 no. Esto implicará que esos 25 cliente atendrán sus propios atributos de red pero compartirán los datos personales de algunos de los titulares. Esto representaría, por ejemplo, familiares del titular.

Los clientes generados con este *script*, se pueden encontrar de igual manera en *GitHub* en la carpeta *Scripts* con el nombre *generated_ clients.json*.

Para incluir esto en la base de datos de *Mongo DB*, se tienen dos opciones. La primera es importar los datos al arrancar el contenedor con la imagen oficial de Mongo. La segunda opción, es crear una imagen personalizad en la que se tenga ya una base de Mongo con los datos pre cargados. La segunda opción es la más valida y la que se va implementar ya que la primera puede generar ciertos problemas con el tiempo que tardan en importarse los datos.

Para implementar la segunda opción, se va a crear un *Dockerfile*, en el que a partir de la imagen oficial de Mongo (v7.0.7), se cargan los datos del archivo JSON en la imagen y a través de un *shell script*, se importan en la base de Mongo. En el *shell script*, se define la tabla y colección dentro de mongo, que son *telecomclients* y *client_ muestra* respectivamente. El contenido de este *Dockerfile* y el *shell script* se encuentran en /docker/mongo dentro del repositorio. Se muestra a continuación, una captura en el terminal, accediendo al contenedor de Mongo y ejecutando algunas consultas para comprobar la correcta carga de los clientes.

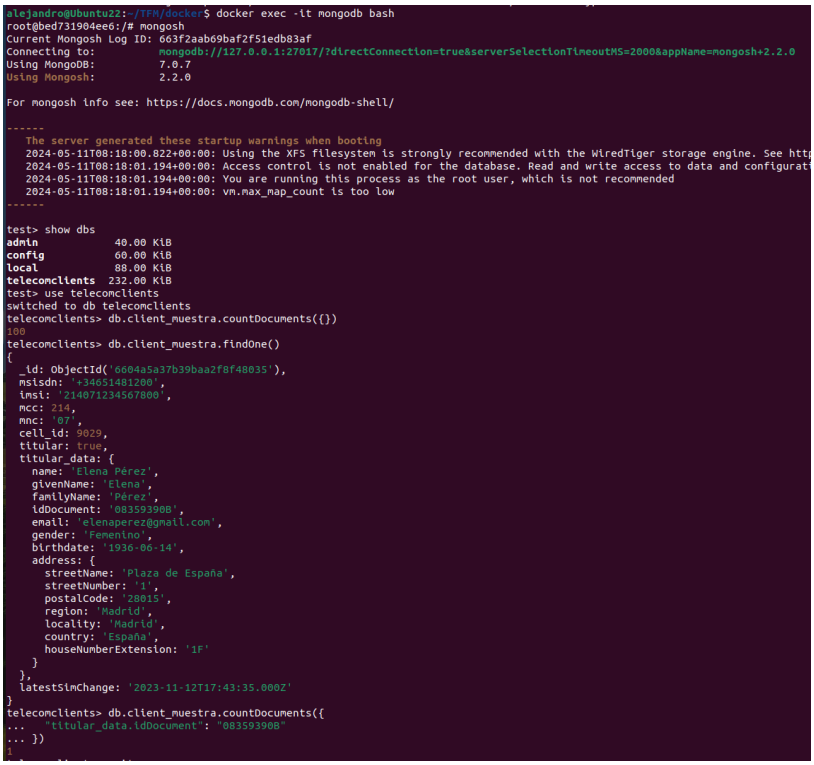


Figura 3.12: Captura de Mongo DB.

Por último, se ha añadido un volumen compartido para que los cambios que se hagan durante la ejecución, se queden persistentes en local. Ya se verá en la migración a *Kubernetes* como seguir persistiendo la información incluso en un *cluster*. Además, para hacer pruebas se expone el puerto de *Mongo DB* dentro del contenedor 27017 al mismo puerto en el ordenador local. Esto no es necesario para el funcionamiento ya que como veremos se utilizará la red de *Docker* para lo comunicación entre los distintos contenedores. Sin embargo, esta exposición se ha utilizado en el ambiente de desarrollo. El manifiesto de *Docker compose* se puede encontrar en la carpeta *docker* del repositorio.

3.2.2. Implementación API de provisión y gestión de usuarios.

En esta sección se va a explicar como se ha implementado un API para la provisión y gestión de usuarios en la base de datos de *Open Gateway*. El objetivo es tener una manera sencilla de manejar la base de datos y no tener que interactuar directamente con el contenedor. Por otro lado, servirá como punto de partida y aprendizaje para implementar las API de *Open Gateway*. El objetivo es que este API permita consultar, crear, manipular y eliminar usuarios de la base de datos.

Para implementar el API, como se ha comentado previamente se va a utilizar el *framework* de Java *Spring Boot*. Para comenzar los proyectos de *Spring Boot*, existe una página llamada *Spring Initializr* que crea un esqueleto base para el proyecto [45]. Desde aquí se puede elegir el lenguaje a utilizar (Java en este caso), el gestor de paquetes (Se va a escoger *Maven*), la versión de *Spring Boot* y el nombre del grupo, artefacto y del propio proyecto. Se escoge también la versión de Java utilizada que es la versión 17. Por otro lado, otra de las grandes ventajas de esta página es la sección de dependencias. En esta sección se pueden incluir todas las dependencias que vayan a ser necesarias con un solo clic.

Por el momento, se añade la dependencia con Mongo para poder realizar la conexión con la base de datos, la dependencia de *starter web* que sirve para desarrollar aplicaciones web y servicios REST, la dependencia de Lombok que se utilizará para facilitar el desarrollo evitando tener que generar algunas partes de código como *getter* y *setter*, la dependencia de *starter test* que agrupa varias bibliotecas útiles para realizar pruebas en aplicaciones *Spring Boot*, incluyendo *JUnit*, *Spring Test*, *Spring Boot Test*, *Mockito*, etc. Por otra parte, se incluye una dependencia de validación que permitirá establecer algunas reglas en el modelo y por último las dependencias de *micrometer* y *prometheus* que servirán para exponer las métricas de monitorización.

A partir de aquí, para realizar todas las API de este trabajo se va seguir la siguiente estructura de paquetes:

- *controller*: En este paquete se van a crear los recursos necesarios para exponer los *endpoint* de las API que se quieran crear. Es la parte que recibirá las peticiones y devolverá el resultado del API. Aquí se define si el tipo de método para cada una de las API (*GET*, *POST*, *PUT*, *DELETE*).

- *service*: En este paquete se crean los recursos necesarios para realizar la lógica del servicio para cumplir con la funcionalidad del API. Esto puede ir desde transformaciones hasta otro tipo de lógicas más complejas.

- *repository*: En este paquete se crean los recursos que van a interaccionar con las bases de datos fuentes de la información. En el caso de este proyecto se define una interfaz para interactuar con la base de *Mongo DB*.

- *model*: En este paquete, se van a crear los recursos que representan los diferentes objetos necesarios en el API. El más evidente y que se usará en todas las API son clases para representar el modelo de cliente que se obtiene de las consultas de *Mongo DB*.

- *exception*: En este paquete, se van a crear los recursos necesarios para que el API devuelva las excepciones que se piden en la especificación. En el caso de el API de provisión como no es una API estandarizada no hay ninguna obligación pero se implementarán de igual forma.

- *security*: En este paquete, se crean los recursos necesarios para cubrir el modelo de seguridad. Este paquete no tiene ya ninguna funcionalidad ya que en un principio, se introdujo la seguridad del entorno verificando en cada servicio el token de acceso con el servidor de autorización utilizando *Spring Security*, siguiendo la arquitectura de la figura 3.6. Dado que se introduce el API GW y se va a la arquitectura de referencia 3.8, este paquete deja de tener sentido.

Por lo tanto, el patrón que se va a seguir es el siguiente:

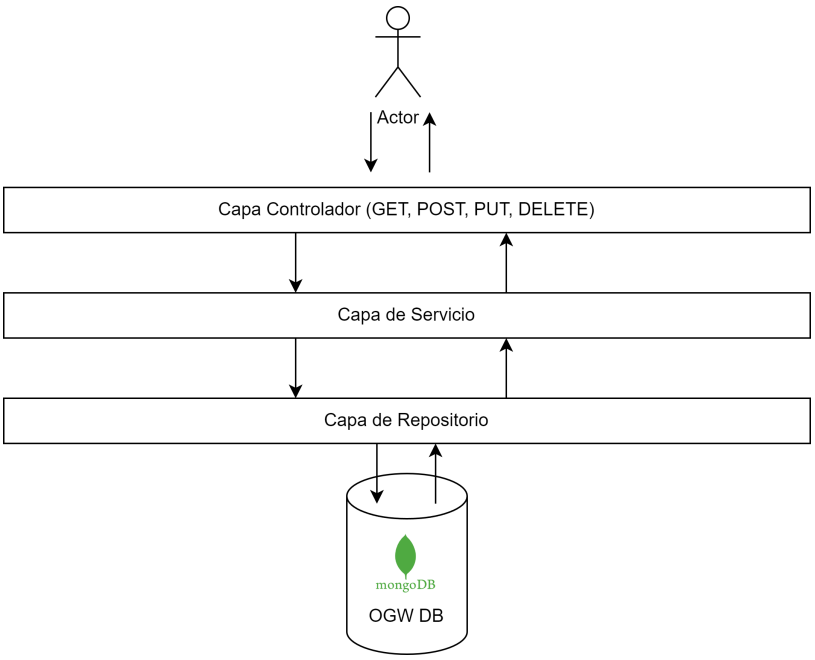


Figura 3.13: Modelo de arquitectura para implementación de las API.

A nivel de implementación, no se va a explicar todo el código pero se comentaran los aspectos más relevantes. En primer lugar, en la clase de controlador se van a crear nueve *endpoints* distintos. Cuatro de estos *endpoint* será para hacer consulta de datos (*GET*), uno para crear nuevos clientes (*POST*), otro para modificarlos (*PUT*) y otros tres para eliminar los clientes (*DELETE*). Para los de tipo *GET*, se tiene un *endpoint* que obtiene todos los usuarios y los otros tres obtienen el usuario especificado por *email*, DNI o número de teléfono. En el caso de DNI o *email*, la consulta puede devolver varios resultados ya que como hemos visto anteriormente, no se tiene la información de un cliente no titular. Para estos casos, la base de datos guarda la información únicamente del titular.

Se muestra a continuación una tabla de resumen con los *endpoint* y una breve descripción de cada una de ellas.

Método	***Endpoint***	**Descripción**
GET	/provision/v0/getall	Se obtienen todos los clientes de la base de datos.
GET	/provision/v0/getclient/msisdn/{msisdn}	Se obtiene el cliente que tiene el número de teléfono igual al proporcionado en la URL.
GET	/provision/v0/getclient/email/{email}	Se obtienen los clientes que tienen el email igual al proporcionado en la URL.
GET	/provision/v0/getclient/dni/{dni}	Se obtienen los clientes que tienen el DNI igual al proporcionado en la URL.
POST	/provision/v0/newclient	Se crea un cliente en base al que se proporciona en el cuerpo de la petición.
PUT	/provision/v0/updateclient/{msisdn}	Se actualiza el cliente con el MSISDN proporcionado en la URL y con los campos introducidos en el cuerpo. Si el cuerpo esta vacío no hay modificación.
DELETE	/provision/v0/deleteclient/msisdn/{msisdn}	Se elimina el cliente que tiene el número de teléfono igual al proporcionado en la URL.
DELETE	/provision/v0/deleteclient/email/{email}	Se eliminan los clientes que tienen el email igual al proporcionado en la URL.
DELETE	/provision/v0/deleteclient/dni/{dni}	Se eliminan los clientes que tienen el DNI igual al proporcionado en la URL.

Tabla 3.1: *Endpoints* del API de provisión.

Este API no es muy compleja ya que la lógica del servicio es muy ligera. La clave es crear un modelo adecuado del cliente. Para ello, se crean las clases *Client.java*, *Address.java* y *Titular.data*. Estas clases, representan los atributos de la base de datos que hemos explicado previamente. Para intentar tener la mayor uniformidad posible

en los clientes que se creen a través de este API, se va a añadir a través de expresiones regulares una serie de reglas en cada uno de los campos.

Estas reglas están basadas en la especificación de los campos. Por ejemplo, en el MSISDN se verifica que tiene el formato comentado previamente o el MCC y MNC que se chequea que tienen tres y dos dígitos respectivamente. Para los nombres, se permiten nombres y apellidos formados hasta de dos *Strings* teniendo en cuenta algunos casos de nombres compuestos o que se introduce el primer y segundo apellido. En el caso del DNI se pide el formato de 8 dígitos y una letra, mayúscula o minúscula. Para el genero, solo se puede introducir la palabra Masculino o Femenino. En el caso del cumpleaños, se sigue el formato año-mes-día. Por último, para los elementos de la dirección que son de tipo *String* se limita a cuatro o cinco los *String* que lo pueden formar. El código postal se pide que sean dígitos de entre una y siete cifras.

Cada vez que llegue una petición para crear un cliente, la anotación *@Valid* va a comprobar que se cumplen estas reglas, en caso de que no se cumpla, se lanza una excepción de tipo 400 *Bad Request*. En el caso de la actualización, se tendrá que hacer la comprobación manualmente ya que se puede dar el caso que el cuerpo de la petición venga vació por lo que saltaría continuamente el error.

A nivel de controlador, el programa no tiene mucho más problemática. Se incluyen mensajes de *log* en todos los puntos clave para que se pueda distinguir si se han respondido con mensaje de éxito o por si lo contrario se devuelve una excepción. Por el momento, los *logs* de cada API se guardan en la carpeta *logs*. Por otra parte, se puede apreciar como la anotación *@CrossOrigin(origins = "*")* está presente. La anotación *@CrossOrigin* se utiliza para permitir peticiones de Origen Cruzado, CORS (*Cross-Origin Resource Sharing*) al controlador. Se va a usar para que las API sean accesibles desde el portal web que se verá más adelante. Esto es inseguro ya que se debería permitir acceso solo al *endpoint* necesario, pero es más cómodo para el desarrollo del trabajo.

En el caso de los métodos de la clase de servicio, simplemente se hace una llamada a la interfaz del repositorio. En el caso de crear un cliente, se hace una comprobación previa de que el usuario no existe y es nuevo. Si ya existe, se lanza una excepción con código 400 *Bad Request*. Se crea el *timestamp* de la última vez que el usuario creó una SIM con la fecha actual ya que se le asigna por primera vez un IMSI. En el caso de la actualización, dado que las comparaciones se hacen a mano, es un poco más extenso pero bastante simple. Se actualiza el *timestamp* si el IMSI va a ser actualizado, en caso contrario se mantiene intacto. Por otro lado, no se puede hacer una actualización del campo titular a *true* si ya existe algún cliente con los mismos datos que sea titular.

Para manejar las excepciones con el formato deseado se crea la clase *CustomException.java*. En esta clase se definen los atributos que deben tener las excepciones personalizadas. Se ha definido esta personalización ya que es el formato que se especifica para los errores en las API *CAMARA*. En la clase *RestExceptionHandler.java*, se

personalizan algunas de las excepciones que pueden saltar con este formato. Se define la excepción para argumentos no válidos como 400 *Bad Request*, la de clave duplicada para casos en los que ya existe el MSISDN con error 409 *Duplicated Key*, la de errores internos con 500 *Internal Server Error* y en general por defecto para los errores no especificados, se pide que se devuelva también 500 *Internal Server Error*.

El último punto que se va a destacar en la programación de todas las API es la parte de los test. En la teoría clásica de programación, se deberían hacer pruebas unitarias, pruebas de integración y pruebas *end to end* de los servicios. En este caso, se ha decidido realizar pruebas unitarias del controlador para cada uno de los *endpoint* y pruebas unitarias de la capa de servicio para todos los métodos. Para la capa del repositorio, no se harán pruebas unitarias por dos motivos, la implementación es muy sencilla y porque habría que tener una conexión con una base de datos embebida como H2 para probar.

Las pruebas de integración no se harán ya que se cubrirán directamente con las *end to end*. Para las pruebas *end to end* hasta este momento se harán de forma manual a través de *Postman*. *Postman* es una herramienta que facilita la interacción con las API de una manera interactiva y visual, evitando hacer las peticiones desde el terminal. Al final del capítulo, se realizará una automatización de las pruebas y el despliegue.

Con respecto a las pruebas unitarias del controlador, se han realizado un total de 105 pruebas para los nueve *endpoints* del API. Aunque es un número grande, es muy difícil que la cobertura llegue al 100 % de los casos ya que hay múltiples campos en los clientes y casuísticas que se pueden dar. Para las API de tipo *GET* y *DELETE*, se prueban principalmente situaciones en las que el identificador (MSISDN, DNI, *email*) se da vacío, no se encuentra ningún cliente con ese identificador o pruebas para casos que deberían ser exitosos. Para el API POST, se prueban de diferentes formas la validación de los campos de entrada. Para el API de tipo *PUT*, se prueban formas de validación y además se prueban las casuísticas especiales para el *timestamp* y el campo titular.

Otro aspecto que todavía no se ha comentado, la conexión con la base de datos y el puerto que expone la aplicación se definen en la carpeta *resources* en un fichero que se llama *application.properties*. En este fichero, se puede comprobar que se ha puesto como conexión el nombre *mongodb*. Este nombre no es más que el del contenedor de Mongo DB. Dado que se está exponiendo en el despliegue de *Mongo DB* en la máquina local, se podría poner *localhost*. Sin embargo, dado que se quiere que funcione posteriormente en *Kubernetes* y siendo independiente de la máquina de despliegue, se usa la red de *Docker*. *Docker* crea ya una red por defecto para los contenedores que se desplieguen con *Docker Compose* así que no es necesario que se especifique en el YAML.

Como se especifica el nombre de *mongodb*, esto significa que si se ejecuta esta aplicación directamente desde *IntelliJ*, la ejecución va a fallar ya que no va a encontrar

en el DNS local una forma de resolver este nombre. Para que funcione correctamente, se debe crear una imagen que ejecute esta aplicación. Para ello, de nuevo se crea un *Dockerfile*.

La imagen base va a ser *eclipse-temurin:21* ya que aunque tradicionalmente se usan las de *OpenJDK*, esta ha quedado deprecada. Se carga el .jar de la aplicación tras haber hecho la compilación del proyecto a través de *mvn clean package* y comprobar que han pasado las pruebas. A partir de ahí, simplemente se ejecuta la aplicación. El *Dockerfile* se puede encontrar en el directorio /docker/provision del repositorio. La aplicación se expone en el puerto 8082 también de *localhost*. Se comprueba que se ha desplegado correctamente y se realizan las pruebas *end to end*. Se muestra el resultado de alguna de estas pruebas que se realizaron.

```
GET http://localhost:8082/provision/v0/get/msisdn/+34651481204 HTTP/1.1
```


```
{
  "id": "6604a5a37b39baa2f8f48039",
  "msisdn": "+34651481204",
  "imsi": "214071234567804",
  "msc": 214,
  "mnc": "07",
  "cellId": 6615,
  "titular": true,
  "titularData": {
    "name": "Lidia Cortés",
    "givenName": "Lidia",
    "familyName": "Cortés",
    "idDocument": "08359469J",
    "email": "lidiacortes@gmail.com",
    "gender": "Femenino",
    "birthdate": "1929-12-27",
    "address": {
      "streetName": "Calle de Goya",
      "streetNumber": "81",
      "postalCode": "28023",
      "region": "Madrid",
      "locality": "Madrid",
      "country": "España",
      "houseNumberExtension": "1D"
    }
  },
  "latestSimChange": "2021-10-19T01:11:11.000Z"
}
```


En este caso, se ha probado que la consulta a través del MSISDN funciona correctamente. Esto se puede comprobar ya que en el fichero de clientes generado, si se busca este número de teléfono se obtienen exactamente estos datos. Aunque no se ha mostrado aquí, las consultas por *email* y DNI funcionan correctamente de igual forma. En caso de introducir un MSISDN que no está en la base de datos, la excepción generada cumple con el formato esperado y devuelve un 404 *Unknown Msisdn*. En las siguientes imágenes, se van a mostrar las pruebas de casos exitosos para la creación,

modificación y eliminación de un cliente. Cabe destacar que estas pruebas no han sido las únicas y que es necesario probar diferentes casuísticas.

3.2.3. Implementación del API *Open Gateway Sim Swap.*

Como se explicó en el capítulo 2, este API tiene como objetivo obtener información de cuando fue la última vez que un usuario cambio de SIM. Los casos de uso que se imaginan para este API son principalmente de antifraude. La especificación del API en la única versión oficial que tiene (v0.4.0), se puede encontrar en el repositorio oficial de *CAMARA* y es pública. La especificación cumple con *OpenAPI* y se puede visualizar fácilmente con *Swagger*.

Para implementar este API en *Spring Boot* se va a utilizar el mismo patrón que se mostró en la figura 3.13. En primer lugar, se va a crear la base del proyecto con *Spring Initializr* tal y como se hizo en el API de provisión. Las versiones de Java y *Spring Boot* serán las mismas al igual que las dependencias. Siguiendo la arquitectura de implementación, en el controlador solo va a haber dos *endpoints* distintos. El primer *endpoint* va a ser *retrieve-date*, en este caso se va a introducir como parámetro de entrada el número de teléfono en el cuerpo de la petición y el API responde con el *timestamp* asociado a ese número.

El segundo *endpoint*, *check*, es quizás más interesante de cara al desarrollador ya como parámetro de entrada se introduce el número de teléfono y además un campo *maxAge* que indica cual es el tiempo límite en el que se está interesado en saber si se cambio de SIM o no. El API devuelve *true* o *false* dependiendo de si el *timestamp* es menor o mayor con respecto al parámetros *maxAge*. Este segundo *endpoint* es más interesante ya que el desarrollador no se ocupa de hacer lógica con el *timestamp* y simplemente debe gestionar una respuesta de tipo booleana. Se muestra una tabla resumen con estos dos *endpoint*:

Método	***Endpoint***	**Descripción**
POST	/sim-swap/v0/retrieve-date	Se obtiene el *timestamp* de la última vez que un usuario cambio de SIM.
POST	/sim-swap/v0/check	Se obtiene un booleano que indica si el *timestamp* es mayor o menor que el parámetro *maxAge* dado en el cuerpo de la petición.

Tabla 3.2: *Endpoints* del API de *Sim Swap.*

A nivel de programa, dado que los parámetros de entrad son JSON, se va a crear en el modelo dos clases más que van a representar los objetos de parámetros de entrada y de salida. Estas clases se van a llamar MsisdnDTO y CheckDTO. Los parámetros de entrada de *retrieve-date* y *check* serán objetos de este tipo respectivamente. Para las respuestas, se podría crear también clases que representasen los objetos JSON pero se va a realizar de una manera más directa utilizando mapas.

La función que va a tener el controlador es primero verificar que los campos de entrada cumples con las expresiones regulares definidas. En el caso del número de teléfono se aplica la expresión regular que ya se ha estado utilizando y en el caso de *maxAge* se dice en la especificación que debe estar entre 1 y 2400 y asignarse a 240 por defecto si el campo viene vacío. En el caso de *retrieve-date*, se verifica también si el cliente existe en la base de datos y sino se devuelve un 404 *NOT FOUND* con el formato especificado en el *Swagger*. Para *check*, se establece por defecto el campo de *maxAge* a 240 y se hace la validación de los campos de entrada de forma manual. De nuevo, el motivo de no usar la anotación *@Valid* es que el campo *maxAge* puede estar vacío y saltaría directamente la excepción.

A nivel de servicio, el *endpoint* de *retrieve-date* no tiene complejidad alguna ya que simplemente se devuelve el campo *latestSimChange* que se guarda en la base de datos. En el caso de check, se lanza la excepción si no se encuentra el cliente en la base de datos, y si se encuentra se hace una comparación del *timestamp* con *maxAge*. Como *maxAge* está en días, se hace una pequeña manipulación del *timestamp* para convertirlo en la misma unidad y comparar. A nivel de repositorio, es muy sencillo ya que únicamente se hacen consultas a la base de datos por MSISDN.

Por último, para hacer los test unitarios del API, de nuevo, se proponen los test unitarios para las clases de controlador y servicio. En el caso del controlador, se prueban las salidas exitosas, salidas en las que sale error por MSISDN no encontrado, salidas para parámetros de entrada incorrectos, etc. En el lado de servicio, solo se prueba el método de *check* ya que el método para *retrieve-date* es simplemente una consulta a la base de datos. Las pruebas de este método no son muy extensas, se prueba un caso en el que el MSISDN no existe, otro para el caso en el que se devuelve *true* y otro para *false*.

Para que el programa pueda correr sin errores, se va a crear nuevamente un contenedor que esté en la misma red *Docker* que la base de datos. El problema es que si se ejecuta directamente el programa desde *IntelliJ*, fallará la conexión con la base de datos por el mismo motivo que antes. Se siguen los mismos pasos, se compila y se pasan las pruebas con *maven* y posteriormente a través de una imagen base de eclipse se carga el .jar y se ejecuta dentro de la imagen. Se puede encontrar el *Dockerfile* correspondiente en el directorio /docker/simswap. Se incluye el despliegue en el YAML de *Docker compose*, el API expone el puerto 8083 a *localhost*. Se muestra a continuación, un ejemplo de las pruebas *end to end* que se hacen a posteriori con *Postman* al *endpoint* de *retrieve-date*.

```
POST http://localhost:8083/sim-swap/v0/retrieve-date HTTP/1.1
Content-Type: application/json
{
  "phoneNumber": "+34651481214"
}
```

```
{
  "latestSimChange": "2021-12-06T09:10:23.000Z"
```

```
}
```

Se puede comprobar que el resultado es coherente, mirando los datos del usuario que hay guardado en la base de datos con ese número de teléfono. Se puede ver como el *timestamp* coincide perfectamente. Por último, algunas capturas de las pruebas al *endpoint check*. De nuevo, es interesante fijarse en los resultados y ver que son coherentes con los clientes de la base de datos. Se muestran ejemplos en los que se devuelve *true*, *false* o donde no se encuentra el cliente.

```
POST http://localhost:8083/sim-swap/v0/check HTTP/1.1
Content-Type: application/json
{
  "phoneNumber": "+34651481212",
  "maxAge": 2
}
```

```
{
  "swapped": false
}
```

```
POST http://localhost:8083/sim-swap/v0/check HTTP/1.1
Content-Type: application/json
{
  "phoneNumber": "+34651481212",
  "maxAge": 2000
}
```

```
{
  "swapped": true
}
```

```
POST http://localhost:8083/sim-swap/v0/check HTTP/1.1
Content-Type: application/json
{
  "phoneNumber": "+35651481212",
  "maxAge": 2
}
```

```
{
  "status": 404,
  "code": "SIM_SWAP.UNKNOWN_PHONE_NUMBER",
  "message": "SIM Swap can‘t be checked because the phone number is
      unknown",
}
```

3.2.4. Implementación del API *Open Gateway Device Status*.

Para la implementación del API de *Device Status* hay que elegir en primer lugar que versión se va a implementar. En este caso, este API ha sufrido diversas modificaciones a lo largo de su corta vida. El motivo es que se quiere aprovechar este API para incluir no solo información de *roaming* sino también información de *reachibility*. Información

de *reachibility* se refiere a información para saber si un usuario está conectado a una sesión de datos, voz o está desconectado.

La versión donde se introduce este tipo de información es la (v0.5.0). Además, en esta versión destaca también una funcionalidad muy relevante y que está cogiendo fuerza en muchas de las API, es la funcionalidad de notificaciones. Es decir, los desarrolladores estarían más interesados en API de tipo notificación que de tipo consulta. Esto es, en lugar de estar preguntando de manera recurrente por si un usuario está en *roaming* o no, les gustaría que la propia operadora les notificase cuando cierto usuario está en *roaming*.

Dado que las notificaciones supondría un cambio de paradigma en la arquitectura diseñada, se implementará la versión 0.4.1. En esta versión solo se incluye la información de *roaming* y está ausente de notificaciones. La especificación sigue el formato *OpenAPI* y se encuentra en el repositorio oficial de *CAMARA* como en el caso de *Sim Swap*.

El desarrollo sigue la misma arquitectura expuesta en los casos anteriores. Para *Device Status*, se va a incluir en el modelo las clases necesarias para admitir el JSON de entrada como objeto y dar la respuesta en forma de objeto. Para ello, se incluyen al paquete modelo las clases *ApiInputDTO.java*, *UeIdDTO.java* y *RoamingResponse.java*. Este API admite como parámetros de entrada el MSISDN o la dirección IPv4 o IPv6 del terminal. En este caso, solo se tendrá en cuenta el MSISDN del terminal. Esto limita un poco los casos de uso y complica la vida al programador ya que se tiene que encargar de que su aplicación recopile el MSISDN del cliente. La dirección IP de la sesión se puede obtener sin intervención directa del usuario.

Como respuesta del API, se debe devolver si el usuario está en *roaming* y en caso de que sí lo esté, se debe proporcionar en qué país está junto a su MCC. Por lo tanto, la solución pasa primero por identificar si el usuario está en *roaming* o no y posteriormente en averiguar en qué país se está haciendo *roaming*. Comenzando con el controlador, esta capa va a hacer directamente una llamada al servicio. En el servicio, lo primero que se hace es consultar si existe en la base de datos el usuario con ese número de teléfono, en caso contrario se lanza la excepción acorde con la especificación.

Posteriormente, se comprueba si un usuario está en *roaming* o no comparando el MCC obtenido del CORE (UDM) con el MCC que se encuentra en el IMSI asociado con la tarjeta SIM. El IMSI tiene un valor fijo y los tres primeros dígitos lo forman el MCC de la red local. Por lo tanto, si el MCC obtenido del CORE y los tres primeros dígitos del IMSI no coinciden, el usuario está en *roaming*. Una vez se ha detectado que el usuario está en *roaming*, la pregunta es a qué país corresponde ese código.

Como se ha visto, existe en Internet una lista de correspondencia entre los MCC y los países asociados. Sería de interés, obtener esta información en un formato que se pudiese introducir en Java a través de una lista. A través de una búsqueda rápida se

encuentra esta información en formato JSON en un repositorio de *GitHub* [46]. En este repositorio no solo se encuentra la información del MCC y el país sino que se indica otra información como el MNC, las bandas o el nombre del operador. Por lo tanto, se debe hacer a partir de este JSON una adaptación para obtener solo la información de interés.

Una vez llegado a este punto, se replantea la situación de añadir esta información como código dentro del proyecto *Spring Boot*. La información de los MCC con los respectivos países es responsabilidad de la ITU y debería ser este organismo el responsable de actualizar las bases de datos con esta información. Por lo tanto, si se tiene esta información en el propio código se corre el riesgo de tener que hacer labores de mantenimiento antes posibles cambios. Aunque no se ha dado con una base de datos oficial con esta información, se va a simular que esta existe. Para ello, la información que relaciona MCC y país se va a almacenar en una base de datos. Ya que se tiene el formato JSON, podría ser útil meter la información directamente en *Mongo DB*. Sin embargo, las consultas a Mongo no son las más eficientes y solo se necesita la información del MCC y del país.

Por este motivo, se decide utilizar una base de datos de tipo clave valor en las que las consultas son más rápidas. Para ello, se va a utilizar *Redis*. Para que el API interactué con *Redis*, se debe desplegar esta base de datos en la misma red *Docker* que el servicio. Además, se va a hacer igual que con Mongo, se va a crear una imagen personalizada en la que se carguen previamente los datos para así no tener problemas en la inicialización de la base. Para poder pasar los datos de formato JSON a un formato que se pueda usar en *Redis*, se ha creado un *script* que haga la conversión. El *script* tiene como entrada el fichero JSON y devuelve como salida un fichero txt con instrucciones de *Redis* para fijar a cada MCC el valor del país correspondiente. Este documento está preparado para ser importado en *Redis*. Se puede encontrar el *script* en el directorio *scripts* del repositorio con el nombre mcc_json_redis_conv.py.

A partir del fichero txt, se crea un *Dockerfile* que coge como base la imagen de *Redis* (v7.2.4) y carga a través de un *shell script* los datos en *Redis*. Esta imagen personalizada se sube al repositorio de *Docker Hub* y se utiliza en el despliegue de *Docker Compose*, exponiendo el puerto 6379. Como se ha venido diciendo, no es necesario exponer los puertos de las bases de datos ya que la comunicación se hace a través de la red interna. Sin embargo, el despliegue con *Docker* es un entorno de pruebas y en *Kubernetes* se evitará la exposición.

Volviendo al desarrollo del API, se incluyen las pruebas unitarias de nuevo del controlador y del servicio. En el controlador, se hacen pruebas para casos exitosos y casos en los que el MSISDN es incorrecto o no se encuentra en la base de datos. No se ha mencionado previamente pero las llamadas al servicio se *mockean* con *Mockito* para aislar las clases. Para las pruebas de servicio, se *mockean* las llamadas las bases de *Redis* y Mongo y se prueban casos de *roaming*, no *roaming* y de excepción.

La conexión con *Redis* se especifica también en la carpeta *resources* en el fichero *application.properties* al igual que Mongo. En la conexión, se establece el nombre del contenedor ya que se usará la red interna. Para crear la imagen de esta aplicación, se sigue el mismo proceso que se ha explicado previamente. Compilando y ejecutando las pruebas se crea el archivo ejecutable de Java .jar y se utiliza este fichero para que se ejecute dentro de la imagen base de eclipse. Esta aplicación corre en el puerto 8084.

Además de las pruebas con *Postman*, se va a mostrar una captura de la base de datos *Redis* con la información que relaciona el MCC con el país. Se muestra el correcto despliegue de *Redis* y algunas de las pruebas realizadas con *Postman*:



Figura 3.14: Base de datos *Redis* para almacenar MCC y los países asociados.

```
POST http://localhost:8084/device-status/v0/roaming HTTP/1.1
Content-Type: application/json
{
  "ueId": {
    "externalId": "",
    "msisdn": "+34651481228",
    "ipv4Addr": "",
    "ipv6Addr": ""
  },
  "uePort": 5060
}
```

```
{
  "roaming": true,
  "countryCode": 740,
  "countryName": [
    "Ecuador"
  ]
}
```

3.2.5. Implementación del API *Open Gateway Know Your Customer.*

Como última API de *Open Gateway* a implementar, se va a desarrollar *Know Your Customer* (KYC). KYC es un API que va a consistir en validar los parámetros de entrada que aporta el cliente de una aplicación móvil con los datos que guarda la

operadora de telecomunicaciones de este mismo cliente. Los principales casos de uso de este API son antifraude y de seguridad. Hay muchas aplicaciones que desean que quienes se crean una cuenta en sus registros son personas con datos reales.

Se va a implementar la versión 0.1.0 del API que es la primera que se ha estandarizado en *CAMARA*. Este API tiene dificultades para estandarizares ya que el tipo de información que guarda una operadora Americana es muy distinta de la que puede guardar una Española u otra Japonesa. Se intentan incluir campos que se puedan cubrir de manera global. Aún así, como se puede comprobar en la especificación, se han incluidos datos que se usan especialmente en Japón como puede ser *nameKanaHankaku*. cabe recordar que este API al igual que todas las de *Open Gateway* cumplen el formato *Open API*.

Además de la funcionalidad del API ya mencionada, también se define en esta versión otro API que es KYC *fill-in*. La funcionalidad mencionada previamente se especifica como KYC *match*. KYC *fill-in* en vez de *matchear* los datos que introduce el usuario, va a servir para que las aplicaciones que la usen puedan auto completar los datos del usuario. En lo que a este trabajo se refiere, se realizará únicamente el API de KYC *match* ya que la versión de *fill-in* consistiría simplemente en consulta a la base de datos y una pequeña transformación de los datos obtenidos.

Para realizar el desarrollo de KYC *match*, se va a seguir el patrón utilizado en el resto de aplicaciones. De nuevo, se añaden en el modelo las clases necesarias para representar en forma de objetos los parámetros de entrada y salida del API. Estas clases van a ser *ApiInputDTO.java* y *ResponseDTO.java*. De lado del controlador, la complejidad es mínima ya que se llama directamente a la lógica de servicio.

En la programación del servicio, hay que tener varias cosas en cuenta. En primer lugar, se comprueba como es habitual si el número de teléfono está en la base de datos y sino se manda la excepción indicada por la especificación. En segundo lugar, la información que se obtiene de una consulta de este API son datos sensibles del usuario. Por esta razón, si al hacer la comparación del DNI de entrada con el DNI que se almacena en la base de datos, no se recibe *true* se mandará una excepción diciendo que el cliente no tiene acceso.

En el caso de que si coincidan, se procederá a hacer la comparación del resto de campos. Para campos de tipo *String* como el nombre, apellido, DNI, género o nombre de la calle, se van a crear métodos específicos para hacer la comparación. La razón de crear estos métodos es en primer lugar, tener un código más limpio y en segundo lugar aumentar los casos de éxito en la comparación. Es decir, si se hiciese una comparación literal de *String*, cualquier diferencia como un espacio en blanco, mayúsculas o minúsculas, etc daría como resultado de la comparación *false*, reduciendo la utilidad del API.

Para la comparación de DNI, se van a eliminar los espacios en blanco, se van

a truncar a una longitud máxima de 20 y se van a convertir todos los caracteres a minúscula. Para la comparación de nombres y apellidos, se van a eliminar los espacios en blanco, se van a truncar a una longitud máxima de 40, se van a eliminar todos los acentos y se eliminan todos los caracteres alfanuméricos. En el caso del nombre de la calle, se utiliza una estrategia parecida pero además, se eliminan de la calle introducida los prefijos del tipo avenida, carretera, calle, glorieta, alameda, etc. Por último, para el género se elimina todos los caracteres no alfanuméricos, los espacios, se convierten a minúsculas y además se traduce al inglés la información de la base de datos para hacer la comparación ya que los parámetros de entrada del API se especifican en inglés.

Aunque la especificación *CAMARA*, no deja claro como se deben implementar las comparaciones, se debe buscar un compromiso entre exactitud de la comparación y casos de éxito en las comparaciones. En la manera que se han implementado las comparaciones, se aumenta de manera considerable el caso de éxito y esto hace que sea un API más interesante para el desarrollador.

Otro de los temas que puede generar cierta preocupación a los desarrolladores de las aplicaciones es cual es la calidad del dato. Con calidad del dato se refiere a si la información que hay almacenada en la operadora está actualizada o no. Cada operadora deberá hacer un estudio de cuantos de los parámetros tienen disponibles, y cuando fue la última vez que se actualizaron. Es probable que líneas de contratos más antiguos tengan la información más desactualizada y sería interesante ver de alguna forma algún método para mitigarlo. Por ejemplo, se podrían ofrecer ciertos descuentos en compras de productos móviles a cambio de la actualización de los datos personales asistiendo personalmente a una tienda de la operadora.

Además, hay datos que son muy difíciles de obtener para empresas de este tipo como puede ser el género ya que cada vez más se considera información sensible y que aportan poco a la hora de contratar los servicios de Internet. Existen otros casos como es el DNI o los nombres y apellidos donde la información que posee una operadora debería ser bastante fiable y servir para validar a usuarios.

A nivel de programación, se han realizado las pruebas unitarias de la misma forma que en los casos anteriores. Se prueban casos de éxito, casos de error y casos de parámetros inválidos. Las pruebas de la clase de servicio pueden ser muy extensas debido al número de campos que hay y las distintas casuísticas que pueden aparecer para cada una de las comparaciones. Se han realizado un gran número de pruebas pero aún así, evidentemente, no se llega a una cobertura del 100 %. Se despliega la aplicación de la misma forma que se han desplegado las anteriores, con *Docker Compose* y generando una imagen específica que ejecute el archivo .jar. Se ejecutará en el puerto 8085 del contenedor y de *localhost*. Se muestra a continuación, una de las pruebas realizadas con *Postman*.

```
POST http://localhost:8085/kyc-match/v0/match HTTP/1.1
Content-Type: application/json
{
```

```
  "phoneNumber": "+34651481210",
  "idDocument": "0835957V",
  "name": "Sofia Hernández",
  "givenName": "Sofia",
  "familyName": "Hernández",
  "nameKanaHankaku": "",
  "nameKanaZenkaku": "Fede",
  "middleNames": "Sanchez",
  "familyNameAtBirth": "YYYY",
  "address": "Carrer De MALLORca 100",
  "streetName": "Carrer de Mallorca",
  "streetNumber": "100",
  "postalCode": "08029",
  "region": "Madrid",
  "locality": "Madrid",
  "country": "España",
  "houseNumberExtension": "VVV",
  "birthdate": "1978-08-22",
  "email": "sofiahernandez@gmail.com",
  "gender": "male"
}

{
  "idDocumentMatch": "true",
  "nameMatch": "false",
  "givenNameMatch": "false",
  "familyNameMatch": "false",
  "nameKanaHankakuMatch": "not_available",
  "nameKanaZenkakuMatch": "not_available",
  "middleNamesMatch": "not_available",
  "familyNameAtBirthMatch": "not_available",
  "addressMatch": "false",
  "streetNameMatch": "false",
  "streetNumberMatch": "false",
  "postalCodeMatch": "false",
  "regionMatch": "true",
   "localityMatch": "true",
  "countryMatch": "true",
  "houseNumberExtensionMatch": "false",
  "birthdateMatch": "false",
  "emailMatch": "true",
  "genderMatch": "false"
}
```


3.2.6. Implementación del servidor de autorización con *Keycloak*.

Como se ha comentado en la fase de diseño, se pretende implementar un servidor de autorización para implementar la capa de seguridad en las API. El objetivo es implementar el flujo *Oauth* 2.0 descrito en la figura 3.8 en la que el cliente pide un token de acceso al servidor de autorización con *client credentials* y debe incluir este token en todas sus peticiones. El API GW será el encargado de verificar con el servidor de autorización si el token es valido o no.

Para implementar el servidor de autorización había varias tecnologías que podían cumplir la funcionalidad como son *Okta*, *Keycloak* o *Authentik*. Finalmente, se ha decidido implementarlo con *Keycloak* debido a la gran documentación y tutoriales que hay para la implantación de flujos similares. Para hacer el despliegue, evidentemente se va a usar una de las imágenes más usadas que es la de *Bitnami*. Además, se quiere que cada vez que se despliegue la configuración quede guardada, por lo que es necesario persistir la información de alguna manera. *Keycloak* guarda toda la información de la cuenta en una base de datos *Sqlite* por defecto. Sin embargo, es recomendable usar una base de datos con más potencial para producción.

Para ello, se va a levantar una imagen de *Postgresql* que se conectará a *Keycloak* para guardar la información. Para que exista persistencia, se crea un volumen en la base de *Postgresql*. El volumen hace que la carpeta donde se guarda la información dentro del contenedor se comparta en una de las carpetas del ordenador local. Cada vez que se haga un cambio, este se mantendrá entre reinicios ya que el contenedor cada vez que arranque cargará los datos de la carpeta del ordenador local. A pesar de que esto ya es suficiente, una vez se tenga la configuración necesaria, se congelarán los datos de *Postgresql* y se creará una imagen personalizada que ya incluya esta información por defecto. El objetivo de este cambio es asegurar el rápido despliegue y correcto funcionamiento independientemente del volumen y de si se despliega en *Docker, Kubernetes* u otra tecnología. La imagen de *Postgresql* personalizada se subirá al repositorio de *Docker Hub* y contendrá todo lo necesario para tener *Keycloak* configurado correctamente.

Antes de hacer todo esto, se debe hacer la configuración por primera vez. En el despliegue con *Docker compose* se especifica la cuenta de administrador con las contraseñas necesarias. En el caso de este contenedor si es necesario que se expongan los puertos en *localhost* para acceder a la interfaz gráfica. Se expone *Keycloak* en el puerto 9080 de *localhost*. Una vez se ha accedido a la consola de administración, se va a crear un nuevo *Realm*, cliente y usuario en el sistema de autorización.

Un *Realm* en *Keycloak* es como un nuevo *tenant*, una nueva instancia lógica de *Keycloak*. Esto puede servir para tener un *Keycloak* para diferentes proyectos sin necesidad de reinstalar todo el programa cada vez que se va a usar para objetivos diferentes. Se va a crear un *tenant* específico para este proyecto llamado TFM. Una vez se ha creado el *Realm*, se va a crear un cliente específico en este *tenant* llamado *opengateway*. En la configuración del cliente hay varias opciones importantes que se deben marcar. En primer lugar, se pondrá * en la casilla de *Valid redirect URIs* esto permitirá la redirección al destino independientemente de cual sea el destino. También se marcará con * en la opción de *Web Origins* esto permitirá evitar los problemas de CORS. El * incluye todas las direcciones y quizás se podría concretar más poniendo la URI del API GW.

Por otro lado, se deben marcar las casillas de *Standard flow, Direct access grants, Service accounts roles* esto permitirá realizar el flujo de *client credentials* y *user*

password si fuese necesario. Por otro lado, aunque dado que se va a usar el flujo basado *client credentials*, se crean diferentes usuarios dentro del cliente dando la posibilidad de realizar el flujo basado en *user password*. Estos usuarios son *testuser* y *sandbank*. Para ver el *client secret* del cliente *opengateway*, se puede acceder a la pestaña *credentials* del mismo.

Para acceder al token de acceso del cliente, se debe hacer una petición al *endpoint* que ofrece *Keycloak*, http://localhost:9080/realms/TFM/protocol/openid-connect/token indicando en el body el *client id*, *client secret* y el tipo de acceso basado en *client credentials*. *Keycloak*, verifica los datos y si son correctos devuelve el token. Para verificar si el token es correcto y que será la manera que utilizará el API GW para hacer la validación, se hace una petición a http://localhost:9080/realms/TFM/protocol/openid-connect/token/introspect indicando de nuevo el *client id*, *client secret* y el token. *Keycloak* devuelve true si el token es valido. Se adjuntan algunas capturas de la interfaz de *Keycloak* y pruebas de token a través de *Postman*.

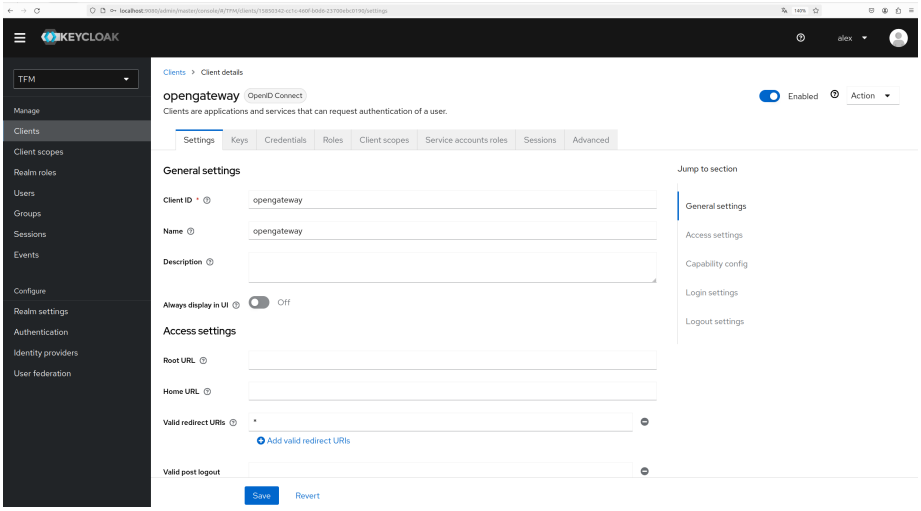


Figura 3.15: Consola de administración de *Keycloak*.

```
POST http://172.17.0.1:9080/realms/TFM/protocol/openid-connect/token
    HTTP/1.1
Content-Type: application/x-www-form-urlencoded
client_id=opengateway
username=testuser
client_secret=1JsLTnY3PicBWEZwcY702I8vIAaeyNs
grant_type=client_credentials
```

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3
     MTUzMTUzM2pMUsImNhInN0cyI6ImVjaXN0b3J5NSIsInN1YiI6IjEyMzc0
     NjQyMCIsImp0aSI6Ik01TkZUb3ZTWFI3UVQyZlBYbzdxUHNFWXEifQ.7Cj3f8H-Jd
     3ifh9tTlSf4NTs5DgI6JlpdsOkBfLf6LSfGh3L6gNs2wW8c6x1-9a7oV",
  "expires_in": 300,
```

```
  "refresh_expires_in": 0,
  "token_type": "Bearer",
  "not-before-policy": 0,
  "scope": "email profile"
}
```

```
POST http://172.17.0.1:9080/realms/TFM/protocol/openid-connect/token/
    introspect HTTP/1.1
Content-Type: application/x-www-form-urlencoded
client_id=opengateway
client_secret=1JsLTnY3PicBWEZwcY702I8vIAaeyNs
token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.
    eyJleHAiOjE3MTUzMTUzM2pMUsImNhInN0cy
    I6ImVjaXN0b3J5NSIsInN1YiI6IjEyMzc0NjQyMCIsImp0aSI6Ik01TkZUb3ZTWFI3UVQyZlBYbzdx
     UHNFWXEifQ.7Cj3f8H-
    Jd3ifh9tTlSf4NTs5DgI6JlpdsOkBfLf6LSfGh3L6gNs2wW8c6x1-9a7oV
```

```
{
  "exp": 1697053014,
  "iat": 1697052714,
  "jti": "ab6d2d0f-2d5f-45c9-8e0f-4d21fced0b29",
  "iss": "http://172.17.0.1:9080/realms/TFM",
  "sub": "1267e9e2-cc31-4fa0-8427-6d4145d6d3ad",
  "typ": "Bearer",
  "azp": "opengateway",
  "session_state": "2d947f94-9589-4622-96c7-4d249f615d9b",
  "acr": "1",
  "allowed-origins": [],
  "realm_access": {
    "roles": [
      "offline_access",
      "uma_authorization"
    ]
  },
  "resource_access": {
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  },
  "scope": "email profile",
  "email_verified": false,
  "preferred_username": "service-account-opengateway",
  "client_id": "opengateway",
  "username": "service-account-opengateway",
  "active": true
}
```

Hay que tener en cuenta que se ha hecho la petición a 172.17.0.1 pero valdría igualmente *localhost*. 172.17.0.1 es la red puente que expone docker a la maquina *host* para facilitar la comunicación entre los contenedores y el *host* [47].

3.2.7. Implementación del API GW con *Kong* y *Konga*.

Para implementar el API GW descrito en la arquitectura diseñada (ver figura 3.8), se va a utilizar *Kong*. Aunque hay otras muchas opciones para realizar las tareas de API GW como, por ejemplo, *APISIX*, *apigee*, *3scale*, etc. Sin embargo, ninguno ofrece una versión tan flexible como *Kong*. *Kong* permite añadir *plugins* de una manera bastante sencilla y personalizable a través de *scripts* de lua.

Por otra parte, en la versión gratuita de *Kong* no hay interfaz gráfica y se debe gestionar todo a través de las API que ofrece desde el terminal o desde *Postman*. Dado el número de API y *endpoint* que hay en este trabajo, sería de agradecer una interfaz gráfica para gestionar *Kong*. Es aquí donde surge *Konga*. *Konga* es una interfaz gráfica de código abierto para gestionar *Kong* y está mantenido por los usuarios.

Evidentemente, también se va a buscar que las configuraciones que se creen se persistan en el tiempo de manera que las rutas y configuraciones de seguridad no se eliminen en cada reinicio. Se va a seguir la misma estrategia que para *Keycloak*. Se va a tener tanto para *Kong* como para *Konga* una base de datos *PostgreSQL* y una vez se haya configurado todo, se va a congelar y se van a crear imágenes personalizadas de *PostgreSQL* que ya contengan toda la información necesaria. Además, se mantendrá un volumen por si es necesario persistir nuevos cambios.

En la versión gratuita ya viene incluido lo necesario para crear rutas y servicios a nuestro API centralizando las peticiones en nuestra arquitectura. Un servicio en *Kong* define el destino de *backend* real donde deben dirigirse las solicitudes, mientras que una ruta define las reglas sobre cómo y cuándo una solicitud debe ser dirigida a ese servicio. Por ejemplo, se definirá un servicio para KYC, en el que se especifique el servicio está en http://kyc:8085/kyc-match/v0/match y se creará una nueva ruta /kyc-match/v0/match que le dirá a *Kong* que las llamadas entrantes a través de este *endpoint* se le debe dirigir al servicio definido para KYC.

Para crear estas rutas y servicios desde *Konga* es muy sencillo ya que la interfaz es muy visual. *Konga* se ha desplegado en el puerto 1337 y el puerto de acceso a *Kong* para las peticiones será el 8100. Se puede ver la configuración en detalle en el YAML de *Docker Compose*. Una vez se definen las rutas y servicios para todas los *endpoint* de las API, la pregunta es como introducir la parte de la validación de token e interacción con *Keycloak*. El primer paso es incluir también en las rutas y servicios las API que permiten pedir un token y validarlo. Son las mismas que se han visto previamente. A continuación, se muestra una imagen con todos los servicios creados:

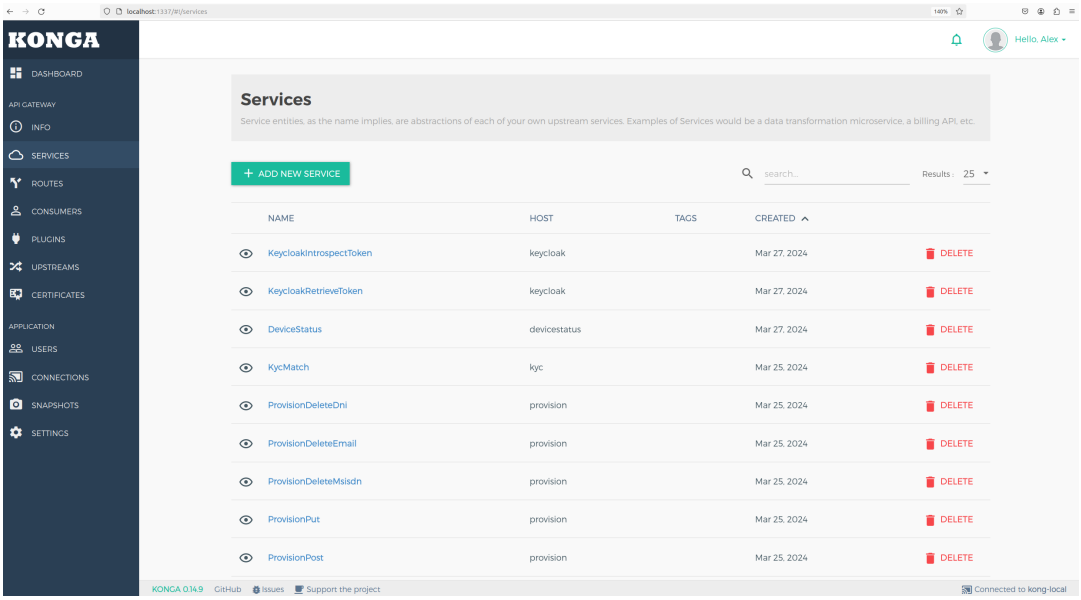


Figura 3.16: Servicios creados en *Kong*.

Por otra parte, para hacer la validación del token de acceso de todas las peticiones a través de *Kong* se va a necesitar crear un *plugin* personalizado. La razón es que la versión gratuita no lo soporta por defecto. El *plugin* que se va a crear se va a realizar con dos *scripts* de lua (*schema.lua* y *handler.lua*). Como sus nombres en inglés indican, una va a definir el esquema de los parámetros que necesita la validación y el otro hará la lógica necesaria para que se haga la validación. Como sabemos, el API de *Keycloak* necesita como parámetros para hacer la validación tres parámetros, *client id*, *client secret* y el token de acceso. Para que Kong pueda gestionar las llamadas a este API, necesita la URL de *Keycloak* el *client id* y el *client secret*. El token de acceso lo obtendrá de la petición entrante. Por lo tanto, en el *schema* se definen estos tres campos.

En el fichero *handler.lua* se especifica como se hace la validación. Básicamente coge los tres campos obligatorios que se deben introducir en la configuración del *plugin* y se obtiene el token de la cabecera de las peticiones entrantes. Con esta información, llama al API de *Keycloak* y verifica a través del campo active si es *true* o *false*. Para que el despliegue de *Kong* tenga este API por defecto, se crea en un *Dockerfile* una imagen personalizada que copia estos ficheros lua en la carpeta de *plugins* de *Kong*. Se puede encontrar estos ficheros en el directorio /docker/kong del repositorio. Una vez que el *plugin* está activo, se asocia a cada uno de los servicios desde *Konga*.

Tras varias pruebas, se decide cambiar la arquitectura de la figura 3.8. La razón es que la validación del campo active está siendo continuamente *false*. El motivo de esto es que el token se está obteniendo haciendo una consulta directamente a *Keycloak* mientras que la validación pasa por el API GW. *Keycloak* considera que como son dos orígenes distintos, el token de acceso no es válido. Por este motivo, se propone la

siguiente arquitectura:

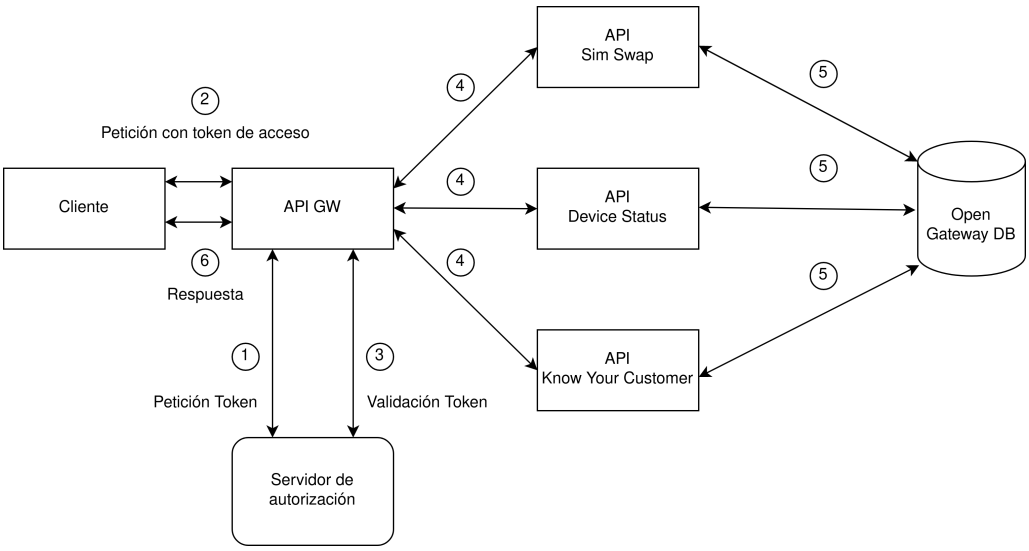


Figura 3.17: Diseño basado en arquitectura de microservicios con servidor de autorización y API GW como punto único de entrada.

El cambio principal en esta arquitectura es que para obtener el token de acceso, el cliente también debe hacerlo a través del API GW. Los servicios que se han definido en *Kong* para ello no incluyen el *plugin* de validación porque sino sería un bucle infinito. De esta manera, se prueba el funcionamiento de las API en esta nueva arquitectura y se verifica que el sistema está funcionando correctamente.

3.2.8. Implementación de portal web para gestión de base de clientes y entorno de pruebas.

Una vez se ha desplegado la arquitectura mínima para tener unas API seguras y centralizadas por un API GW, surge la idea de desplegar un portal web desde el cual se puedan realizar peticiones a todas las API creadas desde una interfaz gráfica e interactiva. De esta manera, se podría gestionar la base de cliente de una manera sencilla y sin tener que tener conocimientos sobre el terminal o herramientas como *Postman*. Este portal web sería ideal para un administrador de la plataforma que tiene que hacer cambios en la base de datos o hacer pruebas de las API de *Open Gateway*.

Con esta idea en mente, se procede a realizar la implementación usando el *stack* de tecnologías típico para desarrollo web. Esto es *HTML*, *CSS* y *Javascript*. *HTML* servirá para crear los elementos de la web, *CSS* para darles el formato necesario y *Javascript* para implementar la lógica de los componentes (Ej: Acciones de los botones). Todo el código se puede encontrar en en la carpeta /docker/web del repositorio. El código esta organizado en carpetas que separan las tres funcionalidades descritas. Para la parte de pruebas de las API de *Open Gateway*, se va a utilizar

directamente el formato *Swagger*, con algunos cambios para adaptarlo al mecanismo de seguridad que se ha establecido. Para habilitar la UI de *Swagger* en la web, se debe descargar el *Swagger* UI del repositorio oficial de *Swagger*, los ficheros se encuentran en el repositorio de este trabajo, en el directorio mencionado [48].

Para desplegar la solución, se ha decidido crear una imagen a través de un *Dockerfile* y partiendo como imagen base la de nginx:1.25.4-alpine. Esta imagen es una versión muy ligera de *Nginx*. Desplegar aplicaciones web en un imagen *Nginx* no es una tarea complicada ya que únicamente hay que copiar los ficheros que se han creado en el directorio /usr/share/nginx/html de la imagen *Nginx*. Como se ha hecho hasta ahora, se carga la imagen en el repositorio de *Docker Hub*. Se despliega la imagen en el mismo fichero YAML de *Docker Compose* bajo el nombre de web y se expone el puerto 80.

A continuación, se va a explicar como funciona la web. En el menú principal, se tienen tres botones para navegar al resto de ventanas. Los botones de la pestaña inferior son *Provision* y *Sandbox*. El botón de *Provision* hace una redirección a una ventana donde el usuario debe elegir si quiere consultar, crear, modificar o eliminar un cliente. Cada una de las ventanas a las que se accede permite introducir los campos necesarios para hacer todas estas actividades. Mencionar, que como mecanismo de seguridad, se solicita también en la web el token de acceso y se llamará a las API de provisión accediendo a través del API GW siguiendo el flujo visto anteriormente. En el botón de *Sandbox*, se accederá a una ventana parecida pero que llevará al usuario a elegir que API quiere probar. Además de estos botones, hay uno en la parte superior derecha que saca un desplegable donde se puede acceder a las ventanas del menú principal, la ventana para la gestión de clientes y la ventana para la prueba de API.

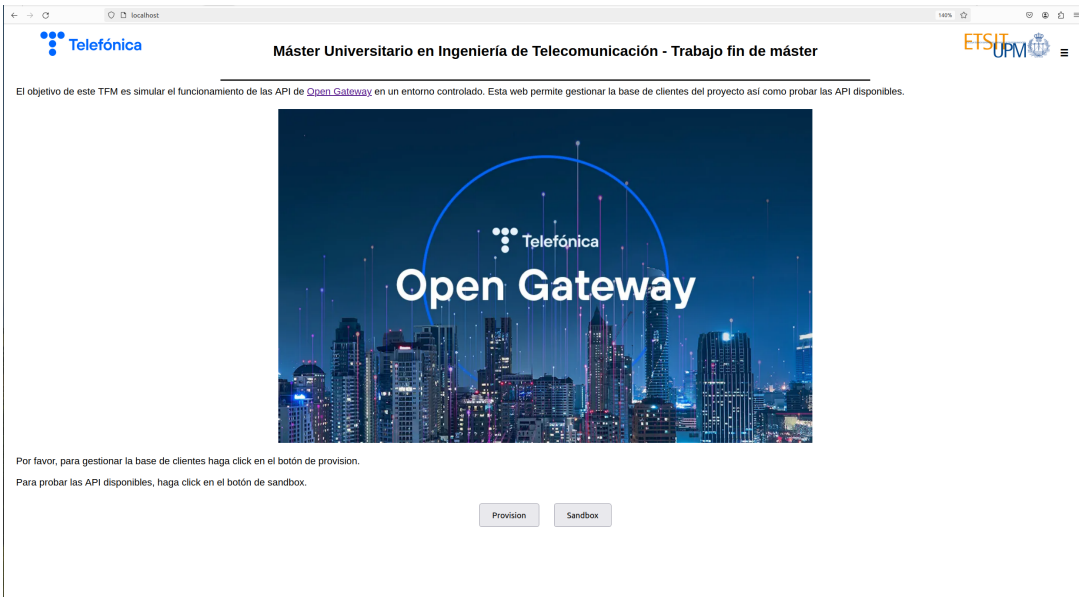


Figura 3.18: Menú principal de la web de administración.

Por otro lado, accediendo al menú de provisión se puede apreciar un botón en la parte superior derecha con un logo de una casa. El objetivo del botón es hacer una redirección al menú principal. Este botón aparecerá en el menú de provisión, en la interfaz de las actividades de consulta, creación, actualización y borrado de clientes y en el menú de prueba de API. Además, en la interfaz de las actividades de gestión de usuario, se puede encontrar a parte de las cajas para introducir los campos necesarios, un menú en la parte superior que permite cambiar de actividad de gestión. Como ya se ha dicho, la web seguirá el flujo descrito en la figura 3.17, por lo que las llamadas al API en el código *Javascript*, se hacen al API GW desplegado en *localhost*:8100. Se muestra a continuación una captura de la consulta de usuarios a través del portal web.

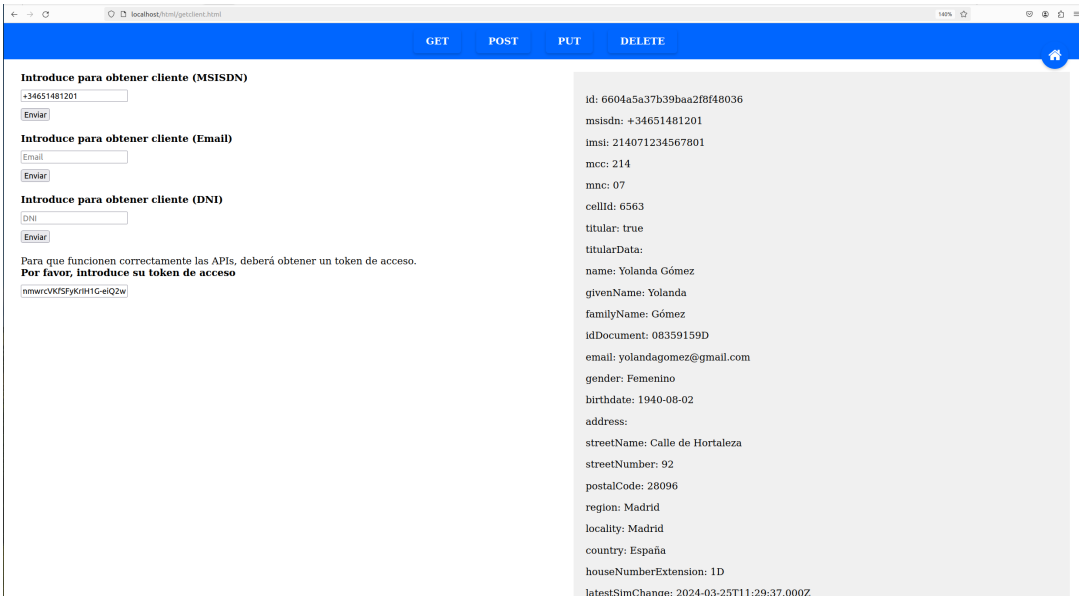


Figura 3.19: Consulta de usuarios a través del portal web.

Tal y como se puede ver, se puede consultar usuarios a través de MSISDN, *email* o DNI utilizando las API desarrolladas. Para el caso de pruebas de API, se va a aprovechar que existe un formato estándar como *Swagger UI* para integrar la especificación de las API *CAMARA* en la web con este formato. *Swagger UI* tiene la opción de probar el API, para ello se ha hecho una modificación en el código del YAML, especificando las direcciones en las que están accesibles las API en este proyecto. Para poder añadir el token de acceso en la petición, hay que introducirlo en el botón de *Authorize* que aparece en la interfaz. Se realiza una prueba con *Sim Swap*, para ello se hace clic en el botón de *Try it out*, el resultado se muestra a continuación en la figura 3.20.

Como se puede comprobar, el resultado es exactamente idéntico al que se podría obtener a través de línea de comandos con *curl* o mediante una aplicación como *Postman*. Esto es una forma mucho más sencilla y visual de probar las API que se han desarrollado.

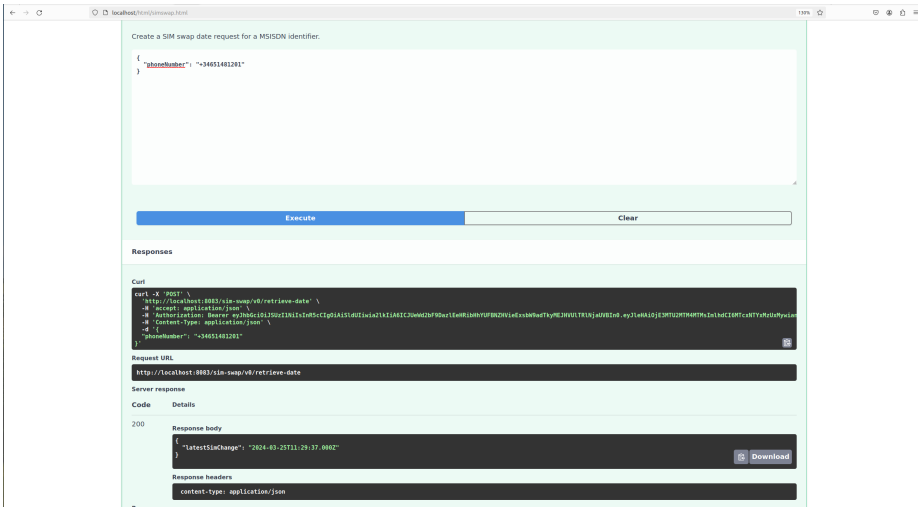


Figura 3.20: Prueba de API *Sim Swap* a través de la web.

3.2.9. Implementación del sistema de monitorización con Prometheus y Grafana.

Tal y como se explicó en la fase de diseño, es súper importante tener la capacidad de observar el comportamiento de los servicios desplegados. Para implementar el sistema de monitorización, hay un *stack* ampliamente adoptado por la industria que está formado por *Prometheus* y *Grafana*. Ambas herramientas son de código abierto. *Prometheus* es una aplicación de monitorización que se va a encargar de recoger las métricas de los servicios y almacenarlos en su base de datos de series temporales. Para hacer sencilla la exploración de los datos, usa *PromQL* (*Prometheus Query Language*). *PromQL* permite explorar de manera efectiva la consulta de datos específicos.

Aunque *Prometheus* ofrece una gran solución para recolectar, almacenar métricas y alarmas, la interfaz gráfica es bastante pobre como para permitir la correcta visualización del funcionamiento del servicio. Es aquí donde surge *Grafana*. *Grafana* va a permitir crear *dashborads* donde visualizar diferentes métricas de diferentes servicios de una manera bastante sencilla. Para obtener los datos, realizará peticiones a *Prometheus* a partir de las API que este ofrece. Sin embargo, no hay que preocuparse por esto ya que *Grafana* detecta por defecto la instancia de *Prometheus* y es capaz de conectarse a el a golpe de clic.

Prometheus debe de ser capaz de obtener los datos de los servicios de alguna manera. Para hacer esto, *Spring Boot* tiene dos dependencias que son la de *actuator* y la de *micrometer*. La dependencia de *actuator* proporciona varios *endpoints* útiles para monitorizar y gestionar las aplicaciones en tiempo de ejecución. Esto se puede ver fácilmente si se accede al *endpoint* de *actuator* de cualquiera de las API. Se muestra un ejemplo en la siguiente figura:

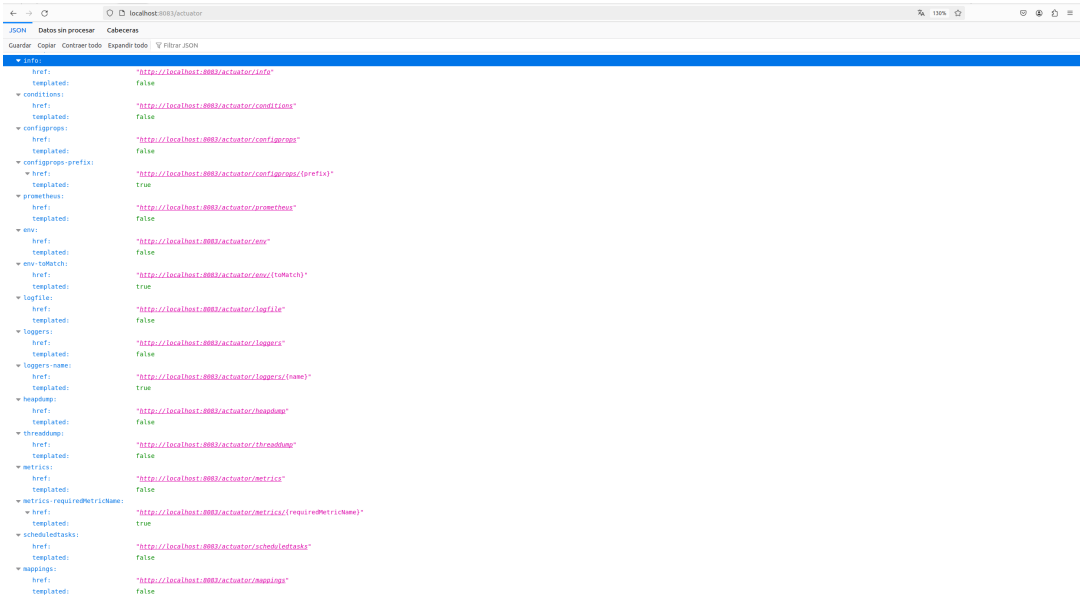


Figura 3.21: Ejemplo de *endpoints* expuestos por *actuator*.

Aunque la información ya está expuesta en el formato adecuado para *Prometheus*, se debe ajustar cual es la frecuencia con la que *Promethues* debe preguntar por esta información. Como siempre hay que encontrar un balance, cuanto más pequeño sea el intervalo de tiempo entre las peticiones, más actualizada estará la información pero más carga habrá en el servicio. *Prometheus* se va a desplegar usando la imagen de *Bitnami* en la versión 2.50.1. Para hacer la configuración de tiempo de refresco, se debe ajustar el fichero en la ruta /opt/bitnami/prometheus/conf/prometheus.yml dentro del contenedor. Para hacer esto, se va a utilizar un volumen compartido que copie el fichero especificado en /docker/prometheus del repositorio del trabajo. En este fichero se ha especificado un intervalo de 15 segundos. Además, se especifica también a que servicios se deben hacer las peticiones. Por otro lado, para persistir los datos se va a usar de nuevo un volumen compartido. Cabe destacar, que la base de datos de *Prometheus* no crece infinitamente almacenando el historial de todo el tiempo que recibe información, por defecto, almacena la información hasta 15 días. Este parámetro también se puede configurar a través del fichero descrito previamente.

Ahora que ya se tiene la información en *Prometheus*, se puede acceder a visualizarlos al puerto en el que se ha desplegado, 9090. Sin embargo, se puede comprobar como la interfaz no es muy visual. Por lo tanto, se pasa a desplegar *Grafana*. Para *Grafana*, se debe crear un *dashboard* adecuado que permite tener una visualización de las métricas más importantes y para los cuatro servicios. Para ello, se ha accedido al menú de *Dashboards* y se ha creado uno llamado *OpenGateway API Dashboard*. En este *dashboard*, se ha incluido información de todo tipo. En primer lugar, aparece el tiempo que el servicio lleva corriendo y a qué hora se desplegó. Por otro lado, se tiene la memoria utilizada tanto *heap* como *no heap*. El termino de memoria *heap*, hace referencia a la memoria que usa la JVM (*Java Virtual Machine*) para para almacenar los objetos e instancias que crea el programa. Por el otro lado, la

memoria *no heap* es la utilizada para almacenar el resto estructuras como por ejemplo, los metadatos que incluyen la información del programa.

Seguidamente, se tiene dos importantes gráficas. En la primera de ellas se compara el consumo de CPU total con respecto al consumo específico de esa aplicación. Por otro lado, se compara en la gráfica el número de CPUs disponibles (7 tal y como se configuró Virtual Box) con respecto a la carga media de la aplicación. En la parte inferior, se muestran tres gráficas también muy relevantes. La primera de ellas, muestra la suma total de peticiones que se ha hecho para cada *endpoint* del API. En segundo lugar, se tiene una representación de la suma total de tiempo que han tardado las peticiones a cada *endpoint*. Por último, se tiene un gráfico que muestra el máximo tiempo de respuesta de entre todas las peticiones que se han hecho. Debajo de estos tres gráficos, se muestran otros dos gráficos más. En primer lugar, se tiene el tiempo de respuesta de cada petición. Cabe destacar que el tiempo de respuesta que se da cuenta lo que tarda el servicio entre que le llega la petición y este responde. Sin embargo, este tiempo difiere del que se puede obtener por ejemplo con *Postman* ya que no se está teniendo en cuenta el tiempo de obtención y validación de token previo a la petición del API. A su lado, se muestran el número total de peticiones en los que se ha recibido un 2XX, un 4XX o un 5XX.

Por último, se tienen las dos últimas gráficas. En la primera de ellas, se muestran las peticiones por segundo que está recibiendo el servicio para cada uno de los *endpoint*. Mientras que en la última gráfica se va a representar el tiempo de respuesta medio para cada uno de los *endpoint*. Se muestra a continuación, el *dashboard* obtenido para *Sim Swap*:

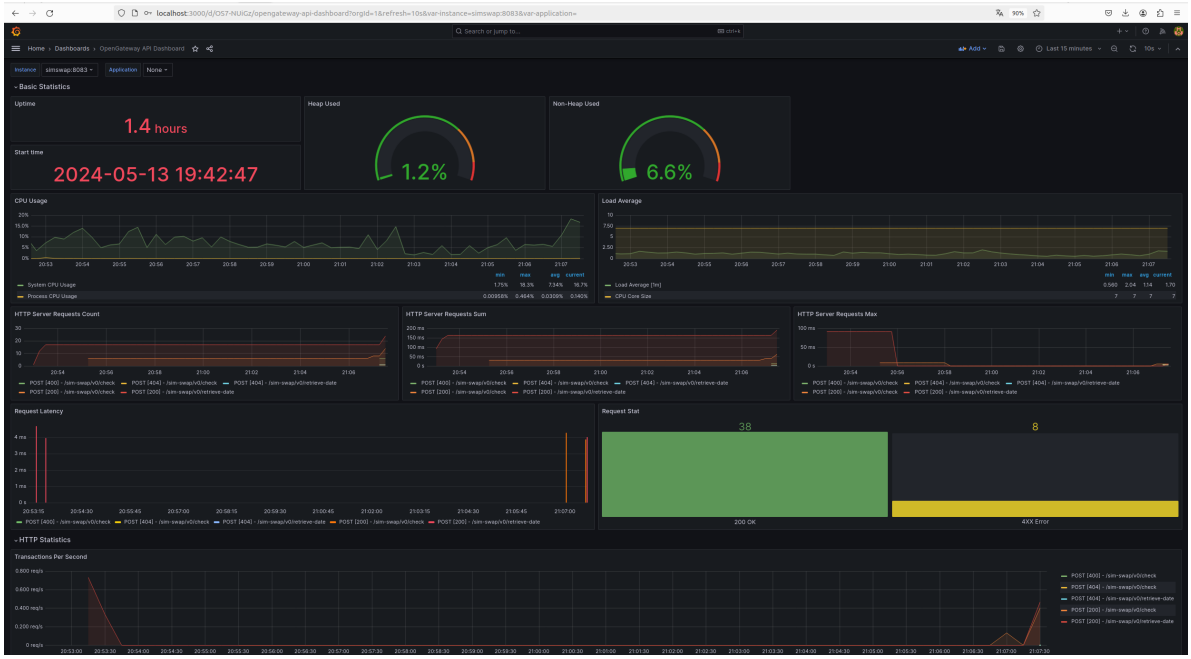


Figura 3.22: Dashboard de Grafana para el API *Sim Swap*.

Por último, comentar que para el despliegue de *Grafana* se ha seguido una estrategia similar a *Kong*, *Konga* y *Keycloak* para persistir la información sin tener que hacer cuenta de los volúmenes que tantos problemas pueden dar. Para mantener la configuración, se ha personalizado la imagen de *Grafana* a través de un *Dockerfile* tal y como se ha hecho en el resto de herramientas. El fichero se puede encontrar en docker-demo/grafana del repositorio. Básicamente, una vez se ha configurado correctamente se va a crear una copia de la base de datos de *Grafana*. En el *Dockerfile*, se copia esta base de datos en la imagen de *Grafana* (v10.3.3). De esta manera, se tendrá en cada reinicio los *dashboard* de *Grafana* configurados.

3.2.10. Implementación del sistema de centralización y visualización de *logs* con *Logstash*, *Elasticsearch* y *Kibana*.

Además del sistema de monitorización que da gran información del desempeño general del sistema, es muy importante saber en caso de fallos donde ha ocurrido el problema. Para ello, tal y como se comento en la fase de diseño, se va a implementar un sistema que centralice los *logs* y que permita visualizarlos de una manera sencilla. Para hacer esto, se va a emplear un *stack* de tecnología ampliamente adoptado en la industria que se llama ELK *stack*. Este *stack* está formado por *Elasticsearch*, *Logstash* y *Kibana*.

Logstash va a servir para procesar los datos de todos los servicios simultáneamente, transformándolos y enviándolos a *Elasticsearch*. *Elasticsearch* va a ser el sitio donde se almacenen los *logs*. La misión principal de *Elasticsearch* es indexar y almacenar datos, permitiendo búsquedas complejas a través de una API REST. Para poder visualizar los *logs* indexados, se va a utilizar *Kibana*. *Kibana* va a ofrecer una interfaz web que va a permitir crear *dashboards* y configuraciones para la visualización de los *logs* recogidos de todas las API.

Por lo tanto, el objetivo es que los *logs* que se produzcan en cada uno de los servicios se guarden en *Elasticsearch* de manera centralizada. Para hacerlo, *Logstash* se encargará de recogerlos y formatearlos de manera adecuada. Por último, se visualizarán a través de *Kibana*.

El primer paso para realizar este flujo es pensar como va a recibir *Logstash* los *logs* de las API. Tal y como se verá en el despliegue con *Kubernetes*, lo más óptimo es que los servicios se comuniquen activamente con *Logstash* sin necesidad de que los *logs* se escriban en algún sitio fuera del contenedor. Por el momento, la estrategia que se va a seguir es diferente.

Si se han revisado ya los proyectos de las API realizadas, se puede haber notado que hay una carpeta llamada *logs*. En esta carpeta, va a escribir cada servicio los *logs*. Esto se ha definido en el fichero *logback-spring.xml* dentro de la carpeta *resources* del proyecto. En este fichero se especifica el formato que debe tener el *log*, donde se debe escribir el *log* y las técnicas de *roll out* necesarias. En este caso, el formato del

log incluye la fecha y hora en que se ha producido, el nivel de criticidad, el mensaje incluido y el hilo que lo ha generado. Por otro lado, se especifica que se escriban en la carpeta *logs* que se ha comentado. la técnica de *roll out* indica que el tamaño máximo del fichero no debe superar los 10MB. A partir de esta capacidad de memoria, se irán eliminando los más antiguos.

Los *logs* de cada API se han escrito de manera que aporten un valor significativo. Por ejemplo, si un método puede devolver una excepción o no, se escriben dos *logs* diferentes dependiendo del resultado. Bien, con esta estructura se tiene que las aplicaciones escriben en la carpeta *logs* todos los *logs* generados. Dado que esto va a correr dentro del contenedor pertinente, la pregunta es como *Logstash* va a recoger esta información. La idea que se va a seguir en el despliegue *Docker* es bastante sencilla. La carpeta interna en la que el servicio va a escribir sus *logs*, se va a compartir a un fichero de la máquina local a través de un volumen. De esta manera, se están centralizando todos los *logs*. Esta carpeta, está en el directorio docker/logstash/logs del repositorio.

Una vez que se han conseguido centralizar los *logs* de todos los servicios, es el turno de que *Logstash* los lea, los formatee y los envíe a *Elasticsearch*. Para ello, se va a desplegar con la versión oficial 8.12.2 y se va a crear el fichero de configuración en la ruta docker/logstash/pipeline del repositorio. En este fichero se especifica que la fuente de información es el fichero de *logs* centralizados y además se marca a través del fichero *sincedb* que solo se lean los *logs* nuevos que aparezcan. Sino estuviese la marca de *sincedb*, saltaría el histórico de *logs* de golpe cada vez que se despliega la solución. Además, se quita una serie de información que añade *Logstash* por defecto ya que es repetitiva y se cambia el nombre a otro de los campos. Por último, se crea un índice temporal en *Elasticsearch* y se le envían los datos.

En el caso del despliegue de *Elasticsearch*, se va a usar la versión 8.12.2 oficial. El despliegue de *Elasticsearch* es algo complejo ya que por defecto está configurado para ocupar 2GB de memoria. Sin embargo, el límite máximo permitido por *Docker* está también en 2GB. Por este motivo, se especifica en el despliegue mediante variables de entorno que no se va a usar el modo cluster y se limita el espacio a 1GB [49]. Por otro lado, también se usa un volumen para persistir la información.

Por último, se despliega y configura *Kibana* para visualizar los *logs*. En el caso de *Kibana* no va a ser necesario persistir la información ya que toda la configuración se guarda en *Elasticsearch* y no en *Kibana*. Por lo tanto, simplemente para visualizar los *logs*, se accede a *Kibana* en el puerto 5601 y tras haber configurado el índice, se muestran los *logs* en la ventana de *Discover*. Se muestra a continuación, la visualización:

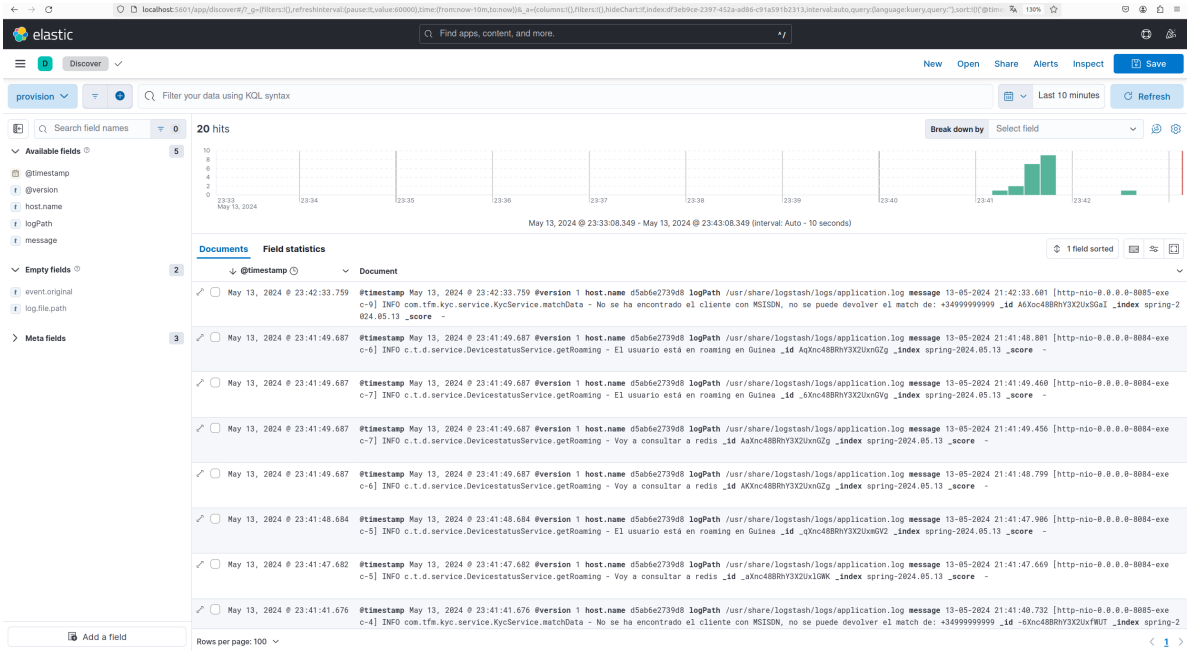


Figura 3.23: Visualización de los *logs* de las API a través de *Kibana*.

Como se puede apreciar, a través de *Kibana* es muy sencillo ver de manera muy visual los *logs* de todas las API. Si el *log* está bien escrito, da una clara idea de donde y como está la ejecución del programa.

3.2.11. Despliegue de la solución con *Kubernetes*.

En este apartado se van a explicar los pasos que se han seguido para migrar la solución de *Docker Compose* a *Kubernetes*. Aunque ambas soluciones sirven para desplegar y orquestar contenedores, existe una diferencia clave. El despliegue en *Docker Compose* se hace en una máquina mientras que *Kubernetes* permite hacer un despliegue distribuido en varios ordenadores (cluster). Aunque se va a seguir trabajando en un único ordenador, se va a habilitar la solución para que fuese posible desplegar con *Kubernetes* en un cluster. La principal razón es habilitar la posibilidad de desplegar la solución en entornos *cloud* con varios nodos donde se puedan aprovechar las características de resiliencia y disponibilidad. Sería un escenario más realista donde se busca mantener el servicio en circunstancias adversas.

Para poder trabajar con *Kubernetes* en un único nodo, se va a utilizar *Minikube*. *Minikube* es una herramienta que permite desplegar un cluster de *Kubernetes* en un único nodo. Para hacer esto, se ofrecen dos posibilidades. La primera de ellas es instalarlo como VM (*Virtual Machine*) y la otra es instalarlo como un contenedor *Docker* más. Dado que ya el sistema de *Ubuntu* en el que se está trabajando es una VM propiamente dicha y que se tiene *Docker* ya instalado, se va a elegir la segunda opción. Para hacer la instalación simplemente se siguen los pasos de la web oficial [50].

Una vez que se tiene instalado el cluster de *Kubernetes* con *Minikube*, se necesita

también *Kubectl* para interactuar con el cluster. *Kubectl* permite a través de la linea de comandos gestionar el cluster realizando actividades como , creación, despliegue, actualización o verificación de recursos. Para instalarlo, se siguen también los pasos descritos en la página oficial [51].

Habiendo instalado *Kubectl* y teniendo el cluster activo, ya solo queda convertir los recursos de *Docker* a *Kubernetes*. En este trabajo, no se va a dar una explicación detallada de la arquitectura de *Kubernetes* y sus diferencias con *Docker* ya que no es el objetivo. Sin embargo, se va a hacer un breve repaso de los conceptos necesarios para entender la migración. Tal y como se ha visto, en el fichero YAML de *Docker Compose* se especifican todos los contenedores que se van a desplegar de manera única. La especificación de cada contenedor incluye en la mayoría de ellos un nombre que les identifique, la imagen que se va a usar para el contenedor, los puertos expuestos y los volúmenes necesarios.

El despliegue en *Kubernetes* no maneja estos conceptos de manera unificada sino que utiliza recursos diferentes. Para el despliegue del contenedor de una manera similar va a ser necesario crear un fichero YAML de *Deployment*, un fichero YAML de *Service* y un fichero YAML para el volumen. Por lo tanto, son necesarios tres ficheros YAML distintos por cada contenedor de *Docker Compose* . El *Deployment* es un recurso de *Kubernetes* que define los *Pods* (Unidad mínima en *Kubernetes* que corresponde con uno o más contenedores), el número de réplicas de los *pods*, los volúmenes y *Service* asociados a los Pods entre otras cosas. El *Service* es el recurso que se va a utilizar para habilitar la comunicación entre *Pods*. Normalmente, el *Service* se define para un conjunto de *Pods*, funcionando como un único servicio de red y punto de acceso. Esto casa muy bien con la idea de tener varios *Pods* replicados en varias maquinas.

Por último, aunque el concepto de volumen sigue siendo el mismo la manera de declararlo es algo diferente. Aunque se puede forzar a tener un volumen del mismo tipo que *Docker*, compartiendo una carpeta de la máquina local con los contenedores, esto deja de tener sentido cuando se tiene un cluster y existen diferentes réplicas en diferentes ordenadores. Habría información disponible en un *Pod* y en otro no. Por este motivo, *Kubernetes* define una nueva forma de declarar los volúmenes. La forma en la que los *Deployment* suelen declarar los volúmenes va a ser a través de lo que se conoce como PVC (*Persistent Volume Claim*). Los PVC no son más que peticiones de almacenamiento para unas características específicas que necesita el *Pod*.

Por ejemplo, en el PVC se debe especificar el tipo de acceso (lectura, escritura, ambas, etc) y la memoria que se necesita. A través de esta petición, si existe un PV (*Persistent Volume*) que tiene estas características se acepta y se asocia el PV al *Deployment* que lo ha solicitado. Dado que tiene que haber un PV existente, la pregunta es quien lo crea y donde se encuentra. Lo más habitual es que los PV se persistan en un servidor concreto que puede ser uno del cluster u otro servidor fuera de él. La creación de los PV debería ser responsabilidad de un administrador, sin embargo, esta labor puede ser tediosa. Por este motivo, surgen los *StorageClass* que

van a permitir la creación automática de PV a partir de los PVC siempre y cuando existan los recursos disponibles.

Sin entrar en la creación de todos estos ficheros, ya se puede apreciar el motivo por el cual se prefirió crear imágenes personalizadas con las configuraciones hechas a que se configurasen las imágenes oficiales cada vez que se arrancan los contenedores a través de la compartición de ficheros con un volumen. Los PVC y PV aunque van a servir para persistir información, va a ser realmente complicado ubicar donde se encuentra la persistencia en el sistema de archivos. Además, dependiendo de donde se despliegue la solución de *Kubernetes*, es posible que no exista un *StorageClass* como el que se va a usar, no existan los PV necesarios, o simplemente el sistema de ficheros no sea el mismo. Por lo tanto, aunque se van a usar los volúmenes para persistir información, se va a tratar que el despliegue de la solución sea lo más independiente posible usando las imágenes que cuentan ya con las configuraciones hechas.

Para crear los ficheros de *Deployment*, *Service* y volumen a partir del fichero YAML de *Docker Compose*, se va a utilizar la herramienta *Kompose*. *Kompose* facilita esta migración generando de manera automática una plantilla de los ficheros de *Kubernetes* necesarios por cada contenedor definido en *Docker*. Para usar e instalar *Kompose* se siguen los pasos descritos en la web oficial [52]. Aunque los ficheros generados son de gran ayuda, va a ser necesario hacer algunas modificaciones para que la solución funcione correctamente.

En primer lugar, la solución que se tiene para centralizar los *logs* en *Docker* deja de tener sentido en *Kubernetes*. La razón es la misma que se explicó previamente, es complicada la gestión de volúmenes compartiendo carpetas de los *Pods* en la máquina local o en un PV. Una solución más efectiva pasaría por enviar en *streaming* los *logs* generados por las API según ocurren a *Logstash*. Con esta idea en mente, se introduce una nueva herramienta en la solución que es *Filebeat*.En cada *Pod* en el que en principio se despliega únicamente las API de *Open Gateway*, se va a desplegar en el mismo *Pod* un contenedor de *Filebeat*. El API de *Open Gateway* va a seguir escribiendo los logs en el directorio /app/logs. Dado que *Filebeat* y las API se despliegan en el mismo *Pod*, son capaces de compartir ficheros a través de volúmenes. De esta manera, se puede crear un volumen compartido en el que el API escribe los *logs* en el fichero habitual y *Filebeat* se va a dedicar a leer esta información y mandarla a *Logstash* según le llega la información. LA configuración de *Filebeat* se hace a través de un *Config Map* y se especifica de donde lee la información y la dirección de *Logstash* para enviarla.

Un pequeño diagrama que muestra esto sería el siguiente:

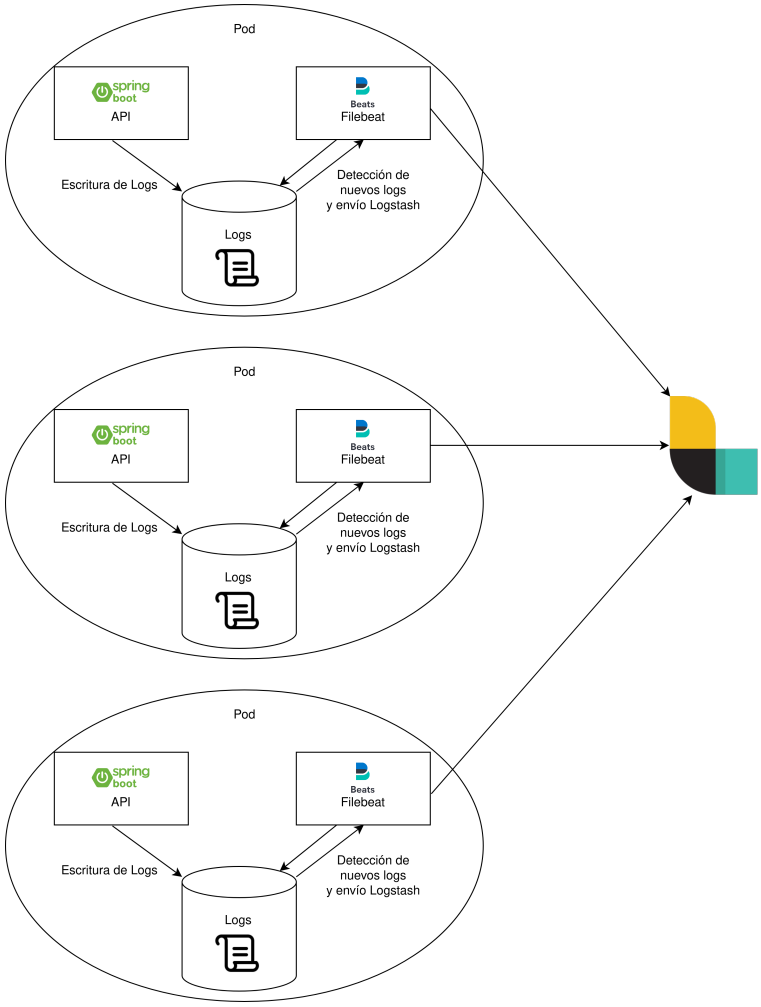


Figura 3.24: Arquitectura de recolección y centralización de *logs*.

Por otro lado, se ajusta la declaración de los PVC ya que por defecto se crean pidiendo un 1GB de memoria por volumen. Teniendo en cuenta que se necesitan siete volúmenes, supondría 7GB de memoria. Por este motivo, se ajusta a cada uno de los volúmenes el espacio necesario. Por otro lado, para que los PVC creen de manera automática los PV, se va a crear un nuevo *Storage Class*. Aunque por defecto *Minikube* ya tiene un *Storage Class* este no es de tipo *Retain*. Esto quiere decir que en cada reinicio, se van a borrar los volúmenes que se habían creado por lo que realmente no hay persistencia. Por este motivo, se decide crear un *Storage Class* adicional que sea de tipo *Retain*. Para los contenedores que aún mantienen la configuración en caliente en lugar de tener imágenes personalizadas, como por ejemplo *Logstash*, se hace a través de *Config Map* como se hizo con *Filebeat*. Se muestra a continuación una tabla resumen de todas las herramientas y *Pods* desplegados junto con los puertos que exponen:

Pod	**Puerto**	**Descripción**
Provisión	8082	El *Pod* incluye el API de provisión y *Filebeat* para el streaming de *logs*.
Sim Swap	8083	El *Pod* incluye el API de *Sim Swap* y *Filebeat* para el *streaming* de *logs*.
Device Status	8084	El *Pod* incluye el API de *Device Status* y *Filebeat* para el *streaming* de *logs*.
KYC	8085	El *Pod* incluye el API de KYC y *Filebeat* para el *streaming* de *logs*.
Web	80	Portal web para administración de clientes y pruebas de API.
Mongodb	27017	Base de datos con la información de clientes para *Open Gateway*. Si el cuerpo esta vacío no hay modificación.
Kong	8100	API GW como punto de acceso único a las API.
Kong-db	5432	Base de *Postgresql* que usa *Kong* para almacenar la información.
Konga	1337	Portal Web para la gestión del API GW.
Konga-db	5432	Base de *Postgresql* que usa *Konga* para almacenar la información.
Keycloak	8080	Servidor de autorización para habilitar la entrada a las API mediante token de acceso.
Keycloak-db	5432	Base de *Postgresql* que usa *Konga* para almacenar la información.
Redis	6379	Base de datos para almacenar la información de MCC y el país asociado.
Logstash	5000	Herramienta para la centralización y formateo de los *logs* de las API.
Elasticsearch	9200	Herramienta para el almacenamiento de las API.
Kibana	5601	Herramienta para la visualización de los *logs* de las API.
Prometheus	9090	Herramienta para el almacenamiento de las métricas de las API.
Grafana	3000	Herramienta para la visualización de las métricas de las API.

Tabla 3.3: Tabla resumen de los *Pods* desplegados.

Cabe destacar que estos puertos expuestos son dentro del cluster a través del *Service* asociado. Muchos de estos *Service* no es necesario que se expongan hacia fuera por ejemplo a *localhost*. En el caso de otros si que será necesario, por ejemplo *Kong*, *Konga*, *Grafana*, *Kibana*, etc. Para la exposición de puertos, se pueden definir los *Service* como *NodePort*, sin embargo, el puerto asociado no es fijo y está en el rango 30000 al 32767. Por este motivo, cada vez que se quiera acceder a un *Service* desde el exterior se usará una forma más manual a través de *kubectl port-forward*. Esta claro que para un entorno más realista se debería definir un *load balancer* o un *ingress controller*. Con todos los cambios mencionados, la solución puede ser desplegada en un cluster de *Kubernetes*. Todos los fichero se encuentran en el directorio *kubernetes*

del repositorio.

3.2.12. Automatización del flujo CI/CD con *Postman* y *Jenkins*.

Como se ha visto, la solución ya está disponible y parece que funciona correctamente. Sin embargo, es muy común que al seguir haciendo pruebas se encuentren algunos *bugs* en las API que se deben solucionar. O simplemente, se requiere una actualización que se adecue a la nueva versión *CAMARA* o a los nuevos requisitos de los clientes. En estos casos, es necesario actualizar el código, crear una nueva imagen subirla al repositorio, volver a desplegar la solución y posteriormente hacer de nuevo el *testing*. Este proceso es un poco engorroso ya que para una simple actualización de código, hay que realizar varias actividades que fáciles o no llevan un tiempo. Es aquí donde entra lo que se conoce como CI/CD (*Continuous Integration and Continuous Delivery/Continuous Deployment*). CI/CD hace referencia al proceso de automatización de la fase de integración, pruebas, entrega y despliegue. Esto es una práctica eficiente y aumenta la velocidad de gestión de entornos de microservicios. A medida que crecen los servicios a gestionar, más complicada es la actualización y mantenimiento de la arquitectura. En el caso de este trabajo, se quiere implementar un *pipeline* de CI/CD que agilice y automatice las tareas de compilación y pruebas unitarias, creación de imágenes, carga de imágenes en registro, despliegue de solución en entorno de preproducción, pruebas en entorno de preproducción y despliegue de la solución en producción.

Para hacer esto hay que tener en cuenta varias cosas. Aunque no es un entorno preproductivo, se va a utilizar el despliegue con *Docker Compose* como si lo fuese. Una vez desplegada la solución con *Docker Compose*, se van a realizar una serie de pruebas automáticas. Las pruebas hasta ahora se han hecho a través de *Postman*. Hay que tener en cuenta que las pruebas no incluyen únicamente peticiones a las API sino que se quiere probar el flujo completo. Esto incluye la solicitud del token de acceso a *Keycloak* y las peticiones a las API a través del API GW incluyendo este token. Para hacer esto, *Postman* ofrece una pestaña llamada *Pre-request scripts*. En esta pestaña, se incluyen todas las actividades que se quieren realizar previas a las consultas del API. Por lo tanto, en esta sección se va a incluir un *script* que obtenga el token de acceso haciendo la petición oportuna a *Keycloak* a través del API GW siguiendo la arquitectura definida en la figura 3.17. Una vez se obtiene el token de acceso, se va a guardar en una variable local de *Postman* para poder introducirlo en el cuerpo de las peticiones del API. De esta manera, se definen todos los casos de prueba.

Con estas peticiones, se pueden hacer las pruebas *end to end* desde *Postman* de una manera sencilla. El siguiente paso es buscar la forma de automatizarlas. En este punto entra en juego la herramienta *Jenkins*. *Jenkins* va a ofrecer una interfaz web para crear *pipelines* a través de *scripts* de *Groovy* que permiten la automatización de cada una de las fases que se definan en el *script*. Las fases que se van implementar en este proyecto son las necesarias para soportar las actividades que se han explicado previamente. Por lo tanto, habrá una primera fase que haga la compilación y pruebas

unitarias con *Maven* de las API. Un segundo paso creará las imágenes con estos nuevos ejecutables generados. Un tercer paso va a subir al registro de *Docker Hub* las nuevas imágenes. El siguiente paso va a ser el despliegue de la solución en *Docker Compose*. Una vez desplegado, se realizarán las pruebas de *Postman* y por último se hará el despliegue en *Kubernetes*. *Jenkins* garantiza el orden y si alguna de las fases fallas, fallará el *pipeline*. El flujo se describe a continuación:

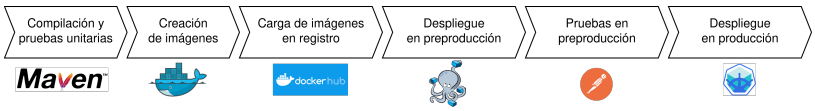


Figura 3.25: CI/CD *pipeline*.

Se instala *Jenkins* siguiendo los pasos descritos en la web oficial [53]. Para que *Jenkins* pueda subir las imágenes a *Docker Hub*, se le debe dar la cuenta de usuario a través de la configuración de credenciales. Por otro lado, también se le debe dar acceso de lectura y escritura en el *host*. Aunque la mayoría de las fases están claras, la parte de pruebas no está todavía finalizada. El motivo es que aunque se ha explicado como con *Postman* se pueden realizar las pruebas, *Jenkins* necesita una forma de definirlas a través de instrucciones de línea de comandos para que pueda ejecutarlas. Para ello, se va a utilizar *Newman*. *Newman* es una herramienta que permite ejecutar las solicitudes de Postman a través de la línea de comandos utilizando *Nodejs*. Es necesario instalar *Nodejs*, *Newman* y por último exportar las pruebas de *Postman* en forma de JSON. Para hacer la instalación de *Nodejs* y *Newman* se pueden seguir los pasos descritos en la documentación oficial [54]. La exportación de las pruebas *Postman* en formato JSON se realiza desde la sección de Exportar de *Postman*. Las pruebas se pueden encontrar en el directorio *postman* del repositorio del trabajo. Se define el pipeline de *Jenkins* y se ejecuta para ver el resultado. Se muestra a continuación, una imagen de la interfaz de *Jenkins* tras varios despliegues:

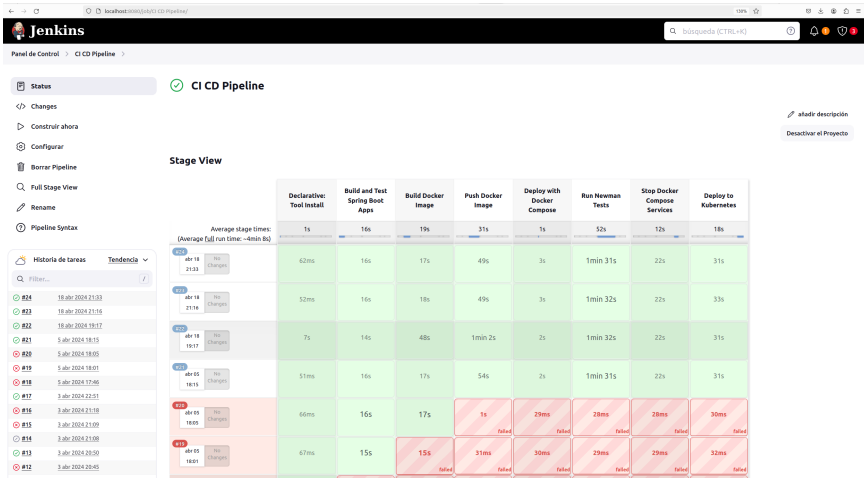


Figura 3.26: Interfaz de *Jenkins* tras el resultado de la automatización.

3.2.13. Resumen de la implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*.

A continuación, se muestra una figura resumen de la arquitectura implementada:

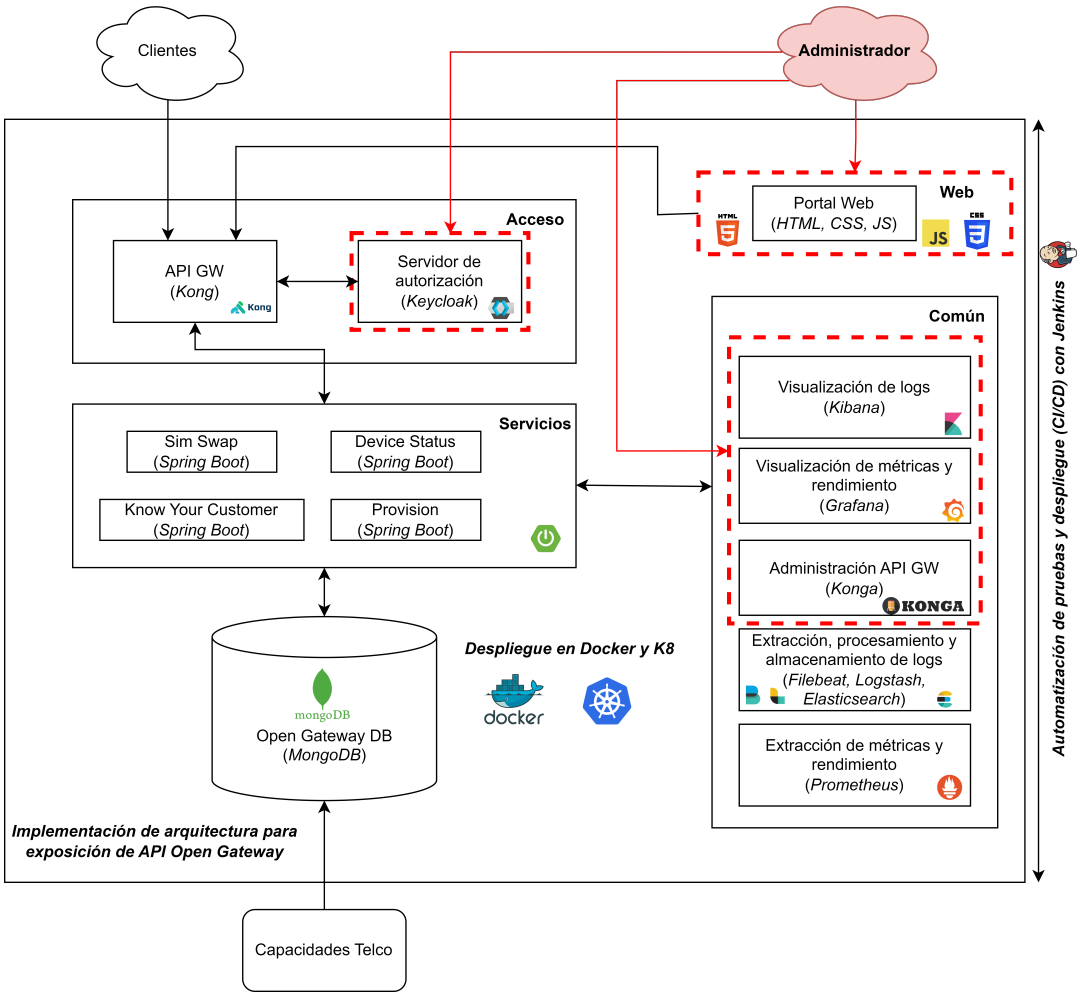


Figura 3.27: Esquema de la implementación de la arquitectura basada en microservicios para la exposición de API en *Open Gateway*.

En esta arquitectura se representan todos los componentes involucrados. Además, se puede apreciar, marcados en rojo, los componentes que el administrador de sistemas debería gestionar. Esto incluye desde la visualización de métricas y *logs*, el portal web, el servidor de autorización y la administración del API GW. Todas estas herramientas simplifican el ciclo de vida y la gestión de la plataforma y servicios ya que poseen interfaces gráficas y evitan tener que usar línea de comandos. Esto le permite a las empresas contratar a personal que no tenga tanto conocimiento técnico para la administración de la plataforma.

A continuación, se va a mostrar una tabla resumen de las herramientas utilizadas, la funcionalidad cubierta y el objetivo perseguido:

Objetivo	**Funcionalidad**	**Herramienta**
Tener visibilidad y trazabilidad de lo que ocurre en la ejecución de los servicios en tiempo real para detectar posibles fallos o para auditorías.	Detección y exposición de los *logs* de todos los servicios.	*Filebeat*
Tener visibilidad y trazabilidad de lo que ocurre en la ejecución de los servicios en tiempo real para detectar posibles fallos o para auditorías.	Procesamiento de los *logs*.	*Logstash*
Tener visibilidad y trazabilidad de lo que ocurre en la ejecución de los servicios en tiempo real para detectar posibles fallos o para auditorías.	Persistencia y almacenamiento de los *logs*.	*Elasticsearch*
Tener visibilidad y trazabilidad de lo que ocurre en la ejecución de los servicios en tiempo real para detectar posibles fallos o para auditorías.	Visualización de los *logs*.	*Kibana*
Tener información de las métricas de los servicios para optimización de recursos y rendimiento.	Obtención y almacenamiento de las métricas de los servicios.	*Prometheus*
Tener información de las métricas de los servicios para optimización de recursos y rendimiento.	Visualización de las métricas de los servicios.	*Grafana*
Tener un control de acceso y un punto único de entrada para una mejor gestión de las API.	API GW de código abierto para cubrir el objetivo.	*Kong*
Tener un control de acceso y un punto único de entrada para una mejor gestión de las API.	Interfaz gráfica de API GW para facilitar la gestión del mismo.	*Konga*
Control de acceso a las API.	Ofrece una solución para gestionar el acceso de los clientes autorizados.	*Keycloak*
Base de clientes centralizada con la información relativa a *Open Gateway*.	Base de datos no relacional para almacenar los datos necesarios en las API de *Open Gateway*.	*Mongo*
Automatizar el ciclo de vida de las API para una mejor gestión de las mismas.	Plataforma para la automatización de las pruebas y despliegue.	*Jenkins*

Tabla 3.4: Tabla resumen de los objetivos, funcionalidades y herramientas empleadas.

Capítulo 4

Diseño e Implementación de la aplicación antifraude.

En este capítulo se va a explicar el proceso seguido para crear un caso de uso que ejemplifique posibles escenarios para las API de *Open Gateway*. El caso de uso consistirá en una aplicación móvil, que represente una entidad bancaria, en la que el uso de las API de *Open Gateway* puede prevenir de algunas transacciones o acciones fraudulentas. Se contará a lo largo de este capítulo, la fase de ideación, diseño e implementación. La aplicación se diseñara de una manera abstracta para cualquier tipo de sistema operativo. Sin embargo, solo se implementará para móviles *Android*

4.1. Aplicación *Sandbank*.

En esta sección se va a detallar y profundizar en la fase de ideación. Tal y como se ha contado en los capítulos anteriores, tanto el API de KYC como de *Sim Swap* o *Device Status* están muy enfocadas en casos de uso antifraude. Esto es algo muy genérico y realmente hay que aterrizar casos de uso concretos y que despierten el interés de los clientes.

Uno de los temores más comunes de las aplicaciones es la suplantación de identidad. Esto es, los usuarios se registran en las aplicaciones con datos erróneos, incorrectos o incompletos. Los motivos pueden ser varios, o bien el usuario ha cometido algún error a la hora de introducir los datos, o se introducen erróneamente a propósito con el objetivo de cometer alguna acción ilegal, que no se utilicen sus datos reales, o cargar facturas contra otro usuario. En muchas aplicaciones, tener los datos incorrectos de un usuario no tiene gran importancia, sin embargo, existen otras aplicaciones donde esta información es crítica. Uno de los casos más claros son las entidades bancarias, ya sea para crear una cuenta o cualquier otra actividad, el banco necesita tener información de sus clientes correcta y actualizada.

Una de las ideas que pueden tener estas entidades bancarias para corroborar los datos que dan sus clientes sería contrastar la información con otras empresas que también necesiten o suelan tener información verídica de las personas. Este tipo de empresas podría ser perfectamente una operadora de telecomunicaciones. Aunque las

operadoras son empresas en las que los datos de los usuarios suelen ser verídicos, una fuente más fiable aún sería tener la información directamente del gobierno o la policía.

Sería muy sencillo para los bancos contrastar la información con la de las operadoras de telecomunicaciones gracias a las API de *Open Gateway*. Como se ha visto en este trabajo, una de las características principales de *Open Gateway* es que se estandarizan y acuerdan las especificaciones de las API entre la gran mayoría de operadoras. Por lo tanto, la entidad bancaria no se preocuparía si el usuario es de Movistar, Orange o Vodafone y lo podría verificar indistintamente. Por otro lado, verificar la información con datos estatales podría ser dificultoso. La principal razón es que es posible que este tipo de información no se pueda consultar a través de API y aunque se pudiese consultar, habría que analizar múltiples aspectos legales sobre la consulta de empresas de datos de la población a figuras gubernamentales.

Otro de los temas en los que podría estar interesado una entidad bancaria es en saber si sus clientes han cambiado de terminal o tarjeta SIM últimamente. La principal razón es que muchas de las compras actuales, se hacen desde dispositivos móviles. Esto tiene ciertas ventajas y desventejas. Como punto fuerte, los usuarios se pueden olvidar del dinero en efectivo e incluso de las tarjetas de crédito o débito. En lugar de ellas se puede pagar desde el móvil. Además, para las compras *online* se suele pedir que el usuario autentique la compra desde la aplicación del banco. Por lo tanto, el móvil se ha convertido en una herramienta principal para cualquier tipo de estafa.

Los métodos tradicionales de robo en el que a base de amenazas, extorsión o agresiones físicas se consigue el móvil del usuario, tienen poco efecto si el dispositivo está bloqueado. Si bien es cierto que existen metodologías para desbloquear los móviles, cada día se implementan más mecanismos de seguridad y es más complejo. Bajo estas circunstancias, los delincuentes optan por otras técnicas ya que un móvil bloqueado no tiene ninguna validez. Una de estas técnicas, consiste en obtener una nueva tarjeta SIM del usuario. De nuevo, intentar robar un teléfono extraer la SIM y usarla en otro dispositivo no parece lo más eficiente ya que de nuevo surge el problema de las contraseñas y PIN de seguridad.

La manera más habitual en la que un estafador obtiene una nueva SIM tiene diferentes pasos. En primer lugar, obtiene información del usuario mediante *Phising* u otras técnicas. Posteriormente, con esta información es capaz de suplantar la identidad del usuario y solicita una nueva SIM a la operadora oportuna. Con la nueva SIM, el estafador es capaz de acceder a la mayoría de información del usuario y además, hace totalmente inservible los códigos típicos de verificación SMS. Si el banco pudiese tener información actualizada de cuando fue la última vez que el usuario cambio de SIM, podría cancelar o pedir otro tipo de verificaciones a estos usuarios.

Dado la gran relevancia de las API implementadas en el sector bancario, se va a diseñar e implementar una aplicación móvil bancaria sencilla que ilustre de alguna manera como se podrían usar las API de *Open Gateway* para las problemáticas

explicadas previamente. La aplicación se va a llamar *Sandbank* haciendo un juego de palabras con banco y con *sandbox*, haciendo referencia a que esta aplicación será una prueba para los casos de uso de *Open Gateway*.

4.2. Diseño de Sandbank.

Tal y como se ha explicado, *Sandbank* es una aplicación bancaria sencilla donde se ilustrará como a través de las API de *Open Gateway*, los bancos pueden resolver los problemas de suplantación de identidad o duplicación de SIM. La aplicación se va a integrar con la arquitectura de microservicios que se ha implementado previamente. La aplicación obtendrá la información de las API de *Open Gateway* realizando la consulta al API GW. El API GW redirigirá la petición al servicio, teniendo en cuenta que la autorización es exitosa. Aunque se entrará en mayor detalle en la fase implementación, se muestra la arquitectura de integración:

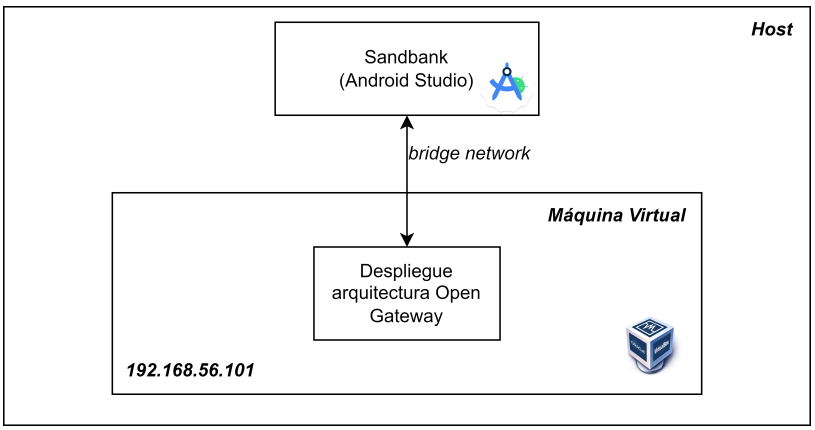


Figura 4.1: Arquitectura de integración entre *Sandbank* y API *CAMARA*.

A nivel de pantallas en la aplicación, la aplicación va a contar con nueve ventanas. El diseño de las ventanas se ha hecho a través de *Figma*. *Figma* también es una aplicación gratuita y es muy útil para el diseño de aplicaciones y páginas web ya que es muy intuitivo y cuenta con una amplia variedad de herramientas. El diseño se puede encontrar de manera pública en la herramienta [55]. El diseño se va a basar en aplicaciones reales como la de *Imagin Bank*, *App Santander*, *Smartbank app*, *BBVA app*, etc. Sandbank debe permitir que los clientes manejen su dinero desde una cuenta de la aplicación.

La primera pantalla que se va a crear es la de bienvenida a la aplicación (*Welcome View*). En esta ventana, debe mostrar un botón para iniciar sesión y otro para registrarse si los usuarios no tienen cuenta. Se incluye además en el diseño el logo de la aplicación. Se puede ver el resultado en la siguiente imagen:

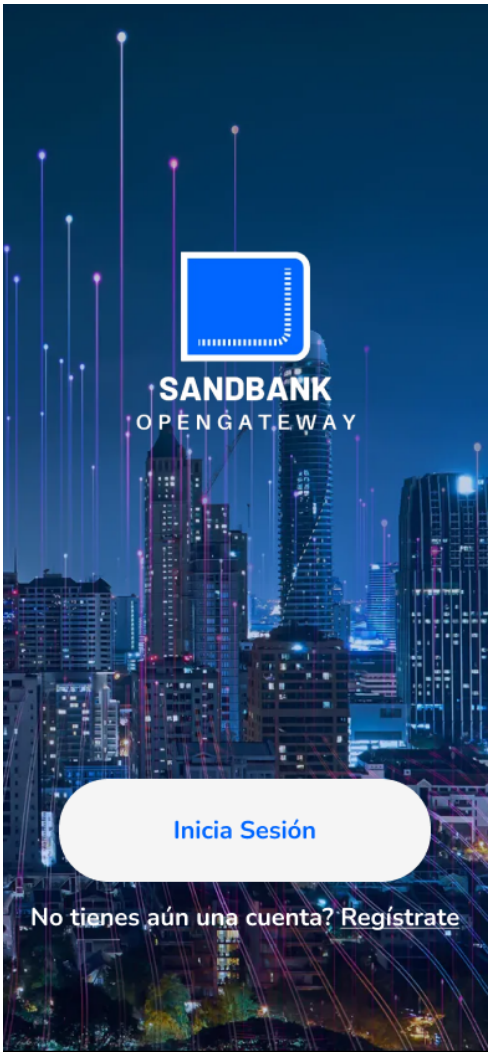


Figura 4.2: Página de Bienvenida de *Sandbank*.

Desde esta ventana, si se hace clic en el botón de Inicia Sesión, se redirigirá a una nueva ventana que permita al usuario identificarse. Esta ventana va a ser la de inicio de sesión (*Log in view*). Para la identificación del usuario en la ventana de inicio de sesión, se va a utilizar el DNI como identificador y una contraseña que se debe crear en la página de registro. En toda página de inicio de sesión de cualquier aplicación, hay una opción que permite recuperar la contraseña en caso de pérdida. Sin embargo, aunque se va a añadir un botón que haga referencia a esta funcionalidad, la implementación va a quedar fuera del alcance. En el caso de la ventana de registro (*Create account view*), el usuario va a tener que introducir sus datos personales para crear la cuenta. Dado que es una aplicación bancaria, se van a tener que introducir la mayoría de datos personales como pueden ser el DNI, el número de teléfono, la fecha de nacimiento, la dirección del domicilio o el correo electrónico. Se van a poner diferentes casillas para cumplimentar esta información. La información se pretende contrastar con la que se recibe del API de KYC, por lo tanto, conociendo los campos que devuelve el API se va a diseñar la estructura que más fácil permita comparar los datos introducidos por

el cliente y los que devuelve el API. Se muestra a continuación estas dos ventanas:



Figura 4.3: Página de inicio de sesión de *Sandbank*.



Figura 4.4: Página de creación de cuenta de *Sandbank*.

La idea detrás de la validación de los datos introducidos con los obtenidos por el API de KYC es que el usuario no pueda crear la cuenta si los datos no son verídicos. Se explicará más en la fase de implementación, pero se tendrá en cuenta el número de campos que no coinciden con los que devuelve el API de *Open Gateway* para habilitar o no la creación de la cuenta. En caso de que se decida que no hay suficientes datos verídicos, se va a redirigir a una ventana de error (*Error Creating Account View*). En el caso de que la creación de la cuenta sea exitosa, se va a redirigir a la ventana del menú principal (*Main Menu View*). A la ventana del menú principal también se va a acceder si los datos de inicio de sesión son correctos.

En la ventana del menú principal, el usuario va a poder realizar varias actividades para la gestión de su cuenta y sus ahorros. En primer lugar, se va a mostrar la cantidad de dinero que tiene el usuario en la cuenta y su número de cuenta. Además, se van a mostrar también las últimas tres transferencias que se han realizado indicando la fecha de las mismas. Por otra parte, se pueden encontrar tres botones en el menú. El botón de la parte inferior derecha, va a permitir simplemente cerrar sesión y salir de la aplicación. El botón de la parte inferior izquierda va a servir para que el usuario pueda realizar una transferencia con el dinero disponible (*Transfer View)*). Por último, se

muestra también un botón en la parte superior derecha cuya funcionalidad va a ser mostrar el perfil del usuario y poder modificar los datos personales (*Profile View*). Se muestran a continuación, la ventana de error al crear la cuenta, el menú principal, la ventana de transferencia y la pantalla de vista del perfil.

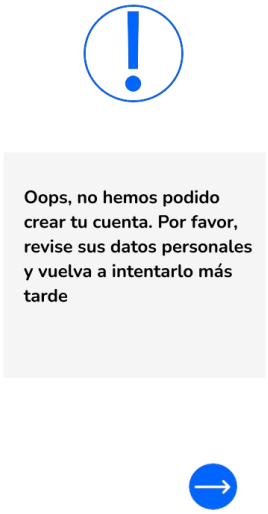


Figura 4.5: Página de error en la creación de cuenta de *Sandbank*.

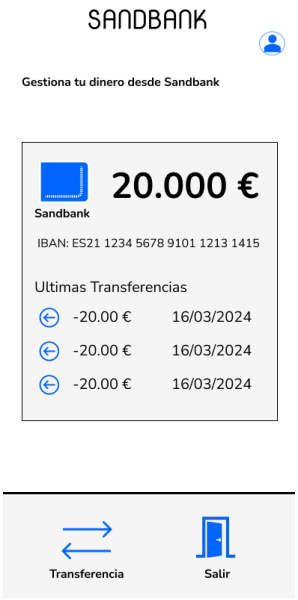


Figura 4.6: Vista del menú principal de *Sandbank*.

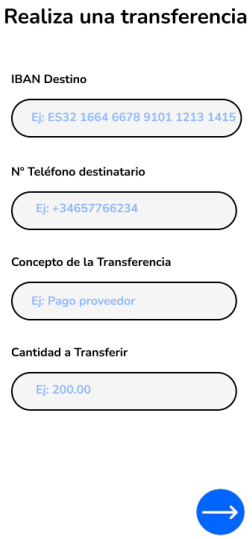


Figura 4.7: Página de lanzamiento de transferencia de *Sandbank*.



Figura 4.8: Página de perfil de usuario de *Sandbank*.

Para poder realizar una transferencia, el usuario debe introducir el número de cuenta destino, el número de teléfono del destinatario, el concepto de la transferencia y la cantidad a transferir. En esencia, el número de teléfono del destinatario puede ser prescindible ya que pueden existir casos que las cuentas representen a empresas y no a particulares. Sin embargo, en esta aplicación se va a pedir este campo para posibles casos de uso en los que la aplicación notifica al usuario que la entidad a la que se le va a transferir dinero no corresponde con la entidad del número introducido. De esta manera, se pueden evitar algunas transferencias erróneas. Antes de que se lleve a cabo la transferencia, se va a validar si el usuario ha cambiado recientemente de SIM. Si el usuario ha cambiado recientemente de SIM y la cantidad que se quiere transferir supera el 90 % de lo disponible, se bloquea la transacción y se muestra una ventana de error (*Transfer Error View*). Para obtener la información del cambio de SIM se usará el API de *Sim Swap* de *Open Gateway*. En el caso de la ventana de perfil de usuario, simplemente se muestran el DNI, número de cuenta y contraseña del usuario. Para poder actualizar los datos hay que pinchar en el botón indicado, redirigiendo la aplicación a la ventana de actualización. Además, se incluye un botón que regresa al menú principal. Se muestran las ventanas de error de transferencia y de actualización de usuario.

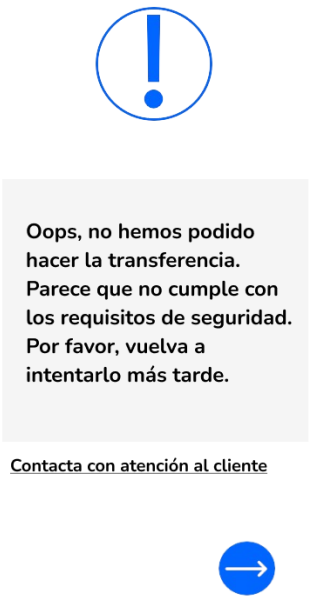


Figura 4.9: Página de error al realizar una transferencia en *Sandbank*.



Figura 4.10: Página de actualización de datos de cliente en *Sandbank*.

En la página de actualización de datos, el usuario puede modificar los datos que había introducido para la creación de la cuenta. Aunque aquí también se podría validar los datos actualizados con los que devuelve el API KYC, quizás deja de ser tan útil ya

que en la primera verificación ya se comprueba la validez del usuario y es posible que los datos que se están intentado actualizar, no estén actualizados en la operadora de telecomunicaciones. Por lo tanto, se va a permitir la modificación de los campos sin hacer la verificación con KYC. Si salta la ventana de error para hacer la transferencia porque se dan las circunstancias descritas previamente, el botón de la parte inferior derecha permite al usuario volver al menú principal.

4.3. Implementación de *Sandbank* con *Android Studio*

Una vez que se ha explicado el motivo, la razón y el diseño de la aplicación se va a pasar a la implementación. Tal y como se ha dicho previamente, se va a implementar la aplicación solo para *Android* y se va a utilizar para ello *Android Studio*. *Android Studio* permite realizar el desarrollo de aplicaciones de una manera sencilla incluyendo incluso un emulador de terminal para las ejecuciones. El desarrollo de aplicaciones *Android* se ha hecho tradicionalmente en *Java*. Sin embargo, en los últimos tiempos ha surgido un nuevo lenguaje de programación *Kotlin* optimizado para el desarrollo en *Android*. Aunque cada vez se usa más *Kotlin*, para este trabajo se ha decidido hacer la implementación en Java debido al conocimiento previo y a la posibilidad de reutilizar código de las implementaciones que se han hecho de las API de *Open Gateway*.

Aunque la manera quizás más sencilla de desarrollar la aplicación es usar *Android Studio* en la máquina virtual de *Ubuntu* que se ha venido utilizando, se ha decidido desarrollar la aplicación en el sistema *Host* (*Windows*) de la VM por algunas razones. En primer lugar, aunque se ha destinado una gran cantidad de recursos, va a ser preferible trabajar en el *Host* con todas las capacidades del ordenador dado que *Android Studio* consume una gran cantidad de recursos. Por otra parte, es algo poco realista que la aplicación corra en la misma máquina que las API del *Open Gateway* ya que serían desarrollos de empresas diferentes. De esta manera, la aplicación no va a llamar a las API a través de la red *localhost* sino que se va a realizar una conexión puente entre la VM y el *Host* para habilitar la comunicación.

La conexión entre la VM y el *Host* se hace de manera sencilla a través de *VirtualBox*. En *VirtualBox*, se accede a los ajustes de la VM y en la sección de red se añade un nuevo adaptador distinto a NAT que es el que viene por defecto y permite la conexión a internet. Para el nuevo adaptador hay que seleccionar la opción de adaptador solo anfitrión. Con esta configuración si se accede a los terminales tanto de la VM como del Host se encuentran las IP de las máquinas para esta conexión. La IP de la VM que es la que va a utilizar para llamar a las API es 192.168.56.101.

Al hacer la implementación en base a esta IP que es concreta del entorno local, imposibilita la prueba en otros entornos. La solución más práctica en el caso de probar la solución sería desplegar todo en la misma máquina con *Docker* o *Kubernetes* y hacer las llamada al API desde *localhost*. De igual manera, para evitar problemas y que se pueda probar la aplicación sin influencia del despliegue de las API se van a crear dos versiones. Se va a crear una primera versión de la aplicación en la que no existen

llamadas a las API de *Open Gateway*. Esto significa que va a haber siempre creación de cuenta independientemente del API KYC y también se van a hacer las transferencias independientemente de lo que respondiese *Sim Swap*. Se va a prescindir de las API y se va a tener una versión funcional de la aplicación bancaria que se puede probar de manera sencilla. Por otro lado, se implementará una segunda versión que incluya toda la funcionalidad descrita en la sección anterior. El código fuente de ambas versiones está disponible en el repositorio de este trabajo.

Uno de los aspectos primordiales de la aplicación es la gestión de la información de usuarios. Dado que se puede iniciar la sesión una vez que se ha creado la cuenta, la aplicación debe llevar cuenta de que usuarios hay registrados y cual es su información (datos personales, dinero en la cuenta, número de cuenta, etc). Es evidente que para hacer esto es necesario emplear una base de datos. Aunque se podría conectar la aplicación con una base de datos real y más preparada para entornos de producción como puedo ser *Postgresql*, se va a decidir utilizar *Sqlite*. Con *Sqlite* la integración es muy rápida ya que viene integrado en *Android Studio*. Para ello, se va a extender *SQLiteOpenHelper* y esto se va a encargar de crear, abrir y gestionar la base de datos *Sqlite*.

Se van a definir todos los campos que se van a almacenar en la tabla y algunas de las operaciones que se utilizarán en el proyecto. Los campos que se deben almacenar por usuario son DNI, número de teléfono, contraseña, *email*, fecha de nacimiento, nombres y apellidos, nombre de la dirección, número de la dirección, código postal, región, localidad, país, número de cuenta bancaria, piso y dinero disponible en la cuenta. Algunas de las operaciones son la consulta de clientes, actualización de parámetros o la consulta de si un usuario existe o no.

Una vez se entiende como se va a gestionar la persistencia de los datos y la aplicación, se va a ir añadiendo las ventanas y la funcionalidad esperada tal y como se explico en diseño. Para copiar las ventanas con el formato diseñado se va a utilizar la carpeta de *resources*. A su vez, en esta carpeta se definen una serie de subcarpetas. En primer lugar y más importante se encuentra la carpeta de *layout* que es donde se van a especificar las vistas con los botones, cajas, texto , etc son sus respectivos formatos. Para tener un cierto orden, se tiene la carpeta *drawable*, *font* y *values*. En *drawable*, se tienen todas las imágenes que se van a utilizar en la aplicación, desde los fondos de pantalla hasta los botones. En la carpeta *font*, se especifican los tipos de letras que se van a usar. Principalmente se ha usado *Nunito*. Por otro lado, en la carpeta *values* se especifican algunos aspectos como los colores o márgenes. Existen también otras carpetas como *navigation*, *mipmap* o *xml*. En la carpeta *mipmap* se define el logo de la aplicación. En el caso de *navigation*, se establece como es el flujo y movimiento entre pantallas. por último, en *xml* se habilitará, por ejemplo, la conexión con la VM.

En la implementación de las clases de Java, se va a implementar normalmente un método para cuando se crea la ventana y otro para cuando se realiza una actividad pinchando en un botón determinado. Para crear la cuenta, además de las API de *Open*

Gateway, se van a imponer una serie de requisitos en los campos. Estos se van a reusar de los que se hicieron para el API de provisión. Por refrescar lo que se hizo, a través de expresiones regulares se establecen normas en los formatos, por ejemplo, la fecha de nacimiento en formato Año-mes-día. Si no se cumple el formato, no se va a poder proceder con la creación de la cuenta y se va a notificar cual es el campo incorrecto. Cuando se crea la cuenta, se va a generar un IBAN nuevo por defecto. El número del IBAN es generado de manera ordenada. Cada usuario registrado, se aumenta en una unidad. Aunque esto no es muy realista, se considera suficiente. Por otro lado, se debe comprobar antes de crear la cuenta si ya existe un usuario registrado con ese DNI, en ese caso, se muestra la ventana de error. Para las cuentas recién creadas, se va a generar automáticamente una cuenta con 20.000 euros.

una vez que la cuenta esta creado, si se intenta modificar los datos de usaurio se va a poder de manera sencilla para todos los campos menos DNI y contraseña que son los campos de inicio de sesión. Por otro lado, para hacer las transferencias, tal y como se ha explicado, se va a restar la cantidad a transferir de lo disponible y se va a llevar un registro de las últimas tres realizadas. Por supuesto, si la cantidad a transferir es superior a lo disponible, se envía la ventana de error y se bloquea la transacción.

Con todo esto, se tiene la primera versión funcional de la aplicación. A continuación, se va a añadir la segunda versión con las API de Open Gateway. Para ello, se ha usado la dependencia de *retrofit*. Esta dependencia facilita la interacción con API web REST, proporcionando una interfaz simple para realizar solicitudes HTTP. En las clases apropiadas, se hace la petición a las API KYC y *Sim Swap*. Para la funcionalidad con el API de KYC, se va a considerar que si hay menos de seis campos que devuelven falso, el usuario puede crear su cuenta. Para el caso de *Sim Swap*, se va a hacer uso del *endpoint* de *check* ya que devuelve directamente un booleando evitando tener que tratar con el *timestamp*. En este caso, se va a preguntar si el usuario ha cambiado de Sim en los últimos 240 días. Si la respuesta es verdadera y además se quiere transferir el 90 % de lo disponible, se bloquea la transacción. Quizás 240 días es un lapso muy grande y el caso de uso cobra sentido en poco margen de tiempo, como uno o dos días. Sin embargo, por el tipo de datos que hay guardados en la base de datos de *Open Gateway*, es muy difícil que haya clientes con un cambio de Sim tan reciente. Por lo tanto para ver casos de error, se ha decidido implementar un valor tan grande.

4.4. Pruebas de la aplicación.

En esta sección, se van a mostrar una serie de pruebas que se han realizado con la aplicación. Además, se detallan algunos puntos clave para que se pueda probar la aplicación exitosamente. En primer lugar, se recomienda probar primero la versión que no incluye las API de *Open Gateway*. Lo primero que se debe hacer es registrar una cuenta ya que la base de datos está inicialmente vacía. Para el registro, se van a implementar una serie de reglas tal y como se ha explicado. Aunque ya se detallaron las reglas en la fase de diseño, se va a hacer un rápido resumen para que cualquiera que pruebe la aplicación pueda usarla de manera sencilla.

Para el nombre y apellidos lo único que se pide es que no superen los 30 caracteres. En el caso del DNI, debe estar formado por 8 cifras y una letra. Para el correo electrónico se pide que siga el formato *caracteres1@caracteres2.string*. El *string* debe estar formado cómo máximo por 2 letras. Para el número de teléfono se sigue el formato estándar y se pide el símbolo + obligatorio. La fecha de nacimiento sigue el formato Año-Mes-Día. Para el nombre de la calle, la región, la localidad y el país se pide que este formado como máximo por 4 *strings*. El código postal debe estar entre 1 y 7 dígitos. En el caso del piso, como máximo debe estar formado por 10 caracteres. Por último, la contraseña debe contener una mayúscula y un dígito. Se muestra a continuación un ejemplo con la fecha de nacimiento errónea:

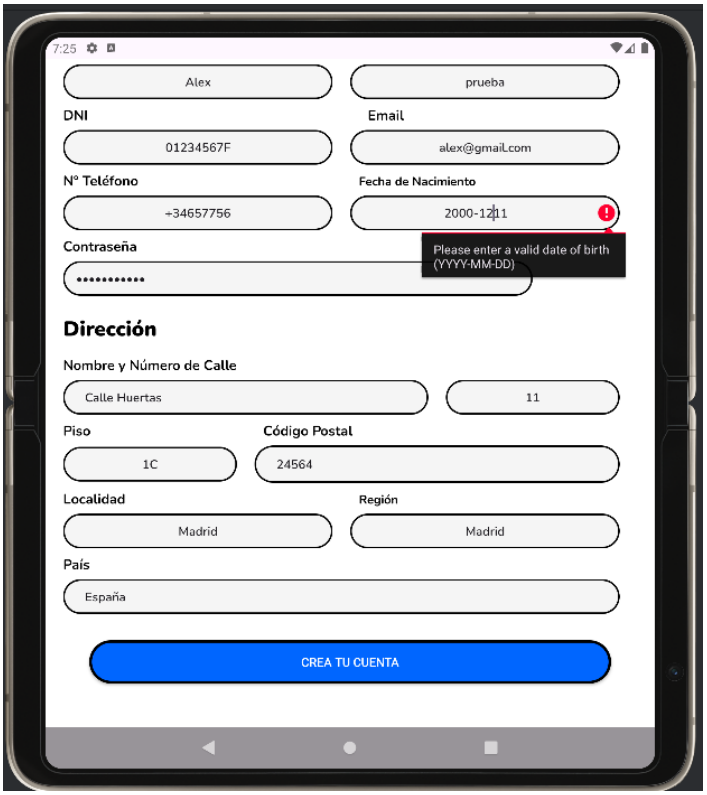


Figura 4.11: Prueba de la ventana de registro de *Sandbank*.

Una vez el registro ha sido exitoso, se redirige al menú principal. Para el caso en el que se quiere hacer una transferencia, se deben introducir los datos de número de cuenta del destinatario, concepto de la transferencia, número de teléfono del destinatario y cantidad a transferir. El número de teléfono debe seguir el formato especificado anteriormente. En el caso del número de cuenta, se va a pedir un número de una entidad bancaria española. Esto significa que el formato es el siguiente ESXX YYYY ZZZZ WWWW VVVV AAAA. Siendo la X, Y, Z, W, V y A dígitos. La cantidad a transferir debe ser un número y el concepto cualquier carácter. Se muestra un ejemplo de configuración y resultado tras restar la cantidad a transferir de lo disponible.



Figura 4.12: Prueba de la ventana de transferencia de *Sandbank*.



Figura 4.13: Prueba del menú principal de *Sandbank* tras transferencia.

Por último, se pueden modificar todos los datos personales salvo el DNI y la contraseña. Se muestra a continuación esta vista:

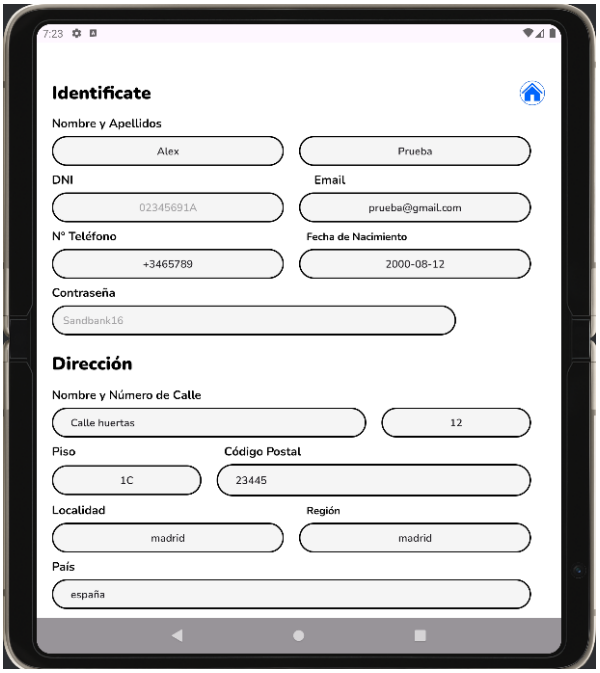


Figura 4.14: Prueba del menú de actualización de datos en *Sandbank*.

Si se quiere probar la aplicación con la interacción de las API, se recomienda desplegar las mismas con *Kubernetes* o *Docker Compose* y modificar el código de la app para hacer las peticiones a *localhost* en lugar de la IP de la VM. Los ficheros que habría que modificar son *CreateAccountActivity.java, TransferWindowActivity.java* y network_security_config.xml. Este último fichero se encuentra en la carpeta xml dentro de res (*resources*) y sirve para habilitar las llamadas a esta dirección que están deshabilitadas por defecto por razones de seguridad.

Capítulo 5

Conclusiones y trabajos futuros

A modo de conclusión, este trabajo ha mostrado cómo las API de *Open Gateway* pueden transformar el sector de telecomunicaciones. Se ha reflejado también algunos de los puntos críticos del proyecto como son la seguridad y privacidad de datos personales. Para demostrar el impacto de las API, se ha desarrollado un entorno de microservicios que podría servir para dar este tipo de servicios. Para ello, se ha partido de la especificación del API que se estandariza en el foro de la GSMA *CAMARA*. El desarrollo ha incluido no solo la funcionalidad de las API *Sim Swap*, *Know Your Customer* y *Device Status*, sino plataformas para la monitorización (*Prometheus* y *Grafana*) que permiten identificar cuellos de botella y problemas de funcionamiento, centralización y almacenamiento de *logs* que permitan tener trazabilidad de que es lo que ocurre en los servicios a través de *Elasticsearch* , *Logstash* y *Kibana*. Control de seguridad con *Keycloak* para dar servicio únicamente a los clientes autorizados, etc. Además, se ha incluido en este entorno un API GW como único punto de entrada, facilitando el acceso y permitiendo un mayor y mejor control de la entrada a las API.

Por otra parte, la arquitectura incluye un portal web de gestión de las API para administradores. Este portal web permite gestionar la base de clientes (*Mongo DB*), añadiendo, modificando, obteniendo o eliminando a estos. También se ofrece la posibilidad de probar las API de *Open Gateway* desde este portal, evitando el uso del terminal o herramientas como *Postman*. Cabe destacar que la solución se ofrece completamente basada en contenedores mediante *Docker* y con los manifiestos necesarios para hacer el despliegue con *Docker Compose* y *Kubernetes*. Dado que hacer cambios en algunos de los servicios y hacer el despliegue en estos entornos puede ser un poco arduo y tedioso, se han elaborado *scripts* de *Jenkins* para automatizar los flujos de pruebas y despliegues.

El despliegue que se ha realizado ha sido en local a través de *Minikube*. Como trabajo a futuro y mejora, se podría hacer un despliegue en diferentes proveedores de nubes públicas comparando el coste y rendimiento. Esto sería sencillo ya que al tener los manifiestos de *Kubernetes*, solo habría que montar un cluster en la nube para poder hacer el despliegue. Aunque la nube pública es el entorno más escalable y que más fuerza está cogiendo en los últimos años, también se podría desplegar de la misma forma en entornos de nube privada con distribuciones como *Red Hat Openshift*

o *VMWare Tanzu*.

Aunque se ha cubierto gran parte de lo que se puede esperar en este tipo de arquitecturas, una implementación real tendría que tratar también otros aspectos que no se han visto como es la facturación y cobro por el uso de las API. Esto implica una definición del modelo de negocio y posiblemente un gestor de sesiones que controlase quién y cuánto está usando las API.

Por otra parte, han quedado fuera del alcance del trabajo las nuevas API con funcionalidad de notificaciones. Este nuevo modelo requeriría introducir en la arquitectura alguna herramienta como *Kafka*, *Solace* o similar que permiten trabajar con modelos de publicación y suscripción a eventos. A nivel de mejoras o trabajos futuros, se podrían retocar varios aspectos. En primer lugar, aunque se han personalizado las excepciones en las API *Spring Boot*, las excepciones de autorización y autenticación las debe enviar el API GW. Se podría actualizar el *script* de *lua* para enviar concretamente la excepción que se define en *CAMARA* para estos aspectos.

A nivel de monitorización, se podrían implementar nuevas métricas que expusiesen el tiempo de ejecución de cada uno de los métodos o de las consultas a la base de datos. Con respecto a los *logs*, se podría añadir un formato más enriquecedor con más detalle y que permita tener mejor trazabilidad. Con respecto a mejoras del portal web, se podría mejorar el formato y la estética usando *frameworks* mas profesionales como *React*.Además, se podría introducir un mecanismo de autenticación que devuelva el token necesario para hacer las pruebas de las API. En el caso en el que se desplegasen las soluciones en un entorno de *cloud* pública, se podría hacer otro *script* en *Jenkins* para automatizar el despliegue en este nuevo ambiente.

En cuanto a la aplicación desarrollada, se ha mostrado el potencial de las API de *Open Gateway* con un caso de uso sencillo. La aplicación *Android* desarrollada muestra diferentes ventanas en las que el usuario se puede crear una cuenta, iniciar sesión, gestionar y modificar sus datos personales y realizar transferencias bancarias a partir del dinero disponible. Durante el proceso de registro, se hace uso del API de KYC para validar que la información proporcionada es verídica. Además, también se usará el API de *Sim Swap* para bloquear todas aquellas transferencias que superen el 90 % del dinero disponible y cuyo titular haya cambiado de SIM recientemente. Con esto, se ha conseguido demostrar el impacto que pueden tener este tipo de API en casos de uso antifraude y cómo las operadoras de telecomunicaciones pueden vislumbrar nuevas oportunidades y casos de negocio.

La aplicación se podría actualizar para que el comportamiento se asemejase más a una aplicación bancaria real. Esto podría incluir la integración con una base de datos real, funcionalidad para recuperar la contraseña en caso de pérdida, verificación de usuario a través de mensaje SMS, etc. De cualquier modo, con la versión actual se refleja esencialmente los casos antifraude en este tipo de aplicaciones.

En definitiva, el trabajo ha supuesto un gran aprendizaje para mí ya que se ha

trabajado con un amplío número de tecnologías y además, gran parte de la información que se quiere exponer viene de la red móvil, teniendo el estándar de 3GPP un rol importante en este trabajo y el proyecto. Esto me ha permitido mejorar tareas de búsqueda e investigación en documentos tan grandes y complejos. A partir de este trabajo, se puede seguir mejorando el entorno y desarrollando nuevas API y casos de uso. Con todo esto, se espera que este trabajo haya despertado el interés por este proyecto y la exposición de servicios a través de API.

Bibliografía

[1] 3GPP. Release 15 description; summary of rel-15 work items. Technical Report TR 21.915, 3rd Generation Partnership Project (3GPP), 2019. Disponible en portal 3GPP: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3389.

[2] David Castro Sánchez. Diseño de una red de acceso radio 5g (ran-5g) aplicado a la zona portuaria de valencia. Proyecto fin de grado, Escuela Técnica Superior de Ingeniería y Sistemas de Telecomunicación, 2019. Disponible en: http://oa.upm.es/70542/1/TFG_DAVID_CASTRO_SANCHEZ.pdf.

[3] 3GPP. 5g system overview. https://www.3gpp.org/technologies/5g-system-overview, 2022.

[4] Luis Bellido David Fernández. Material docente de la asignatura de Redes Definidas Por Software, Virtualización y Servicios Avanzados de Red del Máster Universitario en Ingeniería de Telecomunicación. ETSIT UPM, 2023.

[5] ETSI. Network functions virtualisation (nfv); architectural framework. https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/01.02.01_60/gs_NFV002v010201p.pdf, Visitada en Abril 2024.

[6] José Ignacio Moreno. Material docente de la asignatura de Redes de Comunicaciones del Máster Universitario en Ingeniería de Telecomunicación. ETSIT UPM, 2023.

[7] Jithesh Sathyan. *Fundamentals of Ems, Nms and Oss/bss*. CRC Press, 2016.

[8] TMForum. Tmforum sobre nosotros. https://www.tmforum.org/about-tm-forum/, Visitada en Abril 2024.

[9] José maría del Álamo y Enrique Barra Victor Villagrá. Material docente de la asignatura de Ciberseguridad y privacidad: gestión y operación del Máster Universitario en Ingeniería de Telecomunicación. ETSIT UPM, 2024.

[10] 3GPP. Release 18 description; summary of rel-18 work items. Technical Report TR 23.501, 3rd Generation Partnership Project (3GPP), 2024. Disponible en portal 3GPP: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144.

[11] CAMARA Project. Scope. https://camaraproject.org/scope/, 2023. Accessed: 2024-06-10.

[12] Theodore S. Rappaport, Shu Sun, Rimma Mayzus, Hang Zhao, Yaniv Azar, Kevin Wang, and Felix Gutierrez. Millimeter wave mobile communications for 5g cellular: It will work! *IEEE Access*, 1, 2013.

[13] Johan Sköld Erik Dahlman, Stefan Parkvall. *5G NR The Next Generation Wireless Access Technology*. Elsevier, 2 edition, 2018.

[14] Seyed Hossein Ahmadpanah, Abdullah Jafari Chashmi, and Majidreza Yadollahi. 4g mobile communication systems: Key technology and evolution. *arXiv preprint arXiv:1606.05477*, 2016.

[15] Jeffrey G. Andrews, Stefano Buzzi, Wan Choi, Stephen V. Hanly, Angel Lozano, Anthony C. K. Soong, and Jianzhong Charlie Zhang. What will 5g be? *IEEE Journal on Selected Areas in Communications*, 32(6), 2014.

[16] Manuel Sierra Castañer y Celia Fernández Aller. *Análisis del impacto del 5G en la sociedad*. Fundación alternativas, 2021.

[17] Ericsson. Network exposure function - service exposure. *Ericsson*, 2023. Accessed: 2024-03-10.

[18] 3GPP. Introducing 3gpp. `https://www.3gpp.org/about-3gpp`, 2023. Accessed: 2024-03-10.

[19] IEEE Standards University. 3gpp 5g core network: An overview and future directions. *IEEE Standards University*, 2023. Accessed: 2024-04-10.

[20] Justus Baron and Kirti Gupta. Unpacking 3gpp standards. *Journal of Economics & Management Strategy*, 27(3):433–461, 2018.

[21] David Wisely, Ning Wang, and Rahim Tafazolli. Capacity and costs for 5g networks in dense urban areas. *IET Communications*, 12(19):2502–2510, 2018.

[22] Hugo Tullberg, Petar Popovski, Zexian Li, Mikko A Uusitalo, Andreas Hoglund, Omer Bulakci, Mikael Fallgren, and Jose F Monserrat. The metis 5g system concept: Meeting the 5g requirements. *IEEE Communications magazine*, 54(12):132–139, 2016.

[23] José Manuel Riera Luis Mendo. Material docente de la asignatura de Comunicaciones Móviles del Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación. ETSIT UPM, 2022.

[24] Sameer Kumar Singh, Rohit Singh, and Brijesh Kumbhani. The evolution of radio access network towards open-ran: Challenges and opportunities. In *2020 IEEE Wireless Communications and Networking Conference Workshops (WCNCW)*, pages 1–6. IEEE, 2020.

[25] Larysa Globa, Svitlana Sulima, Mariia Skulysh, Stanislav Dovgyi, and Oleksandr Stryzhak. Architecture and operation algorithms of mobile core network with virtualization. *Mobile computing*, pages 427–434, 2020.

[26] Fei Hu, Qi Hao, and Ke Bao. A survey on software-defined network and openflow: From concept to implementation. *IEEE Communications Surveys & Tutorials*, 16(4):2181–2206, 2014.

[27] Jon Matias, Jokin Garay, Nerea Toledo, Juanjo Unzilla, and Eduardo Jacob. Toward an sdn-enabled nfv architecture. *IEEE Communications Magazine*, 53(4):187–193, 2015.

[28] William Stallings. *SNMP, SNMPv2, and CMIP: The practical guide to network management*. Addison-Wesley Longman Publishing Co., Inc., 1993.

[29] Brian Hedstrom, Akshay Watwe, and Siddharth Sakthidharan. Protocol efficiencies of netconf versus snmp for configuration management functions. *University of Colorado, Master Thesis*, 2011.

[30] Thierry Pollet, Gerard Maas, Johan Marien, and Albert Wambecq. Telecom services delivery in a soa. In *20th International Conference on Advanced Information Networking and Applications-Volume 1 (AINA'06)*, volume 2, pages 5–pp. IEEE, 2006.

[31] Union Europea. Reglamento general de protección de datos. `https://europa.eu/youreurope/business/dealing-with-customers/data-protection/data-protection-gdpr/index_es.htm`, Visitada en Marzo 2024.

[32] Miguel Godinho de Matos and Idris Adjerid. Consumer consent and firm targeting after gdpr: The case of a large telecom provider. *Management Science*, 68(5):3330–3378, 2022.

[33] 3GPP. Ts29591 network exposure function southbound services. `https://www.etsi.org/deliver/etsi_ts/129500_129599/129591/16.01.00_60/ts_129591v160100p.pdf`, 2020.

[34] 3GPP. Ts23502 procedures for the 5g system. `https://www.etsi.org/deliver/etsi_ts/123500_123599/123502/16.07.00_60/ts_123502v160700p.pdf`, 2021.

[35] 3GPP. Ts29522 network exposure function northbound apis. `https://www.etsi.org/deliver/etsi_ts/129500_129599/129522/16.05.00_60/ts_129522v160500p.pdf`, 2020.

[36] Telecoms.com. European telcos once again push for fair contribution to network costs. `https://www.telecoms.com/public-policy/european-telcos-once-again-push-for-fair-contribution-to-network-costs`, 2023. Accessed: 2024-04-10.

[37] Jose Ordonez-Lucena and Felix Dsouza. Pathways towards network-as-a-service: the camara project. In *Proceedings of the ACM SIGCOMM Workshop on Network-Application Integration*, pages 53–59, 2022.

[38] 3GPP. Ts23203 policy and charging control architecture. `https://www.etsi.org/deliver/etsi_ts/123200_123299/123203/15.03.00_60/ts_123203v150300p.pdf`, 2018.

[39] Ministerio de Asuntos Económicos y Transformación Digital. Informe sobre supervisión en españa de normativa europea en materia de acceso a una internet abierta (neutralidad de la red). https://avancedigital.mineco.gob.es/banda-ancha/Informesneutralidadred/NN_informe_ESPANA_2019_28_07_20.pdf, Visitada en Abril 2024.

[40] Nada Salaheddin ELGHERIANI and Nuredin Ali Salem AHMED. Microservices vs. monolithic architectures [the differential structure between two architectures]. *MINAR International Journal of Applied Sciences and Technology*, 4(3):500–514, 2022.

[41] Alejandro Palmier Gómez. Repositorio del trabajo de fin de máster. https://github.com/alexpg67/trabajo_fin_master, 2024.

[42] Docker. Instalación de docker. https://docs.docker.com/engine/install/ubuntu/, Visitada en Abril 2024.

[43] 3GPP. Release 12, numbering, addressing and identification. Technical Report TS 23.003, 3rd Generation Partnership Project (3GPP), 2015. Disponible en: https://www.arib.or.jp/english/html/overview/doc/STD-T63V12_10/5_Appendix/Rel12/23/23003-c90.pdf.

[44] ITU. Mobile network codes (mnc) for the international identification plan for public networks and subscriptions. https://www.itu.int/dms_pub/itu-t/opb/sp/T-SP-E.212B-2018-PDF-E.pdf, 2018.

[45] Spring. Inicializador de un proyecto spring. https://start.spring.io/, Visitada en Abril 2024.

[46] Peter Bakondy. Mcc mnc json list. https://github.com/pbakondy/mcc-mnc-list/blob/master/mcc-mnc-list.json, 2023.

[47] Stack Overflow. Connect to docker using ip instead of localhost. https://stackoverflow.com/questions/58024165/connect-to-docker-using-ip-instead-of-localhost, 2020.

[48] Swagger UI. Repositorio oficial swagger ui. https://github.com/swagger-api/swagger-ui/tree/master, Visitada en Abril 2024.

[49] Dustin Rue. Elasticsearch container will crash under default docker for mac configuration. https://github.com/10up/wp-local-docker/issues/6, 2017.

[50] Minikube. Get started minikube. https://minikube.sigs.k8s.io/docs/start/, Visitada en Abril 2024.

[51] Kubernetes. Get started kubectl. https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/, Visitada en Abril 2024.

[52] Kompose. Instalación de kompose. https://kompose.io/installation/, Visitada en Abril 2024.

[53] Jenkins. Instalación de jenkins. https://www.jenkins.io/doc/book/installing/linux/, Visitada en Abril 2024.

[54] Postman. Instalación de newman. https://learning.postman.com/docs/collections/using-newman-cli/installing-running-newman/, Visitada en Abril 2024.

[55] Alejandro Palmier Gómez. Perfil de figma para el diseño de la aplicación sandbank. https://www.figma.com/design/Llm6gpPSr8psYknFGUtd85/Sandbank-App?node-id=0%3A1&t=oExYcc7ghKf9jsIt-1, Visitada en Abril 2024.

[56] Talent.com. Salario para ingeniero de telecomunicaciones en España. https://es.talent.com/salary?job=ingeniero+telecomunicaciones, 2023.

[57] Factorial. Calculadora de coste de un trabajador para una empresa, 2024. Accedido: 18-mayo-2024.

[58] Días Laborables. ¿Cuántos días laborables hay en el año 2024 en la Comunidad de Madrid? https://www.dias-laborables.es/cuantos_dias_laborables_en_ano_2024_Comunidad%20de%20Madrid.htm, 2024. Accessed: 2024-05-11.

Apéndice A

Aspectos éticos, económicos, sociales y ambientales

A.1. Introducción

En este apéndice se van a explicar algunos de los impactos de este trabajo de fin de máster en aspectos sociales, económicos, medioambientales y éticos.

A.2. Descripción de impactos relevantes relacionados con el proyecto

- **Impacto social**: La confianza y seguridad al usar aplicaciones del mundo digital es muy importante para los usuarios. Muchas personas de cierta edad siguen prefiriendo hacer ciertos procedimientos de manera presencial que a través de métodos en línea por miedo a ser estafados. Es por este motivo, que el proyecto *Open Gateway* puede tener un gran impacto en la sociedad. Este proyecto trae consigo nuevos casos de uso que van a permitir aumentar la seguridad de muchas aplicaciones. Esto lo van a conseguir a través de algunas API que se han desarrollado en este trabajo como *Sim Swap*, KYC y con otras muchas que se están estandarizado en *CAMARA* (*Number Verify*, *Credit Scoring*, *Tenure*, etc). En conclusión, las API que ofrece *Open Gateway* fomentan una sociedad más justa, detectando casos comúnes de fraude.

- **Impacto económico**: El principal impacto económico es el de abrir un nuevo mercado a las operadoras del sector de telecomunicaciones. Además, para el desarrollo de las API de *Open Gateway* se ha incluido una parte de monitorización del rendimiento y consumo de los servicios. Uno de los principales objetivos es el de intentar reducir el consumo y el coste asociado. Por otra parte, la solución se ha desarrollado en tecnologías de contenedores como *Docker* y *Kubernetes* que permiten el despliegue en cualquier entrono tanto de nube privada como de nube pública. Esto facilita que se despliegue en el entorno que menos coste suponga.

- **Impacto medioambiental**: Al igual que en el impacto económico, el desarrollo

de la solución en tecnología de contenedores permite reducir el impacto medioambiental de la solución. La razón es que con tecnologías como *Kubernetes*, los servicios pueden escalar en función de la demanda que reciban. Por lo tanto, en periodos en los que no hay muchas peticiones se podría reducir el número de recursos necesarios de manera automática. Esto supone un gran cambio con respecto a las soluciones monolíticas tradicionales en las que los servidores y recursos se mantienen siempre activos.

- **Impacto ético**: Para ofrecer API de *Open Gateway* que dan datos personales o sensibles de los clientes de las operadoras de telecomunicaciones, es muy importante seguir los marcos regulatorios de España, la Unión Europea o el país de donde sea la operadora. Uno de los marcos más relevantes a nivel de privacidad es el GDPR. Aunque no se ha entrado en excesivos detalles, el impacto de esta normativa en este proyecto se ha explicado en el capítulo 2.

A.3. Conclusiones

En conclusión, la exposición de API, no es solo un reto desde el punto de vista ingenieril sino que, es trascendente en otros ámbitos, influyendo e impactando en temas económicos, sociales, éticos o medioambientales.

Apéndice B

Presupuesto económico

En este anexo se va a realizar una estimación de los costes del proyecto. Para ello, se va a tener en cuenta el coste de personal y el coste de los materiales empleados. El coste de las licencias software no se va a tener en cuenta ya que todas las herramientas empleadas son de código libre (*opensource*). Sin embargo, si hiciese falta una solución que incluyese el soporte de las herramientas y un rendimiento más profesional, habría que usar las versiones de pago.

- **Personal**: Los costes de personal se van a estimar a partir del salario medio de un ingeniero de telecomunicaciones y el tiempo que se ha dedicado a realizar este proyecto. El salario bruto anual de un ingeniero de telecomunicaciones en España es 30.000 €[56]. De estos 30.000 de salario bruto que percibe el trabajador, realmente la empresa está pagando 39.645 teniendo en cuenta las cotizaciones obligatorias como por ejemplo, accidentes de trabajo y enfermedades profesionales [57]. Considerando que la jornada laboral son 8 horas diarias y que hay 251 días laborables al año [58] se obtiene un coste de $19,74$ €/hora.

  Si a continuación se considera que se han dedicado a la realización del trabajo 35 horas semanales durante 9 meses, se obtiene que el total de horas empleadas son 1260. Teniendo en cuenta el coste medio previamente calculado, se tiene que el coste de personal es $24.872,40$ €. Se reflejan los resultados en la tabla B.1.

	Coste horario (€)	**Horas**	**Total (€)**
Costes de personal	$19,74$	1260	**$24.872,40$**

Tabla B.1: Costes de personal.

- **Costes de recursos materiales**: En este apartado se van a calcular los costes de los recursos materiales empleados en el proyecto. Para ello se va a tener en cuenta únicamente el ordenador en el que se ha trabajado. Aunque realmente se puede realizar el trabajo en cualquier ordenador, si se quiere desplegar a la vez la solución de las API y la aplicación de antifraude, se necesita un ordenador potente de al menos 32 GB de RAM y a ser posible un procesador de alto

rendimiento como un *Intel i7* o *Ryzen 7*. Si la solución se despliega en la nube pública, no es necesario un ordenador tan potente pero habría que sumar el coste de *Google*, *Azure*, *AWS* o el proveedor pertinente. Se muestra en la tabla B.2 el coste del ordenador.

	Tiempo de vida (años)	**Uds.**	**Coste (€)**	**Amortización (€/mes)**	**Uso (meses)**	**Total (€)**
Ordenador	5	1	1300	21,67	9	195
TOTAL						**195**

Tabla B.2: Costes de recursos materiales.

Por lo tanto, si se tienen en cuenta todos los costes mencionados, se obtiene el coste final del proyecto como se puede ver en la tabla B.3

	Coste
Costes de personal	22.589, 64€
Costes de material	195€
Subtotal	22.784,64€
IVA	21,00 %
Total	**27.569,41€**

Tabla B.3: Costes totales.